RAILS

CHEAT SHEETS

A Complete Reference for Ruby on Rails

Routing · Controllers · Views & Forms · Active Record · Associations · Querying · Auth & Security · Hotwire · Active Job · Solid Queue · Cache · Cable · Active Storage · Testing & bin/ci · Kamal

78 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

01–06	Introduction & Setup
07–10	Routing
11–17	Controllers
18–24	Views & Forms
25–31	Models & Migrations

DATA & USERS

32–35	Associations
36–42	Querying
43–45	Active Model & Active Support
46–50	Authentication & Security
51–56	Assets & Hotwire

PRODUCTION

57–63	Jobs, Mail, Storage & Real-Time
64–69	Testing & CI
70–74	APIs & Advanced Topics
75–78	Deployment & Upgrades

RAILS

Table of Contents

78 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 06

- 01 Introduction & Philoso…
- 02 Installation & rails new
- 03 Directory Structure
- 04 Configuration & Enviro…
- 05 The rails Command
- 06 Generators & Scaffolds

ROUTING 07 – 10

- 07 Routing Basics
- 08 Resourceful Routes
- 09 Nested, Namespaced …
- 10 Advanced Routing

CONTROLLERS 11 – 17

- 11 Controller Basics
- 12 Params & Strong Para…
- 13 Rendering & Redirects
- 14 Action Callbacks
- 15 Sessions, Cookies & Fl…
- 16 Formats & Error Handli…
- 17 Concerns, Helpers & C…

VIEWS & FORMS 18 – 24

- 18 ERB Templates
- 19 Layouts & content_for
- 20 Partials & Collection R…
- 21 Built-in View Helpers
- 22 form_with Basics
- 23 Advanced Forms
- 24 Custom Helpers & Vie…

ACTIVE RECORD — MODELS & MIGRATIONS 25 – 31

- 25 Active Record Basics
- 26 Migrations — Basics
- 27 Migrations — Schema, …
- 28 Column Types & the A…
- 29 Validations
- 30 Model Callbacks
- 31 Scopes & Model Conc…

ASSOCIATIONS 32 – 35

- 32 belongs_to & has_ma…
- 33 has_one, :through & H…
- 34 Polymorphic, STI & Del…
- 35 Association Methods …

QUERYING 36 – 42

- 36 Query Basics
- 37 Conditions in Depth
- 38 Joins & Eager Loading
- 39 Calculations & Groupi…
- 40 Batches & Bulk Operat…
- 41 Transactions & Locking
- 42 Raw SQL & Multiple Da…

ACTIVE MODEL & ACTIVE SUPPORT 43 – 45

- 43 Active Model
- 44 Active Support Essenti…
- 45 Notifications & Structu…

AUTHENTICATION & SECURITY 46 – 50

- 46 Authentication Genera…
- 47 Authorization
- 48 Built-in Security
- 49 Credentials & Secrets
- 50 Encryption & Signed T…

ASSETS & HOTWIRE 51 – 56

- 51 Propshaft & Asset Deli…
- 52 JavaScript & CSS Opti…
- 53 Turbo Drive
- 54 Turbo Frames
- 55 Turbo Streams
- 56 Stimulus

JOBS, MAIL, STORAGE & REAL-TIME 57 – 63

- 57 Active Job Basics
- 58 Solid Queue & Job Co…
- 59 Action Mailer
- 60 Active Storage
- 61 Action Text
- 62 Action Cable & Solid C…
- 63 Caching & Solid Cache

TESTING & CI 64 – 69

- 64 Testing Basics & Fixtur…
- 65 Model & Integration Te…
- 66 System Tests
- 67 Testing Jobs, Mailers &…
- 68 RSpec Rails & Factory…
- 69 Local CI & Code Quality

APIS & ADVANCED TOPICS 70 – 74

- 70 API-Only Apps & JSON
- 71 Internationalization
- 72 Rack & Middleware
- 73 Engines & Railties
- 74 Debugging & Logging

DEPLOYMENT & UPGRADES 75 – 78

- 75 Production Configurati…
- 76 Dockerfile, Puma & Thr…
- 77 Deploying with Kamal
- 78 Upgrading Rails

Introduction & Philosophy

WHAT RAILS IS

Ruby on Rails is a full-stack web framework written in Ruby, extracted from Basecamp by David Heinemeier Hansson in 2004. It follows **MVC** (models, views, controllers) and favors **convention over configuration**: name things the Rails way and most wiring happens for you. Rails 8 aims to be a **One Person Framework**: database-backed jobs, cache and websockets, auth, and deployment ship in the box.

KEY FACTS

- Rails is a set of gems: `activerecord`, `actionpack`, `actionview`, `activejob`, `actionmailer`, `activestorage`, `actioncable`, `activesupport` and more
- **Omakase**: the default stack is curated (SQLite, Propshaft, Importmap, Hotwire, Solid Queue, Kamal)
- Current release line: **8.1** (8.1.4, Sept 2026). 8.2 is on edge
- Files: `.rb` (Ruby), `.html.erb` (templates), `.yml` (config)

HELLO, WORLD IN RAILS

```
rails new hello && cd hello
bin/rails generate controller Pages home
bin/rails server            # http://localhost:3000/pages/home
```

APP/CONTROLLERS/PAGES_CONTROLLER.RB

```
class PagesController < ApplicationController
  def home
    @greeting = "Hello, World"   # visible in the view
  end                            # implicit render: pages/home
end
```

APP/VIEWS/PAGES/HOME.HTML.ERB

```
<h1><%= @greeting %></h1>
```

CODE EXPLAINED

- → `rails new` — generates a full app with sensible defaults
- → `generate controller` — controller, view, route and test in one step
- → `@greeting` — instance variables flow from controller to view
- → No `render` call needed: Rails renders `pages/home.html.erb` by convention
- → `<%= %>` — ERB output tag, HTML-escaped automatically

THE RAILS WAY

- Configuring table names and paths by hand → follow naming conventions (`Post` model ↔ `posts` table)
- Adding a gem for every need → check the built-in framework first (jobs, cache, auth, storage)
- Fighting the defaults on day one → ship with the omakase stack, swap pieces when you hit a real limit

REQUEST CYCLE

STEP	COMPONENT
Router	`config/routes.rb` maps URL + verb to `controller#action`
Controller	Action Pack: reads params, talks to models
Model	Active Record: one class per table
View	Action View: ERB renders HTML / JSON
Response	Rack hands it back to Puma → browser

RECENT VERSIONS

VERSION	HEADLINE
7.1 (2023)	Dockerfile by default, `normalizes`, `generates_token_for`
7.2 (2024)	`rate_limit`, dev container, browser guard, YJIT on
8.0 (2024)	Solid Queue/Cache/Cable, auth generator, Kamal 2, Thruster, Propshaft
8.1 (2025)	Job continuations, `Rails.event`, `bin/ci`, Markdown rendering
8.2 (edge)	Herb ERB compiler, `Rails.app.creds`, `has_json`

⚠ COMMON MISTAKES

- Learning Rails before Ruby: blocks, symbols and hashes are everywhere, so confusion about Rails is often confusion about Ruby
- Treating conventions as optional: a `Person` model with a `people_table` breaks every helper that guesses names
- Following tutorials for Rails 5/6: Webpacker, Turbolinks and `rails-ujs` are gone from new apps

✓ TIPS & BEST PRACTICES

- Read the official Rails Guides for your exact version; they are versioned at guides.rubyonrails.org
- Run `bin/rails about` to see the Rails, Ruby and gem versions an app actually uses
- Each minor release gets about 1 year of bug fixes and 2 of security fixes; plan upgrades accordingly

QUICK SUMMARY

- `MVC` — router → controller → model → view
- `conventions` — names drive wiring
- `omakase` — curated default stack
- `8.1` — current release line

Installation & rails new

GETTING A NEW APP RUNNING

Install a recent Ruby with a version manager, then `gem install rails`. `rails new` generates the app, runs `bundle install`, and sets up git. Its flags pick the database, CSS and JavaScript approach up front; changing them later is possible but tedious, so choose deliberately.

BEFORE YOU START

- Rails 8.1 needs **Ruby 3.2+**; use the newest Ruby you can (3.4 / 4.0)
- SQLite is the default database and is production-ready in Rails 8
- `rails new` runs `bundle install` and `git init` unless you pass `--skip-bundle` / `--skip-git`
- `bin/setup` installs gems, prepares the database and starts the server: the one command new teammates run

INSTALL AND CREATE

```
mise use -g ruby@3.4          # or rbenv / asdf
gem install rails             # latest 8.1.x
rails -v                      # Rails 8.1.4

rails new blog                              # SQLite, Importmap, Propshaft
rails new shop --database=postgresql --css=tailwind
rails new api_app --api                     # JSON-only, no views/assets
rails new tiny --minimal                    # skip most frameworks
rails new blog -d mysql -j esbuild -c bootstrap

cd blog && bin/setup                        # gems + db:prepare + server
bin/dev                                     # server + CSS/JS watchers
```

CODE EXPLAINED

- → `--database` / `-d` — sqlite3 (default), postgresql, mysql, trilogy, mariadb-*
- → `--css` / `-c` — tailwind, bootstrap, bulma, postcss, sass
- → `--javascript` / `-j` — importmap (default), bun, esbuild, rollup, webpack
- → `--api` — `ActionController::API`, no views, cookies or asset pipeline
- → `_8.0.3_` — RubyGems syntax to run a specific installed version

PIN A RAILS VERSION

```
rails _8.0.3_ new legacy_app          # use an installed older version
```

THE RAILS WAY

- `sudo gem install` on system Ruby → a version manager and a user-owned Ruby
- Adding Webpacker/Node by reflex → start with Importmap; add a bundler (`-j esbuild`) only for npm-heavy apps
- Writing a README of setup steps → keep `bin/setup` current so one command works

COMMON RAILS NEW FLAGS

FLAG	EFFECT
`--skip-test`	No Minitest (use with RSpec)
`--skip-jbuilder`	No Jbuilder JSON templates
`--skip-action-mailbox`	Drop inbound email framework
`--skip-kamal`	No `config/deploy.yml`
`--skip-solid`	No Solid Queue/Cache/Cable
`--devcontainer`	Generate `.devcontainer/` config
`--main`	Point Gemfile at Rails main (edge)

GENERATED SCRIPTS

SCRIPT	USE
`bin/rails`	Every Rails command
`bin/setup`	First-time / after-pull setup
`bin/dev`	Server plus asset watchers
`bin/ci`	Run the local CI steps (8.1)
`bin/jobs`	Start Solid Queue workers
`bin/kamal`	Deploy

⚠ COMMON MISTAKES

- Running `rails new` inside another app folder: the new app picks up the outer Gemfile and fails to bundle
- Choosing `--api` for an app that will later need sessions or HTML forms; adding them back is manual work
- Using the global `rails` binary inside a project; always use `bin/rails` so the app's locked version runs

✓ TIPS & BEST PRACTICES

- Put your favorite flags in `~/.railsrc` (one per line) to apply them to every `rails new`
- Use `rails new --help` to see the exact flags your installed version supports
- Commit right after `rails new` so generator diffs are easy to review later

QUICK SUMMARY

- `gem install rails` — install
- `rails new app -d -c -j` — create with choices
- `bin/setup` — one-command setup
- `bin/dev` — run with watchers

Directory Structure

WHERE THINGS LIVE

Every Rails app has the same layout, which is why you can open any Rails project and find your way around. Code under `app/` is **autoloaded by Zeitwerk**: file paths must match constant names, and you never write `require` for your own classes.

LAYOUT RULES

- `app/models/admin/user.rb` must define `Admin::User`
- Any new folder under `app/` (e.g. `app/services`) is autoloaded automatically
- `lib/` is autoloaded only for subfolders listed in `config.autoload_lib(ignore: %w[assets tasks])`
- `storage/` holds SQLite databases and local Active Storage files

A NEW APP, TRIMMED

```
blog/
├── app/
│   ├── controllers/      application_controller.rb, concerns/
│   ├── models/           application_record.rb, concerns/
│   ├── views/            layouts/application.html.erb
│   ├── helpers/  jobs/  mailers/  channels/
│   ├── assets/           stylesheets/, images/
│   └── javascript/       controllers/ (Stimulus)
├── bin/                  rails, setup, dev, jobs, kamal, ci
├── config/               routes.rb, database.yml, environments/,
│                         initializers/, credentials.yml.enc, deploy.yml
├── db/                   migrate/, schema.rb, seeds.rb
├── lib/                  tasks/ (rake tasks)
├── public/               404.html, robots.txt (served as-is)
├── script/               one-off scripts (8.0+)
├── storage/  log/  tmp/
├── test/                 models/, controllers/, system/, fixtures/
└── Gemfile  Dockerfile  config.ru
```

CODE EXPLAINED

- → `app/` — your application code, autoloaded and reloaded in development
- → `config/` — routes, environments, initializers, credentials, deploy config
- → `db/` — migrations and the generated `schema.rb`
- → `public/` — static files served before Rails sees the request
- → `script/` — one-off and maintenance scripts (`g script` generator, 8.0+)

THE RAILS WAY

- `require_relative "../models/user"` in app code → let Zeitwerk autoload by file name
- A grab-bag `app/lib` or `utils.rb` → name folders after what they hold (`app/services`, `app/queries`)
- Business logic in `lib/` → keep app logic in `app/`; reserve `lib/` for app-independent code

NAMING CONVENTIONS

FILE	CONSTANT
app/models/post.rb	Post
app/models/line_item.rb	LineItem
app/controllers/posts_controller.rb	PostsController
app/controllers/admin/users_controller.rb	Admin::UsersController
app/jobs/send_digest_job.rb	SendDigestJob
app/views/posts/index.html.erb	view for PostsController#index

ZEITWERK COMMANDS

COMMAND	USE
bin/rails zeitwerk:check	Verify every file matches its constant
config.autoload_paths	Add extra root dirs (rarely needed)
inflect.acronym "API"	Teach `api_client.rb` → `APIClient`
Rails.autoloaders.main	Inspect the main loader
reload!	Reload code in the console

⚠ COMMON MISTAKES

- Naming `app/services/pdf_maker.rb` but defining `PDFMaker`: Zeitwerk expects `PdfMaker` unless you add an acronym inflection
- Putting two classes in one file; the second can't be autoloaded by name
- Editing files in `public/` and expecting asset fingerprinting; only `app/assets` is processed

✓ TIPS & BEST PRACTICES

- Run `bin/rails zeitwerk:check` in CI to catch naming mismatches before production eager loading does
- Keep `concerns/` for genuinely shared modules, not as a dumping ground
- Look at `config/application.rb` first in an unfamiliar app: it shows load_defaults and custom settings

QUICK SUMMARY

- `app/` — autoloaded code
- `config/` — settings and routes
- `db/` — migrations and schema
- `file path = constant` — Zeitwerk rule

Configuration & Environments

HOW RAILS IS CONFIGURED

Settings load in layers: `config/application.rb` for everything, then `config/environments/<env>.rb` for development, test or production, then every file in `config/initializers/`. `config.load_defaults` opts the app into each version's new framework defaults.

ENVIRONMENT FACTS

- `Rails.env` comes from `RAILS_ENV` (or `RACK_ENV`), defaulting to development
- `Rails.env.production?` — inquiry methods for each environment name
- Initializers run alphabetically, after the framework loads
- Custom YAML in `config/*.yml` loads with `config_for(:name)`, keyed by environment

CONFIG/APPLICATION.RB

```ruby
module Blog
  class Application < Rails::Application
    config.load_defaults 8.1
    config.time_zone = "Central Time (US & Canada)"
    config.autoload_lib(ignore: %w[assets tasks])
    config.x.payments.retries = 3          # custom namespace
    config.stripe = config_for(:stripe)    # config/stripe.yml
  end
end
```

CODE EXPLAINED

- → `load_defaults 8.1` — turn on every new default up to 8.1
- → `config.x` — a safe namespace for your own settings
- → `config_for(:stripe)` — reads `config/stripe.yml` for the current env, ERB allowed
- → `configure do` — environment file overrides of application.rb
- → Rails.configuration — read settings anywhere at runtime

CONFIG/ENVIRONMENTS/DEVELOPMENT.RB (EXCERPT)

```ruby
Rails.application.configure do
  config.enable_reloading = true
  config.action_mailer.delivery_method = :letter_opener
end
```

THE RAILS WAY

- `if ENV["RAILS_ENV"] == "production"` → `Rails.env.production?`
- Constants sprinkled in initializers → `config.x.feature.setting` or `config_for`
- Hard-coding secrets in config files → credentials or ENV (sheet 49)

READING CONFIG

```ruby
Rails.env                                  # "development"
Rails.env.production?                      # false
Rails.configuration.x.payments.retries     # 3
Rails.configuration.stripe[:public_key]    # from stripe.yml
```

ENVIRONMENTS

ENV	BEHAVIOR
`development`	Reloading on, verbose errors, local services
`test`	Eager load in CI, transactional tests, no real mail
`production`	Eager load, caching on, SSL, logs to STDOUT

USEFUL SETTINGS

SETTING	PURPOSE
`config.time_zone`	Default zone for `Time.zone`
`config.i18n.default_locale`	Default language
`config.active_job.queue_adapter`	Job backend
`config.cache_store`	Cache backend
`config.hosts`	Allowed hostnames
`config.generators`	Generator defaults (e.g. RSpec)

⚠ COMMON MISTAKES

- Bumping the Rails gem but leaving `load_defaults` at an old version; the app silently keeps old behavior
- Reading `ENV` at class-load time in models; the value is frozen at boot and hard to test
- Adding a custom environment (`staging`) without copying production's environment file and database entry

✓ TIPS & BEST PRACTICES

- Prefer one production-like environment configured by ENV over many custom environments
- Use `bin/rails runner 'p Rails.configuration.x'` to check settings without starting the server
- Name initializers with prefixes (`01_`) only when order truly matters

QUICK SUMMARY

- `application.rb` — app-wide
- `environments/*.rb` — per-env overrides
- `initializers/` — boot-time setup
- `load_defaults` — framework defaults by version

The rails Command

YOUR MAIN TOOL

`bin/rails` is the entry point for running the server, opening a console, inspecting routes, running database tasks and generators. Rake tasks are merged in, so `bin/rails db:migrate` and `bin/rails test` both work. Use `bin/rails -T` to list everything available.

COMMAND NOTES

- Short aliases: `s` server, `c` console, `g` generate, `d` destroy, `r` runner, `t` test, `db` dbconsole
- `bin/dev` runs `Procfile.dev` (server + Tailwind/JS watchers) via Foreman
- Pass an environment with `-e production` or `RAILS_ENV=production`
- Inside the console, `reload!` picks up code changes

EVERYDAY COMMANDS

```
bin/rails server -p 4000 -b 0.0.0.0   # bind for containers / LAN
bin/rails console                     # IRB with the app loaded
bin/rails console --sandbox           # roll back all changes on exit
bin/rails runner "puts User.count"    # run Ruby in app context
bin/rails runner script/backfill.rb
bin/rails routes -g posts             # grep routes
bin/rails routes -c Admin::Users      # by controller
bin/rails about                       # versions, env, middleware
bin/rails notes                       # TODO / FIXME / OPTIMIZE
bin/rails dbconsole                   # sqlite3 / psql / mysql
bin/rails -T db                       # list db:* tasks
bin/rails restart                     # touch tmp/restart.txt
```

CODE EXPLAINED

- → `--sandbox` — wraps the session in a transaction, rolled back on exit
- → `runner` — scripts, cron jobs and quick checks with full app access
- → `routes -g` — filter by name, path or controller text
- → `app` / `helper` — console objects for route and view helpers
- → `-T` — list tasks with descriptions, optional filter

CONSOLE HELPERS

```
app.posts_path                  # "/posts"  (route helpers)
helper.number_to_currency(5)    # "$5.00"   (view helpers)
reload!                         # reload app code
```

THE RAILS WAY

- Copy-pasting snippets into `rails c` for recurring jobs → a rake task or `script/` file
- Reading `routes.rb` to find a URL → `bin/rails routes -g name`
- Restarting the console after edits → `reload!`

COMMAND REFERENCE

COMMAND	WHAT IT DOES
`bin/rails new`	Create an app (global `rails` only)
`bin/rails generate / destroy`	Create / remove generated files
`bin/rails db:migrate`	Run pending migrations
`bin/rails test`	Run Minitest suite
`bin/rails credentials:edit`	Edit encrypted secrets
`bin/rails middleware`	List the Rack stack
`bin/rails initializers`	List initializers in run order
`bin/rails stats`	Lines of code by type
`bin/rails log:clear`	Truncate log files
`bin/rails tmp:clear`	Clear cache, sockets, screenshots

⚠ COMMON MISTAKES

- Running `--sandbox` against production data and assuming jobs or emails also roll back; external side effects still happen
- Using `rails server` in production instead of the Puma/Thruster command from the Dockerfile
- Forgetting `-b 0.0.0.0` inside Docker, so the server is unreachable from the host

✓ TIPS & BEST PRACTICES

- Add `IRB.conf[:USE_AUTOCOMPLETE] = false` to `.irbrc` if console autocomplete is slow on big apps
- Production consoles show the environment in the prompt (7.1+); check it before running writes
- Use `bin/rails runner -e test` to debug test-only data setups

QUICK SUMMARY

- `s / c / g / r` — server, console, generate, runner
- `routes -g` — find routes
- `--sandbox` — safe console
- `-T` — list all tasks

Sheet 06

Generators & Scaffolds

scaffold · model · controller · resource · migration · destroy · --skip-* · custom generator templates

CODE THAT WRITES CODE

Generators create files that follow Rails conventions: models with migrations, controllers with views, and full **scaffolds** with CRUD actions, forms and tests. Field lists use `name:type` pairs. Every generator has a matching `destroy` that removes what it created.

GENERATOR NOTES

- Field types: `string`, `text`, `integer`, `decimal`, `boolean`, `date`, `datetime`, `references`, `attachment`, `rich_text`
- Modifiers: `name:string:index`, `email:string:uniq`, `price:decimal{10,2}`
- `--pretend` (`-p`) shows what would be created without writing files
- Scaffolds are a starting point; trim unused actions and views

COMMON GENERATORS

```
bin/rails g model Post title:string body:text published_at:datetime
bin/rails g model Comment post:references body:text
bin/rails g controller Pages home about --skip-routes
bin/rails g resource Tag name:string:uniq      # model + controller + routes
bin/rails g scaffold Product name price:decimal{10,2} active:boolean
bin/rails g migration AddSlugToPosts slug:string:index
bin/rails g migration CreateJoinTablePostTag post tag
bin/rails g authentication                     # sessions + users (8.0)
bin/rails g job SendDigest
bin/rails g mailer User welcome
bin/rails g stimulus dropdown

bin/rails destroy scaffold Product             # undo the files
bin/rails g scaffold Product name -p           # dry run
```

CODE EXPLAINED

- → `post:references` — `post_id` column, index, FK and `belongs_to :post`
- → `resource` — model, empty controller and `resources :tags`, no views
- → `scaffold` — the full CRUD set: routes, 7 actions, views, JSON, tests
- → `AddXToY` / `RemoveXFromY` — named migrations fill in `add_column` for you
- → `config.generators` — default flags for every generator

CONFIG/APPLICATION.RB

```
config.generators do |g|
  g.test_framework :rspec
  g.helper false
  g.stylesheets false
end
```

THE RAILS WAY

- Hand-writing a model, migration and test file → `bin/rails g model` and edit the result
- Deleting generated files one by one → `bin/rails destroy` with the same arguments
- Keeping all 7 scaffold actions "just in case" → delete what the feature doesn't need

WHAT EACH GENERATOR CREATES

GENERATOR	FILES
`model`	model, migration, test, fixture
`controller`	controller, views, helper, routes, test
`resource`	model + migration + controller + `resources` route
`scaffold`	resource + 7 actions, views, Jbuilder, system tests
`migration`	one timestamped migration file
`authentication`	User, Session, Current, controllers, mailer

CUSTOMIZING

APPROACH	HOW
`Override templates`	Copy to `lib/templates/erb/scaffold/`
`Custom generator`	`bin/rails g generator service`
`Skip pieces`	`--skip-test`, `--no-helper`, `--skip-routes`
`Force overwrite`	`--force` (`-f`)

⚠ COMMON MISTAKES

- Generating a scaffold and shipping it unchanged: it permits every field and exposes JSON endpoints you may not want
- Running a generator twice and choosing "overwrite" on edited files, losing your changes
- Forgetting `bin/rails db:migrate` after `g model`, then seeing "pending migrations" errors

✓ TIPS & BEST PRACTICES

- Use `-p` first when you aren't sure what a generator will touch
- Override scaffold templates in `lib/templates` so new scaffolds match your team's style
- Reach for `resource` over `scaffold` when you will write the views yourself

QUICK SUMMARY

- `g model` — model + migration
- `g resource` — model + controller + route
- `g scaffold` — full CRUD
- `destroy` — undo a generator

Sheet 07

Routing Basics

config/routes.rb · get / post · to: "posts#show" · root · as: · _path vs _url helpers · bin/rails routes

MAPPING URLS TO ACTIONS

`config/routes.rb` connects an HTTP verb and path to a `controller#action`. Each named route also generates **URL helpers**: `post_path(@post)` returns `"/posts/1"` and `post_url(@post)` returns the full URL. Routes match top to bottom, so the first match wins.

ROUTING FACTS

- Verbs: `get`, `post`, `patch`, `put`, `delete`, plus `match via: [...]`
- Dynamic segments like `:id` arrive in `params[:id]` as strings
- `as:` names the route; resource routes are named automatically
- Use `_path` in views and redirects, `_url` in mailers and external links

CONFIG/ROUTES.RB

```
Rails.application.routes.draw do
  root "pages#home"                          # GET /

  get  "about",       to: "pages#about"      # about_path
  get  "posts/:id",   to: "posts#show", as: :post
  post "posts",       to: "posts#create"
  get  "search",      to: "search#index", as: :search
  get  "up" => "rails/health#show", as: :rails_health_check
end
```

CODE EXPLAINED

- → `to: "posts#show"` — `PostsController#show`
- → `as: :post` — creates `post_path` and `post_url`
- → Passing a model (`post_path(post)`) calls `post.to_param` (the id)
- → Extra hash keys become the query string
- → `/up` — the generated health check (returns 200 when the app boots)

USING THE HELPERS

```
<%= link_to "About", about_path %>           <!-- /about -->
<%= link_to post.title, post_path(post) %>   <!-- /posts/42 -->
<%= link_to "Find", search_path(q: "rails") %> <!-- /search?q=rails -->
<%= root_url %>                               <!-- http://localhost:3000/ -->
```

THE RAILS WAY

- Hard-coded `href="/posts/#{post.id}"` → `post_path(post)`
- A custom `get` for each CRUD action → `resources :posts` (sheet 08)
- `redirect_to "/posts"` → `redirect_to posts_path`

BIN/RAILS ROUTES OUTPUT

PREFIX	VERB · URI → CONTROLLER#ACTION
root	GET / → pages#home
about	GET /about(.:format) → pages#about
post	GET /posts/:id(.:format) → posts#show
posts	POST /posts(.:format) → posts#create
search	GET /search(.:format) → search#index

HELPER FORMS

HELPER	RETURNS
`post_path(1)`	`"/posts/1"`
`post_url(1)`	`"http://host/posts/1"`
`post_path(1, format: :json)`	`"/posts/1.json"`
`post_path(1, anchor: "c")`	`"/posts/1#c"`
`url_for(post)`	polymorphic: `/posts/1`

⚠ COMMON MISTAKES

- Putting a catch-all `get "*path"` above other routes; routes match in order so it swallows everything after it
- Using `_path` in mailer views; emails need absolute `_url` helpers and `default_url_options[:host]`
- Expecting `params[:id]` to be an Integer; route params are always strings

✓ TIPS & BEST PRACTICES

- Run `bin/rails routes -g post` whenever you forget a helper name
- Visit `/rails/info/routes` in development for a searchable route table
- Set `Rails.application.routes.default_url_options[:host]` once for jobs and mailers

QUICK SUMMARY

- `root` — the home page
- `get "x", to: "c#a"` — a single route
- `as:` — names the helper
- `_path` / `_url` — relative / absolute

Sheet 08

Resourceful Routes

resources · resource (singular) · the 7 actions · only: / except: · member · collection · param:

REST IN ONE LINE

`resources :posts` creates seven routes that map HTTP verbs to the standard CRUD actions, with named helpers for each. Use `only:` or `except:` to keep just the actions you implement. A **singular** `resource` is for things a user has exactly one of, like a profile, so there is no `:id` in the path.

RESOURCE NOTES

- The 7 actions: `index`, `show`, `new`, `create`, `edit`, `update`, `destroy`
- `PATCH` and `PUT` both route to `update`
- `member` routes act on one record (has `:id`); `collection` routes act on the set
- `param: :slug` renames `:id` in the path and in `params`

CONFIG/ROUTES.RB

```ruby
resources :posts                              # all 7
resources :photos, only: [:index, :show]
resources :tags,   except: :destroy
resource  :profile                            # singular, no index, no :id
resources :articles, param: :slug             # /articles/:slug

resources :posts do
  member do
    patch :publish                            # /posts/:id/publish
  end
  collection do
    get :drafts                               # /posts/drafts
  end
end
resources :comments, path: "notes"            # URL /notes, helpers comments_*
```

CODE EXPLAINED

→ `only:` / `except:` — avoid routes to actions that don't exist

→ `resource :profile` — `/profile`, `profile_path`, still uses `ProfilesController`

→ `member` — `publish_post_path(post)`

→ `collection` — `drafts_posts_path`

→ `path:` — change the URL without renaming controllers or helpers

THE RAILS WAY

- `post "posts/:id/publish"` written by hand → `member { patch :publish }`
- Custom actions piling up on one controller → a new resource (`resource :publication`) with standard actions
- `resources :posts` when only 2 actions exist → add `only:`

THE SEVEN ROUTES FOR RESOURCES :POSTS

VERB + PATH	ACTION · HELPER
`GET /posts`	`index · posts_path`
`GET /posts/new`	`new · new_post_path`
`POST /posts`	`create · posts_path`
`GET /posts/:id`	`show · post_path(post)`
`GET /posts/:id/edit`	`edit · edit_post_path(post)`
`PATCH/PUT /posts/:id`	`update · post_path(post)`
`DELETE /posts/:id`	`destroy · post_path(post)`

SINGULAR RESOURCE :PROFILE

VERB + PATH	ACTION
`GET /profile`	`show`
`GET /profile/new · /profile/edit`	`new · edit`
`POST · PATCH · DELETE /profile`	`create · update · destroy`

OPTIONS

OPTION	EFFECT
`controller:`	Use a different controller
`as:`	Rename helpers
`shallow: true`	Shorter nested paths (sheet 09)
`format: false`	Disallow `.json` suffix

⚠ COMMON MISTAKES

- Linking to `destroy` with a plain `link_to`; it sends GET. Use `button_to ... method: :delete`
- Adding `get :drafts` without `collection`, which produces `/posts/:post_id/drafts` inside a block
- Using `resources :profile` (plural) for a singleton and then needing a fake `:id`

✓ TIPS & BEST PRACTICES

- Prefer adding a new controller over adding a 4th custom action: "every action is one of the seven"
- Use `param: :slug` together with `to_param` in the model for pretty URLs
- Check helper names with `bin/rails routes -c posts`

QUICK SUMMARY

- `resources` — 7 CRUD routes
- `resource` — singular, no id
- `member` / `collection` — extra actions
- `only:` — keep it tight

Sheet 09

Nested, Namespaced & Scoped Routes

nested resources · shallow: true · namespace · scope · module: · concern / concerns: · path:

ORGANIZING ROUTES

Nesting expresses ownership in the URL (`/posts/1/comments`). **Namespaces** group controllers in a module and URL prefix, like an admin area. `scope` lets you change only the URL prefix, only the module, or only the helper name, independently. **Concerns** reuse route blocks.

RULES OF THUMB

- Never nest more than one level deep
- `shallow: true` nests only the actions that need the parent (`index` , `new` , `create`)
- `namespace :admin` = URL `/admin` , module `Admin::` , helpers `admin_`
- Nested params arrive as `params[:post_id]` and `params[:id]`

CONFIG/ROUTES.RB

```
resources :posts do
  resources :comments, shallow: true
end

namespace :admin do
  resources :users                  # Admin::UsersController, /admin/users
  root "dashboard#show"             # admin_root_path
end

scope "/app" do                     # URL only
  resources :reports                # ReportsController, /app/reports
end
scope module: "api" do              # module only
  resources :tokens                 # Api::TokensController, /tokens
end
scope "(:locale)", locale: /en|es/ do
  resources :products               # /es/products and /products
end

concern :commentable do
  resources :comments, only: [:index, :create]
end
resources :articles, concerns: :commentable
resources :videos,   concerns: :commentable
```

CODE EXPLAINED

→ Shallow: `post_comments_path(post)` for index/create, `comment_path(c)` for the rest

→ `namespace` — controllers live in `app/controllers/admin/`

→ `scope "(:locale)"` — optional segment with a constraint

→ `concern` — share nested routes across resources

→ `[@post, Comment.new]` — polymorphic nested form URL

NESTED HELPERS

```
<%= link_to "Comments", post_comments_path(@post) %>
<%= form_with model: [@post, Comment.new] do |f| %> ... <% end %>
```

THE RAILS WAY

- `/posts/1/comments/5/likes/2` → shallow nesting, max one level
- An admin area with `if current_user.admin?` in every action → `namespace :admin` plus a base `Admin::BaseController`
- Duplicated nested blocks → a routing `concern`

NAMESPACE VS SCOPE

DECLARATION	URL · MODULE · HELPER
`namespace :admin`	`/admin/users` · `Admin::` · `admin_users_path`
`scope "/admin"`	`/admin/users` · none · `users_path`
`scope module: "admin"`	`/users` · `Admin::` · `users_path`
`scope as: "admin"`	`/users` · none · `admin_users_path`

SHALLOW COMMENT ROUTES

PATH	ACTION
`/posts/:post_id/comments`	`index`, `create`
`/posts/:post_id/comments/new`	`new`
`/comments/:id`	`show`, `update`, `destroy`
`/comments/:id/edit`	`edit`

⚠ COMMON MISTAKES

- Deep nesting that forces every helper to take three objects: `post_comment_like_path(post, comment, like)`
- Namespacing routes without moving controllers into the matching folder; Zeitwerk raises `uninitialized constant Admin::UsersController`
- Forgetting the parent in nested forms: `form_with model: @comment` posts to `/comments`

✓ TIPS & BEST PRACTICES

- Give namespaced controllers a shared parent (`Admin::BaseController`) for auth filters
- Use `scope "(:locale)"` for i18n URLs without duplicating routes
- Use `bin/rails routes -g admin` to confirm what a namespace produced

QUICK SUMMARY

- `nested` — ownership in URL
- `shallow: true` — short member paths
- `namespace` — URL + module + helper
- `scope` — pick which to change

Advanced Routing

dynamic segments · globbing *path · constraints · defaults · redirect() · direct · resolve · mount

BEYOND RESOURCES

Rails routing can match optional and glob segments, restrict routes with regex or request-based **constraints**, redirect without a controller, mount Rack apps and engines, and define custom URL helpers with `direct` and `resolve`. Large route files can be split with `draw`.

ADVANCED NOTES

- Optional segments go in parentheses: `(/:page)`
- Constraints can be a hash, regex, lambda or an object with `matches?(request)`
- `redirect` returns 301 by default; pass `status: 302` for temporary
- `draw(:admin)` loads `config/routes/admin.rb`

CONFIG/ROUTES.RB

```ruby
get "photos/:id", to: "photos#show", constraints: { id: /\d+/ }
get "archive/:year(/:month)", to: "archive#show"      # optional segment
get "docs/*path", to: "docs#show", format: false      # glob: "a/b/c"
get "feed", to: "posts#index", defaults: { format: :rss }

get "/old-blog", to: redirect("/blog")
get "/stories/:name", to: redirect("/articles/%{name}", status: 302)

constraints subdomain: "api" do
  resources :widgets
end
constraints ->(req) { req.session[:admin] } do
  mount MissionControl::Jobs::Engine, at: "/jobs"
end

direct(:docs) { "https://guides.rubyonrails.org" }    # docs_url
resolve("Basket") { [:basket] }    # form_with model: @basket → /basket

draw(:admin)                                          # config/routes/admin.rb
match "*unmatched", to: "errors#not_found", via: :all  # keep last
```

CODE EXPLAINED

→ `constraints: { id: /\d+/ }` — `/photos/abc` won't match

→ `*path` — captures the rest of the URL into `params[:path]`

→ `%{name}` — reuse a captured segment in a redirect

→ `mount` — attach a Rack app or engine at a path

→ `resolve` — teach polymorphic helpers about a singular resource

THE RAILS WAY

- A controller action that only redirects → `to: redirect(...)` in routes
- Checking `request.subdomain` in every action → a routing `constraints subdomain:` block
- One 800-line `routes.rb` → split areas with `draw(:name)`

CONSTRAINT TYPES

FORM	EXAMPLE
Segment regex	`constraints: { id: /\d+/ }`
Request attribute	`constraints: { subdomain: "api" }`
Lambda	`constraints: ->(r) { r.format.json? }`
Object	class with `self.matches?(request)`
Format	`format: false` or `format: :json`

MOUNT TARGETS

MOUNT	EXAMPLE
Engine	`mount Blorgh::Engine, at: "/blog"`
Rack app	`mount Sidekiq::Web => "/sidekiq"`
ActionCable	`mount ActionCable.server => "/cable"`
Lambda	`mount ->(env) { [200, {}, ["ok"]] }, at: "/ping"`

⚠ COMMON MISTAKES

- Placing the catch-all `match "*unmatched"` before engine mounts; it steals their requests
- Forgetting regex anchors don't apply: segment constraints are already anchored, so `^` and `$` raise errors
- Using a constraint lambda for authorization alone; still check permissions in the controller

✓ TIPS & BEST PRACTICES

- Use `format: false` on glob routes so `file.pdf` stays part of the path
- Name redirects (`as:`) if you link to them from views
- Wrap admin-only mounts in an authenticated constraint, e.g. `authenticated` from your auth layer

QUICK SUMMARY

- `constraints` — restrict matches
- `*path` — glob the rest
- `redirect()` — no controller needed
- `mount` / `draw` — engines and split files

Controller Basics

ApplicationController · actions · instance vars to views · implicit render · request · response · ActionController::Base vs API

WHAT A CONTROLLER DOES

A controller is a Ruby class whose public methods are **actions**. Rails creates a new instance per request, runs the action, then renders a response. Instance variables set in the action are available in the view. If the action doesn't render or redirect, Rails renders the template matching the action name.

CONTROLLER FACTS

- Name: plural resource + `Controller` (`PostsController`) in `posts_controller.rb`
- All controllers inherit from `ApplicationController` , which inherits `ActionController::Base`
- Only **public** methods are actions; put helpers under `private`
- Each action must respond exactly once: render, redirect or head

APP/CONTROLLERS/POSTS_CONTROLLER.RB

```
class PostsController < ApplicationController
  before_action :set_post, only: %i[show edit update destroy]

  def index
    @posts = Post.order(created_at: :desc)   # renders posts/index
  end

  def show; end                              # @post from set_post

  def create
    @post = Post.new(post_params)
    if @post.save
      redirect_to @post, notice: "Post created"
    else
      render :new, status: :unprocessable_entity
    end
  end

  private

  def set_post
    @post = Post.find(params.expect(:id))
  end

  def post_params = params.expect(post: [:title, :body])
end
```

CODE EXPLAINED

- → `index` — no render call, so `app/views/posts/index.html.erb` renders
- → `redirect_to @post` — polymorphic, goes to `post_path(@post)`
- → `render :new, status: :unprocessable_entity` — re-show the form with errors (422)
- → `private` — keeps helper methods from becoming routable actions
- → `params.expect` — Rails 8 strong params (sheet 12)

THE RAILS WAY

- Database queries inside views → load data in the action into instance variables
- Fat actions with business rules → move logic into models or plain Ruby objects
- A dozen instance variables per action → pass one object or use a presenter

REQUEST & RESPONSE

```
request.path         # "/posts/1"     request.remote_ip
request.format       # :html / :json  request.get?
request.headers["User-Agent"]
response.status      # 200            response.headers["X-Id"] = "1"
```

BASE CLASSES

CLASS	USE FOR
`ActionController::Base`	HTML apps: views, cookies, flash, CSRF
`ActionController::API`	JSON APIs: lean middleware, no views
`ApplicationController`	Your app-wide filters and helpers
`Rails::HealthController`	`/up` health check

IMPLICIT BEHAVIOR

SITUATION	RAILS DOES
`No render / redirect`	Render `controller/action` template
`Template missing`	HTML: `MissingExactTemplate` (406); API: 204
`` `RecordNotFound` raised ``	404 page in production
`Format not handled`	`ActionController::UnknownFormat` → 406

⚠ COMMON MISTAKES

- Leaving a helper method public; it becomes an action reachable if a route points at it
- Rendering after redirect in the same action, which raises `AbstractController::DoubleRenderError`
- Returning a 200 on failed validation; Turbo needs 422 to show the re-rendered form

✓ TIPS & BEST PRACTICES

- Keep actions short: set up, call the model, respond
- Use endless methods (`def x = ...`) only for true one-liners like `post_params`
- Use `request.variant` (e.g. `:phone`) for device-specific templates

QUICK SUMMARY

- `public method` — an action
- `@ivars` — data for the view
- `implicit render` — template by name
- `private` — non-action helpers

Sheet 12

Params & Strong Parameters

params · params.expect (8.0) · require / permit · nested hashes & arrays · ParameterMissing · action_on_unpermitted_parameters

SAFE INPUT FROM REQUESTS

`params` merges route segments, the query string and the request body into one hash-like `ActionController::Parameters` object. Before mass-assigning it to a model you must **permit** the keys you allow. Rails 8 adds `params.expect`, which permits and checks the shape in one call and returns **400 Bad Request** instead of 500 when a client sends the wrong structure.

PARAMS FACTS

- All values are strings (or arrays/hashes of strings)
- Keys can be read as strings or symbols: `params[:id] == params["id"]`
- Passing unpermitted params to `Post.new` raises `ForbiddenAttributesError`
- Unpermitted keys are dropped and logged in development by default

RAILS 8: EXPECT

```
params.expect(:id)                              # "42"  (scalar)
params.expect(post: [:title, :body])            # permitted hash
params.expect(post: [:title, tag_ids: []])      # array of scalars
params.expect(post: [:title, images: [[:url, :alt]]])   # array of hashes
params.expect(post: [:title, author: [:name, :email]])  # nested hash
params.expect(post: [:title, settings: {}])     # any keys (use sparingly)
```

CLASSIC REQUIRE / PERMIT

```
params.require(:post).permit(:title, :body, tag_ids: [])
params.permit(:page, :q)                        # top-level keys
params.fetch(:filter, {}).permit(:status)       # optional root
```

IN A CONTROLLER

```
def post_params
  params.expect(post: [:title, :body, :published,
                       comments_attributes: [[:id, :body, :_destroy]]])
end
```

CODE EXPLAINED

- → `expect(:id)` — fails with 400 if `id` is missing or is a hash
- → `[[:url, :alt]]` — double brackets mean "array of hashes" with `expect`
- → `tag_ids: []` — an array of scalars (checkbox lists)
- → `settings: {}` — permits any nested keys: only for trusted, free-form data
- → `comments_attributes` — needed for `accepts_nested_attributes_for`

THE RAILS WAY

- `Post.new(params[:post])` → `Post.new(post_params)` with an explicit allow-list
- `params.require(:post).permit(...)` in new code → `params.expect(post: [...])`
- `permit!` to "make it work" → list the keys; `permit!` reopens mass-assignment holes

FORM FIELD → PARAM SHAPE

FIELD NAME	ARRIVES AS
`post[title]`	`{ "post" => { "title" => "Hi" } }`
`post[tag_ids][]`	`{ "post" => { "tag_ids" => ["1", "2"] } }`
`post[author][name]`	nested hash
`post[images][0][url]`	hash with index keys (array of hashes)

ERRORS & SETTINGS

ITEM	BEHAVIOR
`ParameterMissing`	Required key absent → 400
`ForbiddenAttributesError`	Unpermitted params mass-assigned
`UnpermittedParameters`	Raised when set to `:raise`
`action_on_unpermitted_parameters`	`:log`, `:raise` or `false`

⚠ COMMON MISTAKES

- Permitting `:role` or `:admin` in a user form, letting users promote themselves
- Using single brackets for arrays of hashes with `expect` (`images: [:url]`); it expects a single hash
- Calling `.to_h` on unpermitted params, which raises `UnfilteredParameters`

✓ TIPS & BEST PRACTICES

- Set `config.action_controller.action_on_unpermitted_parameters = :raise` in development and test to catch typos
- Keep one `*_params` method per resource, private, at the bottom of the controller
- Permit different keys per role with separate methods (`admin_user_params`)

QUICK SUMMARY

- `params` — merged request input
- `expect` — permit + shape check (8.0)
- `permit` — the allow-list
- `[[...]]` — array of hashes

Rendering & Redirects

SENDING THE RESPONSE

`render` builds a response body in the **same request**, usually a template. `redirect_to` tells the browser to make a **new request** to another URL. Use render to re-show a form with errors and redirect after a successful change (Post/Redirect/Get). `head` sends only a status and headers.

RESPONSE NOTES

- `render` doesn't run the target action; it only uses its template
- Code after `render` / `redirect_to` still runs; add `and return` or `return` if needed
- Turbo expects **422** for invalid forms and **303** after non-GET redirects
- `redirect_to` accepts a model, a path, a URL or `:back` alternatives

RENDER VARIANTS

```
render :edit                                  # posts/edit template
render "shared/notice"                        # another folder
render :new, status: :unprocessable_entity    # 422 for failed save
render json: @post                            # calls to_json
render json: { error: "nope" }, status: :forbidden
render plain: "OK"                            # text/plain
render html: helpers.tag.p("Hi")              # trusted HTML only
render partial: "row", locals: { post: @post }
render turbo_stream: turbo_stream.remove(@post)
head :no_content                              # 204, no body
```

CODE EXPLAINED

- → `status:` — symbol (`:created`) or number (`201`)
- → `render json:` — sets content type `application/json`
- → `status: :see_other` — 303 so DELETE/PATCH redirects become GET
- → `allow_other_host: true` — required for external redirects (open-redirect guard)
- → `and return` — stops the action after an early redirect

REDIRECT VARIANTS

```
redirect_to @post                             # post_path(@post)
redirect_to posts_path, notice: "Deleted", status: :see_other
redirect_to "https://example.com", allow_other_host: true
redirect_back_or_to root_path                 # referrer or fallback
redirect_to edit_post_path(@post) and return if @post.locked?
```

THE RAILS WAY

- `render :index` after a successful create → `redirect_to` so refresh doesn't resubmit
- `render :new` (200) on failure → `render :new, status: :unprocessable_entity`
- `redirect_to request.referer` → `redirect_back_or_to fallback`

RENDER VS REDIRECT_TO

	RENDER · REDIRECT_TO
New request?	no · yes (browser follows `Location`)
Instance vars kept?	yes · no
Typical use	invalid form · successful change
Default status	200 · 302

COMMON STATUS SYMBOLS

SYMBOL	CODE
`:ok`	200
`:created`	201
`:no_content`	204
`:see_other`	303
`:unauthorized`	401
`:forbidden`	403
`:not_found`	404
`:unprocessable_entity`	422
`:too_many_requests`	429

⚠ COMMON MISTAKES

- Calling `redirect_to` twice in one action (often via a before_action plus the action) → `DoubleRenderError`
- Rendering user input with `render html:` or `render inline:` ; it bypasses escaping
- Redirecting to `params[:return_to]` without `url_from` or a host check (open redirect)

✓ TIPS & BEST PRACTICES

- Use `url_from(params[:return_to]) || root_path` to accept only same-host return URLs
- Pass `notice:` / `alert:` to `redirect_to` instead of setting `flash` on a separate line
- For APIs, `head :created, location: @post` is often all a client needs

QUICK SUMMARY

- `render` — same request
- `redirect_to` — new request
- `422` — invalid form
- `303` — after DELETE/PATCH

Action Callbacks

RUN CODE AROUND ACTIONS

Controller callbacks (formerly "filters") run before, after or around actions. They load records, enforce authentication, set locale or time zone, and log. If a `before_action` renders or redirects, the action **does not run**: that is how auth checks halt a request.

CALLBACK RULES

- Callbacks run in the order declared; parent class callbacks first
- `only:` / `except:` restrict which actions trigger them
- `prepend_before_action` puts a callback at the front of the chain
- `after_action` runs only if the action didn't raise

APP/CONTROLLERS/APPLICATION_CONTROLLER.RB

```ruby
class ApplicationController < ActionController::Base
  before_action :require_authentication           # from the auth generator
  around_action :use_user_time_zone

  private

  def use_user_time_zone(&action)
    Time.use_zone(Current.user&.time_zone || "UTC", &action)
  end
end
```

APP/CONTROLLERS/POSTS_CONTROLLER.RB

```ruby
class PostsController < ApplicationController
  allow_unauthenticated_access only: %i[index show]  # skips the auth callback
  before_action :set_post, only: %i[show edit update destroy]
  before_action :authorize_owner!, only: %i[edit update destroy]
  after_action  :track_view, only: :show

  private

  def set_post
    @post = Post.find(params.expect(:id))
  end

  def authorize_owner!
    redirect_to @post, alert: "Not yours" unless @post.user == Current.user
  end                                              # redirect halts the chain

  def track_view = ViewCounterJob.perform_later(@post)
end
```

CODE EXPLAINED

- → `around_action` — must `yield` (or call the block) to run the action
- → `allow_unauthenticated_access` — generator macro wrapping `skip_before_action`
- → `authorize_owner!` — redirect inside a before_action stops the action
- → `only:` — scopes each callback to specific actions
- → `after_action` — good for side effects; the response is already built

THE RAILS WAY

- `@post = Post.find(params[:id])` repeated in four actions → one `before_action :set_post`
- Auth checks inside each action → a `before_action` in `ApplicationController`, skipped where public
- Callbacks with hidden business logic → keep callbacks to setup and guards

CALLBACK METHODS

METHOD	PURPOSE
`before_action`	Load, authenticate, authorize
`after_action`	Logging, headers, analytics
`around_action`	Wrap: time zone, locale, timing
`skip_before_action`	Remove an inherited callback
`prepend_before_action`	Run before others
`append_before_action`	Same as before_action

OPTIONS

OPTION	EXAMPLE
`only:`	`only: %i[edit update]`
`except:`	`except: :index`
`if: / unless:`	`if: -> { request.format.html? }`
`raise: false`	`skip_before_action :x, raise: false`

⚠ COMMON MISTAKES

- Skipping auth with `skip_before_action :require_authentication` and no `only:`, opening every action
- Setting `@post` in a callback that doesn't cover all actions using it → `nil` errors
- Forgetting to `yield` in an `around_action`, so the action never runs and nothing renders

✓ TIPS & BEST PRACTICES

- Name guard callbacks with `!` (`authorize_owner!`) so their halting role is obvious
- Run `bin/rails runner 'p PostsController._process_action_callbacks.map(&:filter)'` to see the chain
- Prefer `Current` attributes over passing the user through many callbacks

QUICK SUMMARY

- `before_action` — guard/load
- `around_action` — wrap with yield
- `render/redirect` — halts
- `only:` / `except:` — scope

Sessions, Cookies & Flash

STATE BETWEEN REQUESTS

HTTP is stateless, so Rails stores small values in **cookies**. `session` is an encrypted cookie store by default. `cookies.signed` prevents tampering and `cookies.encrypted` also hides the value. **Flash** messages survive exactly one redirect, which is how "Post created" appears on the next page.

STORAGE NOTES

- Cookie session limit is about **4 KB**: store ids, never whole objects
- Signing/encryption uses `secret_key_base` from credentials
- `flash.now` shows a message on the **current** render only
- The Rails 8 auth generator stores a signed `session_id` cookie pointing at a `Session` row

SESSION & COOKIES

```
session[:cart_id] = cart.id
session[:cart_id]                          # read in any later request
session.delete(:cart_id)
reset_session                              # on login/logout: blocks fixation

cookies[:theme] = "dark"                   # plain, readable by JS
cookies[:theme] = { value: "dark", expires: 1.year, httponly: true }
cookies.signed[:user_id] = user.id         # tamper-proof, readable
cookies.encrypted[:token] = "s3cret"       # tamper-proof and hidden
cookies.permanent.signed[:remember] = user.id   # 20-year expiry
cookies.delete(:theme)
```

CODE EXPLAINED

- → `reset_session` — new session id; call it when privilege changes
- → `httponly: true` — JavaScript can't read the cookie
- → `signed` vs `encrypted` — integrity only vs integrity + secrecy
- → `notice:` / `alert:` — shortcuts built into `redirect_to`
- → `flash.now` + `render` — no redirect, so a normal flash would leak to the next page

FLASH

```
redirect_to @post, notice: "Saved"                  # flash[:notice]
redirect_to root_path, alert: "Access denied"
flash[:success] = "Welcome!"                        # custom key
flash.now[:alert] = "Fix the errors"; render :new, status: 422
flash.keep                                          # survive one more redirect
```

THE RAILS WAY

- Storing a `User` object in the session → store the id and look it up
- `flash[:alert]` before `render` → `flash.now[:alert]`
- A plain cookie for anything security-related → `cookies.signed` or `.encrypted`

LAYOUT

```
<% flash.each do |type, msg| %>
  <div class="flash flash-<%= type %>"><%= msg %></div>
<% end %>
```

SESSION STORES

STORE	NOTES
`CookieStore`	Default. Encrypted, 4 KB, no server state
`CacheStore`	Uses `Rails.cache` (e.g. Solid Cache)
`ActiveRecordStore`	Gem; DB table, server-side revoke
`Auth generator`	Cookie holds `Session` id; rows revocable

COOKIE OPTIONS

OPTION	EFFECT
`expires:`	Time or duration; omit for browser-session
`httponly:`	Hide from JavaScript
`secure:`	HTTPS only (default in production)
`same_site:`	`:lax` (default), `:strict`, `:none`
`domain:`	`:all` to share across subdomains

⚠ COMMON MISTAKES

- Putting large or growing data in the cookie session and hitting `CookieOverflow`
- Not calling `reset_session` on login, leaving the app open to session fixation
- Trusting a plain `cookies[:user_id]`; users can edit it in the browser

✓ TIPS & BEST PRACTICES

- Use the auth generator's database-backed sessions so users can log out other devices
- Render flash once in the layout, keyed by type for CSS classes
- Rotate `secret_key_base` only with a rotation config, or every signed cookie becomes invalid

QUICK SUMMARY

- `session` — encrypted per-user store
- `cookies.signed / encrypted` — safe cookies
- `flash` — next request
- `flash.now` — this render

Formats & Error Handling

ONE ACTION, MANY FORMATS

`respond_to` lets one action answer HTML, JSON, Turbo Stream, CSV or (8.1) Markdown based on the URL extension or `Accept` header. `rescue_from` turns exceptions into responses in one place. In production, unhandled errors map to status codes and the static pages in `public/`.

FORMAT & ERROR NOTES

- Format comes from `.json` in the URL or the `Accept` header
- An unlisted format raises `ActionController::UnknownFormat` (406)
- `ActiveRecord::RecordNotFound` becomes a **404** in production automatically
- Rails 8.1 can render Markdown: `render markdown: @page` calls `@page.to_markdown`

RESPOND_TO

```
def show
  @post = Post.find(params.expect(:id))
  respond_to do |format|
    format.html                                  # show.html.erb
    format.json { render json: @post }
    format.csv  { send_data @post.to_csv, filename: "post.csv" }
    format.md   { render markdown: @post }       # 8.1: needs #to_markdown
  end
end
```

CODE EXPLAINED

- → `format.html` with no block — renders the default template
- → `send_data` — stream generated content as a download
- → `render markdown:` — 8.1, sets `text/markdown`
- → `rescue_from ... with:` — one handler for every action in the class
- → `exceptions_app = routes` — render error pages through your own controller

RESCUE_FROM

```
class ApplicationController < ActionController::Base
  rescue_from ActiveRecord::RecordNotFound, with: :not_found
  rescue_from Pundit::NotAuthorizedError do
    redirect_back_or_to root_path, alert: "Not allowed"
  end

  private

  def not_found
    respond_to do |format|
      format.html { render "errors/not_found", status: :not_found }
      format.json { render json: { error: "not found" }, status: :not_found }
    end
  end
end
```

THE RAILS WAY

- `begin/rescue` around `find` in every action → let it raise; handle with `rescue_from` or the 404 page
- Separate `show_json` actions → one action with `respond_to`
- `rescue_from StandardError` → rescue specific error classes

CUSTOM ERROR PAGES (CONFIG/APPLICATION.RB)

```
config.exceptions_app = routes   # then: match "/404", to: "errors#not_found", via: :all
```

EXCEPTION → STATUS (PRODUCTION)

EXCEPTION	STATUS
ActiveRecord::RecordNotFound	404
ActionController::RoutingError	404
ActionController::ParameterMissing	400
ActionController::InvalidAuthenticityToken	422
ActiveRecord::RecordInvalid	422
ActionController::UnknownFormat	406

STATIC ERROR PAGES

FILE	SHOWN FOR
public/404.html	Not found
public/422.html	Unprocessable
public/500.html	Server error
public/406-unsupported-browser.html	`allow_browser` rejection
public/400.html	Bad request

⚠ COMMON MISTAKES

- `rescue_from StandardError` hides bugs and swallows errors your error tracker should see
- Expecting the 404 page in development; set `config.consider_all_requests_local = false` to preview it
- Forgetting `format.json` in `respond_to` and getting 406 from API clients

✓ TIPS & BEST PRACTICES

- Report rescued errors with `Rails.error.report(e)` so they still reach monitoring
- Keep `public/500.html` plain HTML; it must render even when the app can't
- Use `request.format.json?` checks sparingly; `respond_to` is clearer

QUICK SUMMARY

- `respond_to` — per-format responses
- `render markdown:` — 8.1
- `rescue_from` — central handlers
- `public/*.html` — production error pages

Sheet 17

Concerns, Helpers & Current

controller concerns · helper_method · Current < CurrentAttributes · rate_limit (7.2) · allow_browser · etag / fresh_when

SHARING CONTROLLER BEHAVIOR

Concerns package callbacks and methods to include in several controllers. `helper_method` exposes a controller method to views. **CurrentAttributes** hold per-request globals (the current user, account, request id) that reset automatically. Rails 7.2 added `rate_limit` and `allow_browser` macros.

NOTES

- Controller concerns live in `app/controllers/concerns/`
- `Current` attributes reset at the start and end of every request and job
- `rate_limit` stores counters in `Rails.cache` (Solid Cache by default)
- `allow_browser versions: :modern` is in new apps' `ApplicationController`

APP/CONTROLLERS/CONCERNS/PAGINATABLE.RB

```
module Paginatable
  extend ActiveSupport::Concern

  included do
    helper_method :page                          # usable in views
  end

  def page = [params[:page].to_i, 1].max
end
```

CODE EXPLAINED

- → `included do` — runs class-level macros in the including controller
- → `helper_method` — `<%= page %>` works in views
- → `Current.user` — available in models, mailers and jobs for this request
- → `rate_limit` — 429 by default, or your `with:` handler
- → `fresh_when` — sets ETag / Last-Modified and returns 304 when cached

APP/MODELS/CURRENT.RB

```
class Current < ActiveSupport::CurrentAttributes
  attribute :session
  delegate :user, to: :session, allow_nil: true
end
```

A CONTROLLER USING THEM

```
class SessionsController < ApplicationController
  include Paginatable
  rate_limit to: 10, within: 3.minutes, only: :create,
             with: -> { redirect_to new_session_url, alert: "Try again later." }
end

class ApplicationController < ActionController::Base
  allow_browser versions: :modern                # 406 for old browsers
end

def show                                         # conditional GET
  @post = Post.find(params.expect(:id))
  fresh_when @post                               # 304 if unchanged
end
```

THE RAILS WAY

- Copying the same filters into five controllers → a concern
- Passing `current_user` through every method → `Current.user`
- A hand-rolled Redis throttle for login → `rate_limit`

RATE_LIMIT OPTIONS

OPTION	MEANING
`to:`	Max requests
`within:`	Window (`1.minute`)
`by:`	Key lambda (default: remote IP)
`with:`	Response when exceeded
`store:`	Cache store to use
`name:`	Separate limits on one controller

CONDITIONAL GET

METHOD	USE
`fresh_when obj`	Auto ETag/Last-Modified, render or 304
`stale?(obj)`	`if stale?(@post) ... end` for custom work
`expires_in 1.hour`	`Cache-Control: max-age`
`http_cache_forever`	Immutable resources

⚠ COMMON MISTAKES

- Using `Current` as a general global for things that outlive a request; it resets between requests
- Treating `rate_limit` as a security boundary on its own; it depends on the cache and the client IP
- Huge "god" concerns that every controller includes; split by behavior

✓ TIPS & BEST PRACTICES

- Use `by: -> { Current.user&.id || request.remote_ip }` for per-user limits
- Set `Current` values in a before_action so jobs enqueued in the request can read them
- Use `fresh_when` on public show pages for cheap 304 responses

QUICK SUMMARY

- `concern` — shared controller code
- `helper_method` — expose to views
- `Current` — per-request globals
- `rate_limit` — throttle actions

ERB Templates

<%= %> · <% %> · <%# %> · auto-escaping · html_safe / raw · template lookup · Herb compiler (8.2 edge)

EMBEDDED RUBY

Views are **ERB** templates: HTML with Ruby inside tags. `<%= %>` outputs the result, `<% %>` runs code without output. Output is **HTML-escaped automatically**, so user data can't inject markup. Templates are found by convention: `app/views/<controller>/<action>.<format>.erb`.

ERB NOTES

- File names: `show.html.erb`, `show.json.jbuilder`, `show.turbo_stream.erb`
- Views see controller instance variables and all helpers
- `-%>` trims the newline after a tag
- Rails 8.2 (edge) switches new apps to the **Herb** HTML-aware ERB engine, which also catches unclosed tags

APP/VIEWS/POSTS/SHOW.HTML.ERB

```
<%# a comment: never sent to the browser %>
<h1><%= @post.title %></h1>
<p>By <%= @post.author.name %> · <%= time_ago_in_words(@post.created_at) %> ago</p>

<% if @post.published? %>
  <%= simple_format(@post.body) %>
<% else %>
  <p class="muted">Draft</p>
<% end %>

<ul>
  <% @post.tags.each do |tag| %>
    <li><%= tag.name %></li>
  <% end %>
</ul>

<%= "<b>bold?</b>" %>          <%# escaped: &lt;b&gt;bold?&lt;/b&gt; %>
<%= sanitize @post.summary %>  <%# keeps safe tags only %>
<%= raw @trusted_html %>       <%# no escaping: trusted input only %>
```

CODE EXPLAINED

- → `<%= %>` — evaluate and output (escaped)
- → `<% %>` — control flow: `if`, `each`, assignments
- → `simple_format` — turns newlines into `<p>` / `<br>`
- → `sanitize` — strips scripts and unsafe attributes
- → `raw` / `.html_safe` — mark a string as safe: never on user input

THE RAILS WAY

- `<% @posts = Post.all %>` in a view → load data in the controller
- Big `if` trees in templates → a helper method or partial
- `.html_safe` on user content → `sanitize` or plain escaped output

TAGS

TAG	MEANING
`<%= expr %>`	Output escaped result
`<% code %>`	Run, no output
`<%# comment %>`	Comment
`<%== expr %>`	Output without escaping (like raw)
`<%- / -%>`	Trim leading / trailing whitespace

TEMPLATE LOOKUP FOR POSTSCONTROLLER#SHOW

REQUEST	TEMPLATE
`GET /posts/1`	`posts/show.html.erb`
`GET /posts/1.json`	`posts/show.json.jbuilder`
`Turbo Stream request`	`posts/show.turbo_stream.erb`
`variant :phone`	`posts/show.html+phone.erb`
`missing in posts/`	falls back to `application/show`

⚠ COMMON MISTAKES

- Using `<%` where you meant `<%=`, so nothing appears and no error is raised
- Calling `.html_safe` on interpolated strings containing user data → XSS
- Writing `<%= @posts.each do |p| %>`, which prints the whole array inspect after the loop

✓ TIPS & BEST PRACTICES

- Use `debug(@post)` or `<%= console %>` (web-console) to inspect values during development
- Turn on `config.action_view.annotate_rendered_view_with_filenames` to see which template rendered what
- Keep logic in helpers or models; templates should read like HTML

QUICK SUMMARY

- `<%= %>` — output
- `<% %>` — logic
- auto-escape — safe by default
- `raw` — trusted only

Layouts & content_for

THE PAGE WRAPPER

A **layout** wraps every view with shared HTML: <head>, navigation, flash and footer. The view's output goes where the layout calls `yield`. Views send extra content to named slots with `content_for`, such as the page title or page-specific scripts.

APP/VIEWS/LAYOUTS/APPLICATION.HTML.ERB

```
<!DOCTYPE html>
<html>
<head>
  <title><%= content_for(:title) || "Blog" %></title>
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <%= csrf_meta_tags %>
  <%= csp_meta_tag %>
  <%= stylesheet_link_tag :app, "data-turbo-track": "reload" %>
  <%= javascript_importmap_tags %>
  <%= yield :head %>
</head>
<body>
  <%= render "shared/nav" %>
  <%= render "shared/flash" %>
  <main class="<%= content_for?(:sidebar) ? "with-sidebar" : "" %>">
    <%= yield %>
  </main>
  <aside><%= yield :sidebar %></aside>
</body>
</html>
```

A VIEW FILLING THE SLOTS

```
<% content_for :title, "Edit #{@post.title}" %>
<% content_for :sidebar do %>
  <%= render "posts/help" %>
<% end %>
<h1>Edit post</h1>
```

CHOOSING A LAYOUT

```
class Admin::BaseController < ApplicationController
  layout "admin"
end
render :show, layout: "print"                    # one action
```

LAYOUT RULES

- Default: `layouts/application.html.erb`
- A `layouts/posts.html.erb` file is used automatically for `PostsController`
- `layout "admin"` in a controller picks one explicitly; `layout false` disables it
- `provide` is like `content_for` but stops at the first value (streaming-friendly)

CODE EXPLAINED

- → `yield` — the view's content
- → `yield :sidebar` — a named slot, empty if no view filled it
- → `content_for?` — test whether a slot has content
- → `csrf_meta_tags` — tokens for JavaScript requests
- → `stylesheet_link_tag :app` — every stylesheet in `app/assets/stylesheets` (Propshaft)

THE RAILS WAY

- Copying `<head>` into every view → one layout with `yield`
- Instance variables like `@page_title` set in controllers → `content_for :title` in the view
- Huge layout files → partials for nav, flash and footer

LAYOUT RESOLUTION

SETUP	LAYOUT USED
`Nothing set`	`layouts/<controller>` or `application`
`layout "admin"`	`layouts/admin.html.erb`
`layout :pick`	Method returning a name
`layout false`	No layout
`render layout: "x"`	This response only

HEAD HELPERS

HELPER	OUTPUTS
`csrf_meta_tags`	CSRF token meta tags
`csp_meta_tag`	CSP nonce meta tag
`javascript_importmap_tags`	Import map + entry module
`favicon_link_tag`	Favicon link
`turbo_refreshes_with`	Morph refresh meta tags

⚠ COMMON MISTAKES

- Calling `content_for :title` after the layout already rendered it (e.g. inside a lazy-loaded frame)
- Forgetting `csrf_meta_tags`, which breaks Turbo and fetch-based non-GET requests
- Nesting layouts by rendering one layout from another instead of using `content_for` slots

✓ TIPS & BEST PRACTICES

- Use `content_for :head` for page-specific `<meta>` or preload tags
- Nested layouts: set `content_for` slots, then `render template: "layouts/application"`
- Add `data-turbo-track="reload"` to asset tags so deploys trigger a full reload

QUICK SUMMARY

- `yield` — main content
- `content_for` — named slots
- `layout "x"` — pick a layout
- `content_for?` — slot filled?

Sheet 20

Partials & Collection Rendering

render "form" · locals: · render @posts · collection: / as: · strict locals comment (7.1) · cached: true · spacer_template

REUSABLE VIEW PIECES

A **partial** is a template whose file name starts with an underscore. `render "form"` renders `_form.html.erb` from the same folder. Rendering a model or collection uses conventions: `render @posts` renders `posts/_post.html.erb` once per post, with a local named `post`. Rails 7.1 **strict locals** declare which locals a partial accepts.

PARTIAL NOTES

- `render "shared/nav"` → `app/views/shared/_nav.html.erb`
- Pass data through `locals`, not instance variables
- Collection rendering is one call, much faster than a loop of `render`
- Inside a collection partial: `post_counter`, `post_iteration.first?` / `.last?`

APP/VIEWS/POSTS/_POST.HTML.ERB

```
<%# locals: (post:, show_author: true) -%>
<article id="<%= dom_id post %>">
  <h2><%= link_to post.title, post %></h2>
  <% if show_author %><p>by <%= post.author.name %></p><% end %>
</article>
```

CODE EXPLAINED

- → `locals: (post:, show_author: true)` — required and optional locals; unknown ones raise
- → `dom_id post` — `"post_42"`, used by Turbo for targeting
- → `render @posts` returns `nil` on an empty collection, so `||` adds a fallback
- → `cached: true` — multi-fetch fragment cache for every item
- → `spacer_template` — rendered between items

RENDERING IT

```
<%= render @posts %>                                          <%# _post per item %>
<%= render @posts, show_author: false %>                      <%# extra local %>
<%= render partial: "posts/post", collection: @posts, as: :post %>
<%= render(@posts) || tag.p("No posts yet") %>                <%# empty fallback %>
<%= render partial: "post", collection: @posts, cached: true %>
<%= render partial: "post", collection: @posts, spacer_template: "divider" %>

<%= render "form", post: @post %>                             <%# _form.html.erb %>
<%= render @post %>                                           <%# single model %>
<%= render "shared/card" do %>Body text<% end %>              <%# partial with a block %>
```

THE RAILS WAY

- `<% @posts.each do |p| %><%= render "post", post: p %><% end %>` → `render @posts`
- Partials reading `@post` → pass `post:` as a local
- Guessing which locals a partial needs → declare strict locals

SHORTCUT → EXPANDED

SHORTCUT	EQUIVALENT
`render @post`	`render partial: "posts/post", locals: { post: @post }`
`render @posts`	`render partial: "posts/post", collection: @posts`
`render "form", post: p`	`render partial: "form", locals: { post: p }`
`render [@a, @b]`	Mixed models, each with its own partial

COLLECTION LOCALS

LOCAL	VALUE
`post`	The current item
`post_counter`	0-based index
`post_iteration.index`	Same as counter
`post_iteration.first?`	True on first item
`post_iteration.last?`	True on last item

⚠ COMMON MISTAKES

- Naming the file `post.html.erb` without the underscore; `render "post"` can't find it
- Rendering partials in a loop (N renders) instead of collection rendering
- Passing a local the strict-locals comment doesn't declare → `ActionView::Template::Error`

✓ TIPS & BEST PRACTICES

- Use strict locals in every new partial; it documents the partial and catches typos
- Combine `cached: true` with `touch: true` on associations for Russian-doll caching (sheet 63)
- Put cross-controller partials in `app/views/shared/` or `application/`

QUICK SUMMARY

- `_name.html.erb` — a partial
- `render @posts` — collection by convention
- `locals:` — pass data in
- strict locals — declare inputs

Built-in View Helpers

HELPERS YOU USE DAILY

Action View ships hundreds of helpers for links, tags, numbers, dates and text. They escape output, build correct URLs from routes, and add Rails conventions such as CSRF tokens and fingerprinted asset paths. Use `button_to` for anything that changes data.

HELPER NOTES

- `link_to` always makes a GET link; `button_to` makes a form (POST by default)
- `tag.div`, `tag.span` build any element; `class:` accepts arrays and hashes
- Number and date helpers respect I18n locale settings
- Helpers are available in views, and in the console via `helper.`

LINKS, BUTTONS & TAGS

```
<%= link_to "Posts", posts_path, class: "nav" %>
<%= link_to @post.title, @post %>
<%= link_to "Docs", "https://guides.rubyonrails.org", target: "_blank", rel: "noopener" %>
<%= button_to "Delete", @post, method: :delete,
      form: { data: { turbo_confirm: "Sure?" } } %>
<%= link_to "Cancel", :back %>
<%= mail_to "hi@example.com" %>
<%= image_tag "logo.svg", alt: "Logo", width: 120 %>
<%= tag.div "Saved", class: ["flash", { "flash-error": @error }] %>
<%= tag.span data: { controller: "clipboard" } do %>Copy<% end %>
```

CODE EXPLAINED

- → `button_to ... method: :delete` — a tiny form, safe for non-GET actions
- → `data: { turbo_confirm: }` — Turbo confirm dialog before submit
- → `class: [..., { "x": cond }]` — conditional classes
- → `l(...)` — localized date/time formatting
- → `pluralize` — uses Active Support inflections

NUMBERS, DATES & TEXT

```
number_to_currency(1234.5)            # "$1,234.50"
number_with_delimiter(1234567)        # "1,234,567"
number_to_percentage(63.8, precision: 0)   # "64%"
number_to_human_size(1_500_000)       # "1.43 MB"
time_ago_in_words(3.hours.ago)        # "about 3 hours"
l(Date.current, format: :long)        # "October 23, 2026"
truncate("A long sentence", length: 10, separator: " ")  # "A long..."
pluralize(2, "comment")               # "2 comments"
excerpt("Rails is fun", "is", radius: 3)      # "...ls is fu..."
```

THE RAILS WAY

- `<a href="<%= post_path(p) %>">` → `link_to p.title, p`
- `link_to "Delete", ..., data: { turbo_method: :delete }` for destructive actions → `button_to`
- Formatting money with `"$#{price}"` → `number_to_currency(price)`

ASSET & URL HELPERS

HELPER	OUTPUT
`image_tag "a.png"`	`<img src="/assets/a-3f2e….png">`
`asset_path "a.png"`	Fingerprinted path
`url_for(@post)`	`/posts/1`
`current_page?(posts_path)`	`true` / `false`
`link_to_unless_current`	Text when already on that page

TEXT HELPERS

HELPER	DOES
`simple_format`	Newlines → <p> /
`highlight`	Wrap matches in <mark>
`word_wrap`	Insert line breaks
`strip_tags`	Remove HTML tags
`cycle("odd", "even")`	Alternate values in loops
`dom_id(post)`	`"post_1"`

⚠ COMMON MISTAKES

- Making delete links with `link_to`; crawlers and prefetchers can trigger GET links
- Passing user input as `link_to` URLs without checking the scheme (`javascript:` links)
- Using `number_to_currency` on floats for stored money; keep money as decimals or integer cents

✓ TIPS & BEST PRACTICES

- Use `tag.attributes(...)` (7.1+) to spread attributes into hand-written tags
- Put app-specific formatting in a helper (`format_price`) that wraps the built-ins
- Try helpers quickly in `bin/rails c` with `helper.number_to_human(12345)`

QUICK SUMMARY

- `link_to` — GET links
- `button_to` — actions that change data
- `tag.*` — build elements
- `number_* / time_ago_*` — formatting

form_with Basics

FORMS BOUND TO MODELS

`form_with model: @post` builds a form whose URL, HTTP method, field names and values come from the model. A new record posts to `posts_path`; a saved one patches `post_path(@post)`. Field names like `post[title]` line up with `params.expect(post: [...])` in the controller.

FORM NOTES

- Every form includes a hidden CSRF `authenticity_token`
- `f.submit` label is "Create Post" or "Update Post" automatically
- Turbo submits the form via fetch; respond with 422 or a redirect (303)
- Use `url:` for forms not tied to a model, such as search

APP/VIEWS/POSTS/_FORM.HTML.ERB

```erb
<%= form_with model: post do |f| %>
  <% if post.errors.any? %>
    <div class="errors">
      <h2><%= pluralize(post.errors.count, "error") %> prevented saving:</h2>
      <ul><% post.errors.full_messages.each do |msg| %><li><%= msg %></li><% end %></ul>
    </div>
  <% end %>

  <%= f.label :title %>
  <%= f.text_field :title, autofocus: true, required: true %>

  <%= f.label :body %>
  <%= f.textarea :body, rows: 8 %>

  <%= f.label :status %>
  <%= f.select :status, Post.statuses.keys, include_blank: "Pick one" %>

  <%= f.check_box :featured %> <%= f.label :featured %>
  <%= f.submit class: "btn" %>
<% end %>
```

CODE EXPLAINED

→ `model: post` — URL, method and `post[...]` names inferred

→ `errors.full_messages` — "Title can't be blank"

→ `f.textarea` — 8.0 name; `text_area` still works

→ `check_box` — adds a hidden `0` so unchecked boxes still submit

→ `method: :get` — search params go in the query string

A SEARCH FORM (NO MODEL)

```erb
<%= form_with url: search_path, method: :get do |f| %>
  <%= f.search_field :q, value: params[:q] %>
  <%= f.submit "Search", name: nil %>
<% end %>
```

THE RAILS WAY

- Hand-written `<input name="post[title]">` → `f.text_field :title`
- Separate new/edit forms → one `_form` partial used by both
- `form_tag` / `form_for` → `form_with` (the unified API)

FIELD HELPERS

HELPER	INPUT
`text_field / textarea`	text / multi-line
`email_field / password_field`	email / password
`number_field / range_field`	number / slider
`date_field / datetime_local_field`	date pickers
`check_box / radio_button`	checkbox / radio
`select / collection_select`	dropdowns
`hidden_field / file_field`	hidden / upload

FORM_WITH OPTIONS

OPTION	EFFECT
`model:`	Bind to a record
`url:`	Explicit action URL
`scope:`	Param prefix without a model
`method:`	`:get`, `:patch`, `:delete`
`data: { turbo: false }`	Plain browser submit
`html: { class: }`	Attributes on `<form>`

⚠ COMMON MISTAKES

- Rendering the form with status 200 after a failed save; Turbo won't replace the page
- Forgetting `name: nil` on a GET search submit, which adds `commit=Search` to the URL
- Using `form_with model: @comment` for a nested resource; pass `[@post, @comment]`

✓ TIPS & BEST PRACTICES

- Use `f.label :title, "Headline"` to override text; otherwise it reads from I18n
- Wrap field + error in a small helper so invalid fields get an error class
- Rails adds `field_with_errors` wrappers; style them or set `config.action_view.field_error_proc`

QUICK SUMMARY

- `form_with model:` — bound form
- `f.*_field` — inputs
- `errors.full_messages` — show errors
- `url:` — non-model forms

Advanced Forms

ASSOCIATIONS & COMPLEX FORMS

Forms often edit more than one record. `collection_select` and `collection_check_boxes` set associations from a list. `fields_for` with `accepts_nested_attributes_for` saves child records in the same submit. When a form doesn't map to one table, build a **form object** with Active Model.

NOTES

- Nested attributes params end in `_attributes` and must be permitted
- Include `:id` and `:_destroy` to update and remove existing children
- `collection_check_boxes` sends `tag_ids: []` with a hidden empty value
- File uploads need `f.file_field`; `form_with` sets multipart automatically

MODEL

```ruby
class Project < ApplicationRecord
  belongs_to :category
  has_many :tasks, dependent: :destroy
  has_and_belongs_to_many :tags
  has_one_attached :logo
  accepts_nested_attributes_for :tasks, allow_destroy: true,
    reject_if: ->(attrs) { attrs["name"].blank? }
end
```

FORM

```erb
<%= form_with model: @project do |f| %>
  <%= f.collection_select :category_id, Category.order(:name), :id, :name,
        prompt: "Category" %>
  <%= f.collection_check_boxes :tag_ids, Tag.all, :id, :name %>
  <%= f.file_field :logo, accept: "image/*", direct_upload: true %>

  <%= f.fields_for :tasks do |tf| %>
    <%= tf.text_field :name %>
    <%= tf.check_box :_destroy %> <%= tf.label :_destroy, "Remove" %>
  <% end %>
  <%= f.submit %>
<% end %>
```

CONTROLLER PARAMS

```ruby
params.expect(project: [:category_id, :logo, tag_ids: [],
                        tasks_attributes: [[:id, :name, :_destroy]]])
```

CODE EXPLAINED

- → `collection_select` — `(method, collection, value_method, text_method)`
- → `tag_ids` — Rails writes the join rows for you
- → `direct_upload: true` — browser uploads straight to storage (sheet 60)
- → `fields_for :tasks` — one block per existing task
- → `reject_if` — skip blank rows instead of failing validation

THE RAILS WAY

- Saving parent and children in separate controller actions → nested attributes or a form object
- Complex nested attributes across three levels → an Active Model form object
- `options_for_select(Category.all.map { ... })` → `collection_select`

CHOICE HELPERS

HELPER	USE
`select`	Array or hash of options
`collection_select`	Records → dropdown
`grouped_collection_select`	Optgroups
`collection_radio_buttons`	Records → radios
`collection_check_boxes`	Many-to-many ids
`time_zone_select`	Zone dropdown

FORM OBJECT PATTERN

PIECE	PURPOSE
`include ActiveModel::Model`	Works with `form_with`
`attribute :email, :string`	Typed attributes
`validates ...`	Validation messages
`def save`	Write several models in a transaction
`form_with model: @signup, url:`	Point at its controller

⚠ COMMON MISTAKES

- Forgetting `:id` in permitted nested attributes, so every submit creates duplicate children
- Missing `allow_destroy: true`, so `_destroy` is ignored
- Using `collection_check_boxes` but permitting `:tag_ids` without `[]`

✓ TIPS & BEST PRACTICES

- Add new nested rows dynamically with a `<template>` and a small Stimulus controller
- Reach for a form object once a form touches more than two models
- Use `f.fields_for :address` for `has_one` / `belongs_to` too

QUICK SUMMARY

- `collection_select` — pick a record
- `fields_for` — nested records
- `accepts_nested_attributes_for` — model side
- form object — multi-model forms

Custom Helpers & View Components

YOUR OWN VIEW BUILDING BLOCKS

Helpers are modules in `app/helpers` whose methods every view can call. They suit small formatting and markup snippets. For larger reusable UI with logic and tests, many apps use **components**: Ruby classes paired with templates (ViewComponent) or written fully in Ruby (Phlex).

NOTES

- All helpers are included in all views by default (`config.action_controller.include_all_helpers`)
- Build HTML in helpers with `tag` and `safe_join`, never string concatenation
- `capture` turns a block of ERB into a string
- Components are gems: `view_component` (GitHub) and `phlex-rails`

APP/HELPERS/APPLICATION_HELPER.RB

```ruby
module ApplicationHelper
  def status_badge(post)
    css = post.published? ? "badge-green" : "badge-gray"
    tag.span(post.status.humanize, class: ["badge", css])
  end

  def nav_link(text, path)
    link_to text, path, class: ["nav", { active: current_page?(path) }]
  end

  def card(title, &block)
    tag.div(class: "card") do
      safe_join([tag.h3(title), capture(&block)])
    end
  end
end
```

CODE EXPLAINED

- → `tag.span(..., class: [...])` — escaped, attribute-safe HTML
- → `current_page?` — highlight the active nav link
- → `capture(&block)` — render the caller's ERB block to a string
- → `safe_join` — join escaped and safe strings without `.html_safe`
- → `content` — the block passed to a ViewComponent

VIEWCOMPONENT

```ruby
class AlertComponent < ViewComponent::Base
  def initialize(type: :info)
    @type = type
  end

  def call = tag.div(content, class: "alert alert-#{@type}")
end
# view: <%= render AlertComponent.new(type: :error) do %>Oops<% end %>
```

THE RAILS WAY

- `"<span class='#{css}'>#{text}</span>".html_safe` → `tag.span(text, class: css)`
- Helpers that query the database → pass data in from the controller
- A 200-line partial with logic → a component with its own test

HELPERS VS PARTIALS VS COMPONENTS

TOOL	BEST FOR
`Helper`	Small formatting, one-liners
`Partial`	Markup reused with simple locals
`ViewComponent`	UI with logic, slots, unit tests
`Phlex`	Views written in pure Ruby
`Presenter / decorator`	Model-specific display logic

BUILDER HELPERS

HELPER	USE
`tag.div / tag.p`	Any element
`content_tag(:div, ...)`	Older equivalent
`safe_join(arr, sep)`	Join with escaping
`capture { }`	Block → string
`token_list / class_names`	Conditional class strings

⚠ COMMON MISTAKES

- Building HTML by concatenating strings and calling `.html_safe`, which reintroduces XSS
- One giant `ApplicationHelper`; split by domain (`PostsHelper`, `MoneyHelper`)
- Calling helpers from models; use a presenter or pass formatted values instead

✓ TIPS & BEST PRACTICES

- Test helpers with `ActionView::TestCase` (`assert_dom_equal`)
- Use ViewComponent previews to build a component gallery for design review
- Use `class_names` (alias `token_list`) for conditional CSS classes in plain ERB

QUICK SUMMARY

- `app/helpers` — view methods
- `tag` / `safe_join` — safe HTML
- `capture` — block to string
- components — testable UI classes

Active Record Basics

ONE CLASS PER TABLE

Active Record is Rails' ORM: each model class maps to a database table and each instance to a row. Columns become attributes automatically, read from the schema at boot. Saving runs validations and callbacks, then issues the SQL for you.

CONVENTIONS

- Model `Post` ↔ table `posts`; `LineItem` ↔ `line_items`
- Primary key `id`; foreign keys `<model>_id`
- `created_at` / `updated_at` are set automatically
- `save` returns `true` / `false`; `save!` raises on failure

APP/MODELS/POST.RB

```
class Post < ApplicationRecord
end
```

CRUD IN THE CONSOLE

```
post = Post.new(title: "Hello")     # in memory
post.new_record?                    # true
post.save                           # INSERT … → true
post.persisted?                     # true

Post.create(title: "Quick")         # new + save
Post.create!(title: "")             # raises RecordInvalid if invalid

post.title = "Hi"
post.changed?                       # true
post.changes                        # {"title"=>["Hello", "Hi"]}
post.save                           # UPDATE only changed columns

post.update(title: "Hey")           # assign + save
post.update_column(:views, 10)      # skip validations/callbacks
post.increment!(:views)

post.destroy                        # DELETE + callbacks
post.destroyed?                     # true
post.reload                         # re-read from the DB
```

CODE EXPLAINED

- → `ApplicationRecord` — your base class; add app-wide model behavior here
- → `new` vs `create` — build only vs build and save
- → `changes` — dirty tracking: `title_changed?`, `title_was`
- → `update_column` — direct SQL, no validations, no `updated_at`
- → `destroy` vs `delete` — callbacks vs raw SQL

THE RAILS WAY

- `if post.save == false` → `if post.save` … `else render …`
- Bang methods in controllers with manual rescue → `save` + branch; use `!` in scripts and jobs
- `self.table_name = "posts"` → only when breaking conventions on purpose

SAVE METHODS

METHOD	VALIDATIONS · CALLBACKS · ON FAILURE
`save / update`	yes · yes · returns false
`save! / update! / create!`	yes · yes · raises
`update_attribute`	no · yes · saves anyway
`update_column(s)`	no · no · direct SQL
`touch`	no · after_touch · updates timestamps

STATE PREDICATES

METHOD	TRUE WHEN
`new_record?`	Not yet saved
`persisted?`	Saved and not destroyed
`changed?`	Unsaved attribute changes
`previously_new_record?`	Just created by last save
`saved_change_to_title?`	Last save changed title

⚠ COMMON MISTAKES

- Ignoring the return value of `save`, so failed validations silently do nothing
- Using `update_column` to "fix" validation errors and storing invalid data
- Defining attributes with `attr_accessor` that shadow real columns

✓ TIPS & BEST PRACTICES

- Use `saved_change_to_*?` in `after_save` / `after_commit` callbacks; `*_changed?` is reset by then
- Use `Post.columns_hash` or `Post.attribute_names` to inspect the schema in the console
- Add `annotaterb` (gem) to print schema comments at the top of model files

QUICK SUMMARY

- `Post` ↔ `posts` — convention
- `save` / `save!` — false vs raise
- `update` — assign + save
- `destroy` — delete with callbacks

Migrations — Basics

g migration · create_table · t.references · t.timestamps · add_column · change vs up / down · db:migrate · db:rollback

VERSIONED SCHEMA CHANGES

A **migration** is a Ruby class that changes the database schema. Each file is timestamped and runs once; Rails records it in `schema_migrations`. Most migrations use a single `change` method that Rails knows how to reverse, so the same file both applies and rolls back the change.

MIGRATION FACTS

- Running migrations regenerates `db/schema.rb`; commit both
- Never edit a migration that already ran on a shared database; add a new one
- `t.references :user` adds `user_id`, an index and (with `foreign_key: true`) a FK
- Default primary key is a `bigint id`

DB/MIGRATE/20261023120000_CREATE_POSTS.RB

```ruby
class CreatePosts < ActiveRecord::Migration[8.1]
  def change
    create_table :posts do |t|
      t.string     :title, null: false
      t.text       :body
      t.boolean    :published, default: false, null: false
      t.references :user, null: false, foreign_key: true
      t.timestamps                          # created_at, updated_at
    end
    add_index :posts, :title
  end
end
```

CODE EXPLAINED

- → `Migration[8.1]` — locks behavior to the Rails version that wrote it
- → `null: false` — database-level NOT NULL
- → `foreign_key: true` — real FK constraint
- → `remove_column` with a type — lets `change` reverse it
- → `db:migrate:redo` — quick check that `change` reverses cleanly

CHANGING TABLES

```ruby
class AddSlugToPosts < ActiveRecord::Migration[8.1]
  def change
    add_column :posts, :slug, :string
    add_index  :posts, :slug, unique: true
    rename_column :posts, :body, :content
    remove_column :posts, :legacy_flag, :boolean   # type keeps it reversible
  end
end
```

THE RAILS WAY

- Editing an old migration to fix a column → write a new migration
- Validations only in the model → add `null: false`, unique indexes and FKs too
- Writing `up` / `down` for simple changes → `change` with reversible commands

RUNNING THEM

```
bin/rails db:migrate                    # run pending
bin/rails db:rollback STEP=2            # undo last two
bin/rails db:migrate:status             # up / down list
bin/rails db:migrate VERSION=20261023120000
bin/rails db:migrate:redo               # rollback + migrate
```

COLUMN COMMANDS

COMMAND	REVERSIBLE?
`create_table / drop_table (with block)`	yes
`add_column / remove_column (with type)`	yes
`rename_column / rename_table`	yes
`add_index / remove_index (with columns)`	yes
`change_column`	no: use up/down or change_column_null
`execute "SQL"`	no: use reversible

DATABASE TASKS

TASK	EFFECT
`db:create / db:drop`	Create / drop the DB
`db:prepare`	Create + load schema or migrate
`db:reset`	Drop, create, load schema, seed
`db:schema:load`	Load schema.rb (fresh DB)
`db:seed`	Run db/seeds.rb

⚠ COMMON MISTAKES

- Running `db:reset` on a database with real data; it drops everything
- Renaming a migration class or file after it ran; the version number no longer matches
- Adding a `null: false` column with no default to a table that already has rows

✓ TIPS & BEST PRACTICES

- Use `bin/rails g migration AddXToY x:type` so the body is written for you
- Run `db:migrate:redo` before pushing to prove the migration rolls back
- Use `strong_migrations` (gem) to catch locking or unsafe migrations on big tables

QUICK SUMMARY

- `change` — reversible migration
- `t.references` — FK column + index
- `db:migrate` / `db:rollback` — run / undo
- schema.rb — commit it

Sheet 27

Migrations — Schema, Indexes & Seeds

schema.rb vs structure.sql · add_index · add_foreign_key · change_column_null · reversible · db:prepare · db:seed

KEEPING THE SCHEMA HONEST

The schema file is the authoritative snapshot of the database; new databases load it rather than replaying every migration. Use indexes for lookups and uniqueness, foreign keys and check constraints for integrity, and `reversible` or up/down for changes Rails can't reverse on its own. Seeds load starter data.

SCHEMA NOTES

- `schema.rb` is database-agnostic Ruby; `structure.sql` keeps vendor-specific features (views, triggers)
- Switch with `config.active_record.schema_format = :sql`
- Unique indexes are the only reliable uniqueness guarantee
- Seeds should be idempotent: running twice shouldn't duplicate data

INDEXES & CONSTRAINTS

```ruby
class HardenOrders < ActiveRecord::Migration[8.1]
  def change
    add_index :orders, [:user_id, :created_at]              # composite
    add_index :orders, :number, unique: true
    add_index :orders, :status, where: "status = 'open'"    # partial (PG/SQLite)
    add_foreign_key :orders, :users, on_delete: :cascade
    add_check_constraint :orders, "total >= 0", name: "total_non_negative"
    change_column_null :orders, :email, false, ""           # backfill NULLs
    change_column_default :orders, :status, from: nil, to: "open"
  end
end
```

CODE EXPLAINED

- → Composite index — order matters: it also serves queries on `user_id` alone
- → `where:` — partial index, smaller and faster for hot subsets
- → `change_column_null ..., false, ""` — fill NULLs then add NOT NULL
- → `from:` / `to:` — makes a default change reversible
- → `find_or_create_by!` — idempotent seeds

IRREVERSIBLE PIECES

```ruby
def change
  reversible do |dir|
    dir.up   { execute "CREATE VIEW open_orders AS SELECT * FROM orders WHERE status = 'open'" }
    dir.down { execute "DROP VIEW open_orders" }
  end
end
```

THE RAILS WAY

- `validates :email, uniqueness: true` alone → also `add_index ..., unique: true`
- `Model.create!` in seeds → `find_or_create_by!`
- Running every migration on a fresh machine → `bin/rails db:prepare` (loads the schema)

DB/SEEDS.RB

```ruby
%w[News Tutorials Releases].each do |name|
  Category.find_or_create_by!(name: name)
end
User.find_or_create_by!(email: "admin@example.com") do |u|
  u.password = "change-me-now"
end
```

SCHEMA FORMATS

FORMAT	KEEPS
`schema.rb`	Tables, indexes, FKs, check constraints, enums (PG)
`structure.sql`	Everything the DB can dump: views, triggers, functions

INDEX OPTIONS

OPTION	USE
`unique: true`	Enforce uniqueness
`where:`	Partial index
`order: { created_at: :desc }`	Sort order
`algorithm: :concurrently`	PG: no table lock (with `disable_ddl_transaction!`)
`name:`	Custom (long names are truncated)

⚠ COMMON MISTAKES

- Adding an index on a large PostgreSQL table without `algorithm: :concurrently`, locking writes
- Putting data changes (`Model.update_all`) in schema migrations that later break when the model changes
- Seeds with `create!` that fail on the second run

✓ TIPS & BEST PRACTICES

- For data backfills, use a one-off `script/` file or a job, not a migration
- Add `disable_ddl_transaction!` with concurrent indexes
- Check `db/schema.rb` diffs in code review; surprise changes mean someone's local DB drifted

QUICK SUMMARY

- `schema.rb` — DB snapshot
- `add_index unique:` — real uniqueness
- `reversible` — custom up/down
- `seeds.rb` — idempotent starter data

Column Types & the Attribute API

string / text / decimal / datetime / json · attribute · enum · normalizes (7.1) · store_accessor · serialize · has_json (8.2 edge)

TYPED ATTRIBUTES

Active Record casts column values to Ruby types: `decimal` → `BigDecimal`, `datetime` → `ActiveSupport::TimeWithZone`, `json` → Hash. The **attribute API** adds virtual attributes or overrides a column's type. `enum` maps integers or strings to named states, and `normalizes` cleans values before validation and queries.

TYPE NOTES

- Integer columns cast `"42"` → `42`, `""` → `nil`, `"abc"` → `0` (no error; validate numericality)
- `enum` generates predicates (`draft?`), bang setters (`published!`) and scopes
- `normalizes` also applies to `where(email: ...)` lookups
- Rails 8.2 (edge) adds `has_json` for typed JSON settings

APP/MODELS/USER.RB

```ruby
class User < ApplicationRecord
  enum :role, { member: 0, editor: 1, admin: 2 }, default: :member
  enum :plan, %w[free pro team].index_by(&:itself), prefix: true

  normalizes :email, with: ->(e) { e.strip.downcase }
  attribute :newsletter, :boolean, default: true     # virtual or override
  store_accessor :settings, :theme, :locale          # keys inside a JSON column
  serialize :legacy_prefs, coder: JSON               # text column holding JSON
end
```

CODE EXPLAINED

- → `enum :role, {...}` — Rails 7+ syntax with the name as first argument
- → `default:` — default value for new records
- → `prefix: true` — avoids name clashes (`plan_free?`)
- → `store_accessor` — attribute-style access to JSON keys
- → `serialize ... coder: JSON` — for legacy text columns

USING THEM

```ruby
u = User.new(role: :editor, email: "  Ann@X.COM ")
u.email                    # "ann@x.com"
u.editor?                  # true
u.admin!                   # update(role: :admin)
User.admin                 # scope
User.where.not(role: :member)
User.roles                 # {"member"=>0, "editor"=>1, "admin"=>2}
u.plan_pro?                # prefix: true → plan_*
u.theme = "dark"           # stored in settings JSON
User.find_by(email: "ANN@x.com")   # normalized lookup works
```

THE RAILS WAY

- `STATUSES = %w[draft live]` + string comparisons → `enum :status`
- `before_validation { self.email = email.downcase }` → `normalizes :email`
- Many boolean columns for options → a `jsonb` settings column with `store_accessor`

MIGRATION TYPE → RUBY

COLUMN TYPE	RUBY CLASS
`string / text`	String
`integer / bigint`	Integer
`decimal`	BigDecimal (set `precision:` / `scale:`)
`float`	Float
`boolean`	true / false
`date · datetime`	Date · TimeWithZone
`json / jsonb`	Hash / Array
`uuid (PG)`	String

ATTRIBUTE API

CALL	EFFECT
`attribute :x, :integer`	Typed virtual attribute
`attribute :price, :decimal`	Override column cast
`attribute :tags, default: -> { [] }`	Fresh default each time
`ActiveRecord::Type.register`	Custom type class

⚠ COMMON MISTAKES

- Reordering a hash `enum` mapped to integers; existing rows silently change meaning
- Storing money in `float` columns; use `decimal` with scale 2 or integer cents
- Mutating a JSON attribute in place (`settings["x"] = 1`) without reassigning on old Rails; reassign or use `store_accessor`

✓ TIPS & BEST PRACTICES

- Prefer string-backed enums (`index_by(&:itself)`) for readable data and safe reordering
- Add `validate: true` to `enum` (7.1+) to get validation errors instead of `ArgumentError`
- Use database defaults and `attribute ... default:` together so both SQL and Ruby agree

QUICK SUMMARY

- type casting — columns → Ruby types
- `enum` — named states
- `normalizes` — clean input (7.1)
- `attribute` — virtual/typed attrs

Validations

validates presence / uniqueness / length / format / numericality · validate :custom · errors · valid? · on: :create · validates_with

KEEPING BAD DATA OUT

Validations run when you call `save`, `create`, `update` or `valid?`. If any fail, the record collects messages in `errors` and isn't saved. Rails ships validators for presence, length, format, numericality, uniqueness and more; custom rules go in `validate` methods.

VALIDATION NOTES

- `belongs_to` validates presence of the parent by default
- `uniqueness` runs a SELECT; pair it with a unique index to beat race conditions
- Error messages come from I18n: `activerecord.errors.models.user.attributes.email.taken`
- Skipped by: `update_column`, `insert_all`, `update_all`, `save(validate: false)`

APP/MODELS/USER.RB

```ruby
class User < ApplicationRecord
  validates :name,  presence: true, length: { maximum: 50 }
  validates :email, presence: true, uniqueness: { case_sensitive: false },
                    format: { with: URI::MailTo::EMAIL_REGEXP }
  validates :age,   numericality: { only_integer: true, greater_than_or_equal_to: 13 },
                    allow_nil: true
  validates :terms, acceptance: true, on: :create
  validates :password, confirmation: true, length: { minimum: 12 }, if: :password_digest_changed?
  validate  :birthday_not_in_future

  private

  def birthday_not_in_future
    errors.add(:birthday, "can't be in the future") if birthday&.future?
  end
end
```

CODE EXPLAINED

- → Multiple validators can share one `validates` line
- → `allow_nil: true` — skip the rule when the value is nil
- → `on: :create` — only for new records (also `:update` or a custom context)
- → `if:` / `unless:` — conditional validation
- → `errors.add(:base, ...)` — a message not tied to one field

INSPECTING ERRORS

```ruby
u = User.new(email: "bad")
u.valid?                          # false (runs validations)
u.errors.full_messages            # ["Name can't be blank", "Email is invalid"]
u.errors[:email]                  # ["is invalid"]
u.errors.where(:email).first.type   # :invalid
u.errors.add(:base, "Account locked")
```

THE RAILS WAY

- `if params[:email].blank?` checks in controllers → model validations
- `validates_presence_of :name` (old style) → `validates :name, presence: true`
- Uniqueness validation without an index → add the unique index as the real guard

BUILT-IN VALIDATORS

VALIDATOR	EXAMPLE OPTION
`presence / absence`	`presence: true`
`length`	`{ in: 3..20 }, maximum:`
`format`	`{ with: /\A\d{5}\z/ }`
`numericality`	`{ greater_than: 0 }`
`inclusion / exclusion`	`{ in: %w[S M L] }`
`uniqueness`	`{ scope: :account_id }`
`comparison`	`{ greater_than: :start_date }`
`confirmation / acceptance`	password / terms

CONTEXTS & TRIGGERS

CALL	VALIDATES?
`save / create / update`	yes
`valid?(:publish)`	Custom context
`save(validate: false)`	no
`update_attribute`	no
`insert_all / upsert_all`	no

⚠ COMMON MISTAKES

- Using `^` and `$` in format regexes; they match per line. Use `\A` and `\z`
- Relying on `uniqueness` alone; two concurrent requests can both pass
- Validating presence of a boolean with `presence: true`; `false` is blank. Use `inclusion: { in: [true, false] }`

✓ TIPS & BEST PRACTICES

- Put reusable rules in a custom `EachValidator` class (`validates :url, url: true`)
- Use `validates_with` for rules that span several attributes
- Show `errors.full_messages_for(:field)` next to each input

QUICK SUMMARY

- `validates` — declare rules
- `valid?` / `errors` — check and read
- `validate` — custom method
- `on:` / `if:` — conditional rules

Model Callbacks

HOOKS IN THE RECORD LIFECYCLE

Callbacks run at points in a record's life: before or after validation, save, create, update and destroy. They are useful for derived values and side effects tied to persistence. Use `after_commit` for anything outside the database (jobs, emails, cache), because it only runs once the transaction is really committed.

CALLBACK RULES

- `throw :abort` in a `before_*` callback cancels the save or destroy
- Callbacks run inside the save transaction, except `after_commit` / `after_rollback`
- `after_create_commit`, `after_update_commit`, `after_destroy_commit`, `after_save_commit` are shortcuts
- Bulk methods (`update_all`, `delete_all`, `insert_all`) skip callbacks

APP/MODELS/POST.RB

```ruby
class Post < ApplicationRecord
  before_validation :set_slug, if: -> { slug.blank? }
  before_save       :strip_title
  before_destroy    :prevent_if_published, prepend: true
  after_create_commit  -> { NotifyFollowersJob.perform_later(self) }
  after_update_commit  :expire_cache, if: :saved_change_to_title?
  around_save       :log_timing

  private

  def set_slug     = self.slug = title.to_s.parameterize
  def strip_title  = title&.strip!
  def expire_cache = Rails.cache.delete(["post", id])

  def prevent_if_published
    throw :abort if published?
  end

  def log_timing
    started = Time.current
    yield                                    # runs the save
    Rails.logger.info "saved in #{Time.current - started}s"
  end
end
```

CODE EXPLAINED

- → `before_validation` — set values before rules check them
- → `prepend: true` — run before `dependent: :destroy` callbacks
- → `after_create_commit` — enqueue jobs only after the row is visible to workers
- → `saved_change_to_title?` — dirty check that works after save
- → `around_save` — must `yield`

THE RAILS WAY

- `after_save { Mailer.deliver_later }` → `after_commit` so rolled-back saves don't send mail
- Chains of callbacks doing business workflows → an explicit method or service object
- Callbacks that change other models → be explicit in the caller where it's visible

ORDER ON CREATE

STEP	CALLBACK
1	before_validation
2	after_validation
3	before_save
4	around_save / before_create
5	INSERT
6	after_create
7	after_save
8	after_commit (after transaction)

OTHER HOOKS

CALLBACK	WHEN
`after_initialize`	Every instantiation (incl. loads)
`after_find`	Loaded from DB
`after_touch`	After `touch`
`after_rollback`	Transaction rolled back
`before_destroy`	Before DELETE; can abort

⚠ COMMON MISTAKES

- Enqueuing jobs in `after_save`; the worker may run before commit and not find the record
- Calling `save` inside an `after_save` callback, causing loops or double writes
- Using `after_initialize` for heavy work; it runs for every loaded record

✓ TIPS & BEST PRACTICES

- Keep callbacks about the record itself; orchestrate cross-model work explicitly
- Use `ActiveRecord.after_all_transactions_commit { }` (7.2+) in plain methods for post-commit work
- Inspect a chain with `Post._save_callbacks.map(&:filter)`

QUICK SUMMARY

- `before_*` — prepare or abort
- `throw :abort` — cancel
- `after_commit` — side effects
- bulk methods — skip callbacks

Sheet 31

Scopes & Model Concerns

scope · scopes with arguments · default_scope (and why to avoid it) · unscoped · class methods returning relations · model concerns

NAMED, CHAINABLE QUERIES

A **scope** names a query so you can chain it: `Post.published.recent.limit(5)`. Scopes always return a relation, even when the condition is nil. **Model concerns** group scopes, validations and methods shared by several models, such as soft delete or slugs.

SCOPE NOTES

- A scope is a class method that returns `all` when its body returns `nil`
- Scopes work on associations: `user.posts.published`
- `default_scope` applies to every query, including `new` and associations
- `unscoped` removes default scopes (and everything else in the chain)

APP/MODELS/POST.RB

```ruby
class Post < ApplicationRecord
  include Sluggable

  scope :published, -> { where(published: true) }
  scope :recent,    -> { order(created_at: :desc) }
  scope :by_author, ->(user) { where(user: user) if user }
  scope :since,     ->(time = 1.week.ago) { where(created_at: time..) }

  def self.search(q)                          # class method alternative
    return all if q.blank?
    where("title LIKE ?", "%#{sanitize_sql_like(q)}%")
  end
end
```

CODE EXPLAINED

- → `->(user) { ... if user }` — returns nil → acts as `all`
- → `time..` — endless range: `created_at >= time`
- → `sanitize_sql_like` — escapes `%` and `_` in user input
- → `class_methods do` — class-level methods from a concern
- → `to_param` — models with slugs build `/posts/my-title` URLs

APP/MODELS/CONCERNS/SLUGGABLE.RB

```ruby
module Sluggable
  extend ActiveSupport::Concern

  included do
    before_validation :set_slug, if: -> { slug.blank? }
    validates :slug, uniqueness: true
  end

  class_methods do
    def find_by_slug!(slug) = find_by!(slug: slug)
  end

  def to_param = slug
  private def set_slug = self.slug = title.to_s.parameterize
end
```

THE RAILS WAY

- Repeated `where(published: true)` in controllers → `scope :published`
- `default_scope { where(deleted_at: nil) }` → an explicit `kept` scope
- Copying the same slug code into three models → a concern

CHAINING

```ruby
Post.published.recent.by_author(Current.user).since.limit(10)
Post.published.search(params[:q]).page(2)
```

SCOPE VS CLASS METHOD

	SCOPE · CLASS METHOD
Returns nil	converted to `all` · returns nil (breaks chain)
Arguments	lambda args · normal params
Best for	one-line conditions · multi-step logic

DEFAULT_SCOPE PITFALLS

PROBLEM	RESULT
Applies to `new`	Sets attributes on new records
Applies to joins	Hidden filters in reports
unscoped	Also drops association conditions
Ordering	Breaks `reorder` expectations

⚠ COMMON MISTAKES

- Calling `.first` or `.to_a` inside a scope; it must return a relation
- Writing `where("title LIKE '%#{q}%'")` with interpolation (SQL injection)
- Adding `default_scope` for soft deletes, then fighting it with `unscoped` everywhere

✓ TIPS & BEST PRACTICES

- Name scopes as adjectives or filters (`published` , `by_author`) so chains read like sentences
- Combine with `merge` to reuse another model's scope in a join: `joins(:user).merge(User.active)`
- Keep concerns cohesive: one behavior per concern

QUICK SUMMARY

- `scope` — named, chainable query
- nil → `all` — safe optional filters
- `default_scope` — avoid
- concern — shared model behavior

belongs_to & has_many

THE CORE ASSOCIATION PAIR

`belongs_to` goes on the model whose table holds the foreign key; `has_many` goes on the other side. Together they add readers, writers, builders and query methods: `post.comments.create!(...)`, `comment.post`. Rails infers the class and key names from the association name.

ASSOCIATION FACTS

- `belongs_to` is **required** by default; use `optional: true` to allow nil
- The FK column lives on the `belongs_to` side: `comments.post_id`
- Association readers are cached; call `reload_post` / `post.comments.reload` to refresh
- `dependent:` decides what happens to children when the parent is destroyed

MODELS

```ruby
class Post < ApplicationRecord
  belongs_to :author, class_name: "User", foreign_key: "user_id"
  has_many   :comments, dependent: :destroy
  has_many   :approved_comments, -> { where(approved: true) }, class_name: "Comment"
end

class Comment < ApplicationRecord
  belongs_to :post, counter_cache: true          # needs posts.comments_count
  belongs_to :parent, class_name: "Comment", optional: true
end
```

CODE EXPLAINED

- → `class_name:` / `foreign_key:` — when the name doesn't match the class
- → Scoped `has_many` — a lambda adds conditions to an association
- → `counter_cache: true` — keeps `posts.comments_count` updated
- → `build` vs `create!` — memory only vs saved
- → `size` — count query, cached count, or loaded length, whichever is cheapest

GENERATED METHODS

```ruby
post.comments                        # relation (lazy)
post.comments.create!(body: "Nice") # sets post_id
post.comments.build(body: "Draft")  # unsaved, in memory
post.comments.size                   # uses counter cache when present
post.comments.where(approved: false).count
post.comment_ids                     # [1, 4, 9]
comment.post                         # parent (cached)
comment.post = other_post            # reassign
comment.build_post(title: "x")       # belongs_to builder
Post.where.associated(:comments)     # posts with at least one comment
```

THE RAILS WAY

- `Comment.where(post_id: post.id)` → `post.comments`
- `Comment.create(post_id: post.id, ...)` → `post.comments.create(...)`
- `post.comments.count` in a loop → a `counter_cache` column

DEPENDENT: OPTIONS

OPTION	ON PARENT DESTROY
`:destroy`	Load and destroy each child (callbacks run)
`:delete_all`	One SQL DELETE (no callbacks)
`:destroy_async`	Destroy children in a background job
`:nullify`	Set child FKs to NULL
`:restrict_with_error`	Add error, abort destroy
`:restrict_with_exception`	Raise, abort destroy

COMMON OPTIONS

OPTION	SIDE
`optional: true`	belongs_to
`touch: true`	belongs_to: bump parent `updated_at`
`inverse_of:`	both: share in-memory objects
`strict_loading: true`	both: forbid lazy loads
`through:`	has_many (sheet 33)

⚠ COMMON MISTAKES

- Adding `has_many :comments` but forgetting the `post_id` column (use `t.references`)
- Using `dependent: :destroy` on huge collections; it loads every row. Consider `:destroy_async` or `:delete_all`
- Adding `counter_cache` to an existing table without backfilling: `Post.reset_counters(id, :comments)`

✓ TIPS & BEST PRACTICES

- Pair `dependent:` with a DB `on_delete:` FK rule to protect against raw SQL deletes
- Use `touch: true` on `belongs_to` to keep parent cache keys fresh
- Check generated methods in the console: `Post.reflect_on_association(:comments)`

QUICK SUMMARY

- `belongs_to` — has the FK
- `has_many` — the collection side
- `dependent:` — child cleanup
- `counter_cache` — cheap counts

Sheet 33

has_one, :through & HABTM

has_one · has_many through: · has_one through: · source: · join models · has_and_belongs_to_many

ONE-TO-ONE AND MANY-TO-MANY

has_one is has_many limited to one record. For many-to-many, has_many :through goes via a **join model** that can hold its own data (role, position, timestamps).

has_and_belongs_to_many uses a bare join table with no model; it's simpler but can't grow.

NOTES

- has_one keeps the FK on the other table, like has_many
- Join model tables: memberships(user_id, team_id, role)
- HABTM table name is both plurals in alphabetical order: posts_tags
- Assigning user.team_ids = [1, 2] writes join rows for you

MODELS

```ruby
class User < ApplicationRecord
  has_one  :profile, dependent: :destroy
  has_many :memberships, dependent: :destroy
  has_many :teams, through: :memberships
  has_many :owned_teams, -> { where(memberships: { role: "owner" }) },
           through: :memberships, source: :team
  has_one  :address, through: :profile          # has_one :through
end

class Membership < ApplicationRecord
  belongs_to :user
  belongs_to :team
  validates :user_id, uniqueness: { scope: :team_id }
end

class Team < ApplicationRecord
  has_many :memberships, dependent: :destroy
  has_many :users, through: :memberships
end

class Post < ApplicationRecord
  has_and_belongs_to_many :tags          # table posts_tags, no model
end
```

CODE EXPLAINED

→ create_profile! — has_one gets build_ / create_ methods

→ through: :memberships — query hops through the join model

→ source: :team — which association on the join model to follow

→ << — saves the join row immediately

→ HABTM — no model, no extra columns, no callbacks on the join

USING THEM

```ruby
user.create_profile!(bio: "Hi")              # has_one builder
user.teams << team                           # creates a Membership
user.memberships.create!(team: team, role: "owner")
user.owned_teams                             # through with a condition
team.users.where(memberships: { role: "admin" })
post.tag_ids = [1, 3]                        # rewrites posts_tags
```

THE RAILS WAY

- HABTM for anything that might get attributes → has_many :through from day one
- Querying the join table with raw SQL → filter via where(memberships: {...})
- Uniqueness only in Ruby → add a unique index on [user_id, team_id]

WHICH ASSOCIATION?

RELATIONSHIP	USE
User has one Profile	has_one / belongs_to
Post has many Comments	has_many / belongs_to
Users ↔ Teams with role	has_many :through
Posts ↔ Tags, nothing else	has_many :through or HABTM
Doctor → Patients via Appointments	has_many :through

JOIN MIGRATION

COMMAND	CREATES
g model Membership user:references team:references role	join model table
g migration CreateJoinTablePostTag post tag	HABTM table posts_tags
add_index ... unique: true	prevent duplicate pairs

⚠ COMMON MISTAKES

- Putting the FK on the wrong side of has_one (it belongs on profiles.user_id)
- Forgetting dependent: :destroy on the join association and leaving orphan memberships
- Using source: when the names already match, or omitting it when they don't (HasManyThroughSourceAssociationNotFoundError)

✓ TIPS & BEST PRACTICES

- Name join models after the relationship (Membership, Enrollment), not UserTeam
- Use has_many :through with -> { distinct } when the same record can be reached twice
- For ordered lists, add a position column to the join model

QUICK SUMMARY

- has_one — single child
- has_many :through — join model
- source: — follow a different name
- HABTM — bare join table

Sheet 34

Polymorphic, STI & Delegated Types

polymorphic: true · as: · _type column · single table inheritance · type column · delegated_type

ONE ASSOCIATION, MANY CLASSES

A **polymorphic** `belongs_to` lets one model attach to several others: comments on posts *and* photos, stored as `commentable_type` + `commentable_id`. **Single table inheritance** stores subclasses in one table with a `type` column. **Delegated types** keep shared columns in one table and type-specific columns in their own tables.

NOTES

- Polymorphic FKs can't have database FK constraints
- STI subclasses share every column; unused ones stay NULL
- `delegated_type` (6.1+) is Basecamp's approach for feeds of mixed records
- The `type` column stores the class name string

POLYMORPHIC

```ruby
class Comment < ApplicationRecord
  belongs_to :commentable, polymorphic: true    # commentable_type, commentable_id
end
class Post  < ApplicationRecord; has_many :comments, as: :commentable; end
class Photo < ApplicationRecord; has_many :comments, as: :commentable; end

# migration: t.references :commentable, polymorphic: true, null: false
photo.comments.create!(body: "Wow")
comment.commentable                           # => #<Photo id: 3>
```

SINGLE TABLE INHERITANCE

```ruby
class Vehicle < ApplicationRecord; end          # table vehicles, column type
class Car   < Vehicle; end
class Truck < Vehicle
  validates :payload, presence: true
end

Car.create!(name: "Civic")                      # type = "Car"
Vehicle.all                                     # Cars and Trucks
Truck.all                                       # WHERE type = 'Truck'
```

DELEGATED TYPE

```ruby
class Entry < ApplicationRecord                 # entryable_type, entryable_id
  delegated_type :entryable, types: %w[Message Comment], dependent: :destroy
end
class Message < ApplicationRecord
  has_one :entry, as: :entryable, touch: true
end

Entry.messages                                  # scope per type
entry.message?                                  # predicate
Entry.create!(entryable: Message.new(subject: "Hi"), account: acct)
```

CODE EXPLAINED

- → `polymorphic: true` — two columns instead of one FK
- → `as: :commentable` — the other side of a polymorphic association
- → STI — subclass queries add `WHERE type = ...` automatically
- → `delegated_type` — shared table for listing, own tables for details
- → Delegated types avoid STI's sparse NULL columns

THE RAILS WAY

- `post_id` + `photo_id` nullable columns on comments → one polymorphic association
- STI with many type-specific columns → delegated types
- Case statements on `type` strings → subclass methods (polymorphism)

CHOOSING

NEED	USE
Child attaches to many parent types	polymorphic belongs_to
Subclasses with nearly identical columns	STI
Mixed types listed together, different columns	delegated_type
Shared behavior only, separate tables	concern

GOTCHAS

ISSUE	FIX
Renaming a class	Update stored `*_type` / `type` values
No FK on polymorphic	Validate presence, clean up in callbacks
Column named `type` but not STI	`self.inheritance_column = nil`
Index for polymorphic	`[commentable_type, commentable_id]`

⚠ COMMON MISTAKES

- Naming a regular column `type`, which triggers STI and raises `SubclassNotFound`
- Eager loading a polymorphic association with `joins`; it can't join multiple tables. Use `includes` / `preload`
- Huge STI tables where each subclass uses different columns

✓ TIPS & BEST PRACTICES

- Override `sti_name` and `find_sti_class` to keep stored type names stable across class renames
- Add `polymorphic: true, index: true` in migrations (the default index covers type + id)
- Start with delegated types for activity feeds or timelines

QUICK SUMMARY

- `polymorphic: true` — many parent types
- STI — subclasses in one table
- `delegated_type` — shared + specific tables
- `as:` — the has_many side

Sheet 35

Association Methods & Options

build · create · << · destroy_all · class_name: · touch: · strict_loading · deprecated: true (8.1)

WORKING WITH COLLECTIONS

Every association returns a **CollectionProxy** that behaves like a relation and an array. Some methods save immediately (<<, `create`), others only in memory (`build`, `new`). Options control naming, cleanup and loading. Rails 8.1 can mark associations **deprecated** to find remaining callers before removing them.

NOTES

- `<<` on a saved parent saves the child immediately
- `collection.delete(obj)` follows `dependent:`; with no option it nullifies the FK
- `strict_loading` raises when an association loads lazily (N+1 guard)
- `deprecated: true` (8.1) warns, raises or logs on every use

COLLECTION METHODS

```
post.comments << Comment.new(body: "Hi")          # saves now
post.comments.build(body: "Later")                # memory only
post.comments.create!(body: "Saved")
post.comments.find(5)                             # scoped to this post
post.comments.exists?(approved: false)
post.comments.destroy(comment)                    # one, with callbacks
post.comments.destroy_all                         # all, with callbacks
post.comments.delete_all                          # SQL, follows dependent:
post.comments.clear                               # remove all from collection
post.comments.loaded?                             # already in memory?
post.comments.load_async                          # start query in background
```

CODE EXPLAINED

- → `find` on an association — 404s if the comment isn't on this post
- → `destroy` vs `delete_all` — callbacks vs single SQL statement
- → `load_async` — run the query in a thread while the action continues
- → `strict_loading: true` — forces you to preload with `includes`
- → `deprecated: true` — modes `:warn`, `:raise`, `:notify`

OPTIONS ON DECLARATIONS

```
class Post < ApplicationRecord
  belongs_to :user, touch: true, inverse_of: :posts
  has_many   :comments, -> { order(:created_at) }, strict_loading: true
  has_many   :old_comments, class_name: "Comment", deprecated: true   # 8.1
end

# config/application.rb (8.1): how deprecated associations report
config.active_record.deprecated_associations_options = { mode: :warn, backtrace: true }
```

THE RAILS WAY

- `Comment.find(params[:id])` for a nested resource → `@post.comments.find(params[:id])`
- Deleting rows in a loop → `destroy_all` or `delete_all`
- Grepping for an old association before removal → mark it `deprecated: true` and watch the logs

SAVES IMMEDIATELY?

METHOD	SAVED?
`<< / push / concat`	yes (if parent saved)
`create / create!`	yes
`build / new`	no, saved with parent
`collection_ids = [...]`	yes
`replace(records)`	yes

DECLARATION OPTIONS

OPTION	PURPOSE
`class_name:`	Different class
`foreign_key:`	Different FK column
`primary_key:`	Match on another column
`inverse_of:`	Bidirectional in-memory link
`touch:`	Bump parent timestamps
`autosave: true`	Save changed children with parent
`validate: false`	Skip child validation

⚠ COMMON MISTAKES

- Calling `post.comments.build` and then `render`, which shows the unsaved comment in lists
- Using `.count` on a loaded collection (extra query); use `.size`
- Expecting `delete` to run callbacks; it doesn't

✓ TIPS & BEST PRACTICES

- Turn on `config.active_record.strict_loading_by_default` in development to surface N+1s
- Use `inverse_of:` when Rails can't infer it (custom `foreign_key`, scopes)
- Use `load_async` for independent queries in the same action

QUICK SUMMARY

- `<<` / `create` — save now
- `build` — save later
- `strict_loading` — no lazy loads
- `deprecated: true` — retire safely (8.1)

Query Basics

ASKING THE DATABASE

Query methods return an `ActiveRecord::Relation`, which is **lazy**: the SQL runs only when you iterate, call `to_a`, or use a terminal method like `first`, `count` or `pluck`. Relations chain, so you can build a query step by step.

QUERY FACTS

- `find` raises `RecordNotFound`; `find_by` returns `nil`
- `first` / `last` order by primary key when there's no order
- `take` adds `LIMIT 1` with no ordering
- `pluck` returns plain values without building model objects

FINDING & FILTERING

```
Post.find(1)                              # raise if missing
Post.find([1, 2, 3])                      # array
Post.find_by(slug: "hello")               # nil if missing
Post.find_by!(slug: "hello")              # raise if missing
Post.where(published: true)               # relation
Post.where(published: true).where.not(user_id: nil)
Post.order(created_at: :desc).limit(10).offset(20)
Post.first; Post.last(3); Post.take
Post.pluck(:id, :title)                   # [[1, "Hi"], ...]
Post.pick(:title)                         # first value only
Post.ids                                  # [1, 2, 3]
Post.exists?(slug: "hello")               # true / false
Post.where(user: user).any?               # SELECT 1 ... LIMIT 1
Post.select(:id, :title)                  # partial objects
Post.distinct.pluck(:user_id)
Post.find_or_create_by!(slug: "about") { |p| p.title = "About" }
```

CODE EXPLAINED

- → `where(user: user)` — accepts a model; uses its id
- → `order(created_at: :desc)` — hash form avoids SQL strings
- → `pick` — `pluck` + `first` in one query
- → `exists?` / `any?` — cheapest way to check for rows
- → `find_or_create_by!` — block runs only when creating

SEEING THE SQL

```
Post.where(published: true).to_sql
# SELECT "posts".* FROM "posts" WHERE "posts"."published" = 1
Post.where(published: true).explain
```

THE RAILS WAY

- `Post.all.select { |p| p.published }` (Ruby) → `Post.where(published: true)` (SQL)
- `Post.all.map(&:id)` → `Post.ids` or `pluck(:id)`
- `Post.where(...).count > 0` → `.exists?`

TERMINAL METHODS (RUN SQL)

METHOD	RETURNS
`to_a / each / map`	Model instances
`first / last / take`	One record or nil
`count / sum / average`	Number
`pluck / pick / ids`	Plain values
`exists? / any? / none?`	Boolean
`find_each`	Batches (sheet 40)

CHAINABLE METHODS

METHOD	ADDS
`where / rewhere`	WHERE
`order / reorder`	ORDER BY
`limit / offset`	LIMIT / OFFSET
`select / distinct`	SELECT columns
`joins / includes`	Associations
`group / having`	GROUP BY
`none`	Empty relation, no query

⚠ COMMON MISTAKES

- Calling `.all` and filtering in Ruby; it loads every row
- Using `find_by` and forgetting it can return `nil` → `NoMethodError` later
- Expecting `last` to mean "newest" without an order on a non-sequential key (UUIDs)

✓ TIPS & BEST PRACTICES

- Use `Post.none` to return an empty chainable relation from a method
- Set `self.implicit_order_column = "created_at"` for UUID primary keys
- Use `.load_async` in controllers for slow independent queries

QUICK SUMMARY

- Relation — lazy, chainable
- `find` / `find_by` — raise vs nil
- `pluck` — values, not objects
- `exists?` — cheap existence check

Conditions in Depth

WRITING WHERE CLAUSES SAFELY

Hash conditions cover most needs: equality, IN lists, ranges and NULL. When you need SQL fragments, pass values through **placeholders** so Rails quotes them. Never interpolate user input into a SQL string.

CONDITION NOTES

- `where(status: nil)` → `IS NULL`; `where(id: [1, 2])` → `IN (1, 2)`
- Ranges: `1..10` → `BETWEEN`; endless `10..` → `>= 10`
- `where.not` with NULLs: `NOT IN` excludes NULL rows; add `.or(where(x: nil))` if needed
- Conditions on joined tables use the table name as a hash key

HASH & RANGE CONDITIONS

```
Post.where(status: "live", user_id: [1, 2, 3])
Post.where(deleted_at: nil)                       # IS NULL
Post.where(created_at: 1.week.ago..)              # >= 1 week ago
Post.where(views: ..100)                          # <= 100
Post.where(published_on: Date.current.all_month)  # BETWEEN month start/end
Post.where.not(status: "draft")
Post.where(status: "live").or(Post.where(featured: true))
Post.joins(:user).where(users: { admin: true })
Post.where.missing(:comments)                     # LEFT JOIN … IS NULL
Post.where.associated(:author)                    # INNER JOIN, not null
```

CODE EXPLAINED

- → `all_month` — Active Support range helper for dates
- → `or` — both sides must have compatible joins/selects
- → `where.missing` / `where.associated` — no hand-written LEFT JOIN
- → `?` / `:named` — Rails quotes and escapes the values
- → `Arel.sql` — mark a trusted raw ordering string

SQL FRAGMENTS WITH PLACEHOLDERS

```
Post.where("title LIKE ?", "%#{Post.sanitize_sql_like(q)}%")
Post.where("views > :min AND views < :max", min: 10, max: 100)
Post.where("lower(email) = ?", email.downcase)
Post.where(Post.arel_table[:views].gt(100))       # Arel
Post.order(Arel.sql("FIELD(status, 'live', 'draft')"))

# NEVER: Post.where("title = '#{params[:title]}'")
```

THE RAILS WAY

- String conditions for simple equality → hash conditions
- `"created_at > '#{date}'"` → `where(created_at: date..)`
- Manual `LEFT OUTER JOIN ... WHERE id IS NULL` → `where.missing(:assoc)`

HASH → SQL

RUBY	SQL
`where(a: 1)`	`a = 1`
`where(a: [1, 2])`	`a IN (1, 2)`
`where(a: nil)`	`a IS NULL`
`where(a: 1..5)`	`a BETWEEN 1 AND 5`
`where(a: 1...5)`	`a >= 1 AND a < 5`
`where.not(a: 1)`	`a != 1`
`where(users: { x: 1 })`	`users.x = 1`

INJECTION-SAFE HELPERS

HELPER	USE
`sanitize_sql_like`	Escape % and _
`sanitize_sql_array`	Build a fragment safely
`connection.quote`	Quote one value
`Arel.sql`	Trusted raw SQL only

⚠ COMMON MISTAKES

- Interpolating `params` into `where` or `order` strings → SQL injection
- Passing user input to `order(params[:sort])`; allow-list sort columns instead
- Forgetting NULL rows disappear from `where.not(status: "draft")`

✓ TIPS & BEST PRACTICES

- Allow-list sorting: `order(%w[title created_at].include?(s) ? s : :created_at)`
- Prefer `where(..)` ranges over string comparisons for dates and numbers
- Run Brakeman (`bin/brakeman`) to catch unsafe SQL strings

QUICK SUMMARY

- hash conditions — `=`, `IN`, `NULL`, ranges
- `where.not` / `or` — negate / combine
- `?` placeholders — quote input
- `where.missing` — no-child records

Joins & Eager Loading

ASSOCIATIONS IN QUERIES

`joins` uses an association to **filter** in SQL. `includes`, `preload` and `eager_load` **load** associated records up front, avoiding the **N+1 problem**: one query for posts plus one per post for its author. Pick based on whether you need to filter by the association, display it, or both.

NOTES

- `joins` is INNER JOIN; records without the association disappear
- `includes` picks `preload` (separate queries) or `eager_load` (one LEFT JOIN)
- `includes` switches to `eager_load` when you `references` or filter on the table
- Nested: `includes(comments: :author)`

N+1 AND THE FIX

```ruby
# N+1: 1 query for posts + 1 per post for its author
Post.limit(20).each { |p| puts p.author.name }

# 2 queries total
Post.includes(:author).limit(20).each { |p| puts p.author.name }
```

JOINING VS LOADING

```ruby
Post.joins(:author).where(users: { verified: true })      # filter only
Post.joins(:comments).distinct                            # posts with comments
Post.left_outer_joins(:comments).where(comments: { id: nil })
Post.includes(:author, comments: :user)                   # display nested
Post.preload(:tags)                                       # always separate queries
Post.eager_load(:author).where(users: { verified: true }) # one JOIN query
Post.includes(:author).where(users: { verified: true }).references(:users)
Post.joins(:author).merge(User.verified)                  # reuse a scope
```

GUARD AGAINST N+1

```ruby
Post.strict_loading.first.author     # raises StrictLoadingViolationError
user.posts.strict_loading!           # on one relation
```

CODE EXPLAINED

- → `joins` — no association objects loaded, just filtering
- → `distinct` — avoid duplicate posts from one-to-many joins
- → `preload` — `WHERE id IN (...)` per association
- → `eager_load` — single query with LEFT OUTER JOIN
- → `merge` — apply another model's scope to the joined table

THE RAILS WAY

- Looping and calling an association per row → `includes` up front
- `joins` + reading `post.author` in the view → `includes` (joins doesn't load)
- Hunting N+1s by reading logs → `strict_loading` in dev or the Bullet gem

WHICH METHOD?

NEED	METHOD
Filter by association	joins
Include rows with no association	left_outer_joins
Show association data	includes / preload
Filter and show	eager_load or includes + references
Count per parent	joins + group + count, or counter cache

SQL PRODUCED

CALL	QUERIES
`joins(:author)`	1 · INNER JOIN
`preload(:author)`	2 · posts, then users IN (…)
`eager_load(:author)`	1 · LEFT OUTER JOIN
`includes(:author)`	2 (or 1 if referenced)

⚠ COMMON MISTAKES

- Using `includes` then `.where("users.verified = 1")` without `references(:users)`, giving "no such column"
- Over-eager loading associations the page never shows
- Forgetting `distinct` after joining a `has_many`, which multiplies rows

✓ TIPS & BEST PRACTICES

- Enable `config.active_record.strict_loading_by_default = true` in development and fix each error
- Use counter caches instead of loading collections just to count them
- Check the log: lines tagged `CACHE` or repeated `SELECT users` hint at N+1

QUICK SUMMARY

- `joins` — filter
- `includes` — load to display
- `eager_load` — one JOIN query
- `strict_loading` — catch N+1

Calculations & Grouping

count · sum · average · minimum / maximum · group · having · select with alias · distinct

LET THE DATABASE DO THE MATH

Calculation methods run aggregate SQL and return numbers or hashes, without loading records. Add `group` to get a hash keyed by the grouped value. For several aggregates at once, `select` with aliases and read them as attributes.

NOTES

- `count` → Integer; `average` → BigDecimal; `sum` on empty → `0`
- Grouped calculations return a Hash: `{ "draft" => 3, "live" => 9 }`
- `count` on a loaded relation still queries; `size` uses the loaded array
- `having` filters groups after aggregation

SINGLE VALUES

```ruby
Order.count                                # 120
Order.where(status: "paid").sum(:total)    # 4821.50
Order.average(:total)                      # 40.18 (BigDecimal)
Order.minimum(:created_at)
Order.maximum(:total)
Order.distinct.count(:user_id)             # unique customers
Order.calculate(:sum, :total)              # generic form
Order.async_sum(:total)                    # Promise (7.1+): .value later
```

GROUPING

```ruby
Order.group(:status).count                 # {"paid"=>90, "open"=>30}
Order.group(:user_id).sum(:total)          # {1=>120.0, 2=>87.5}
Order.group(:user_id).having("SUM(total) > ?", 100).sum(:total)
Order.group("DATE(created_at)").count      # per day
Order.joins(:user).group("users.country").count
Post.left_joins(:comments).group(:id).count("comments.id")
```

MULTIPLE AGGREGATES

```ruby
stats = Order.select("user_id, COUNT(*) AS orders_count, SUM(total) AS spent")
             .group(:user_id).order(spent: :desc).limit(5)
stats.first.spent                          # alias readable as attribute
```

CODE EXPLAINED

- → `distinct.count(:col)` — `COUNT(DISTINCT col)`
- → `async_sum` — run in the background, read `.value` when needed
- → `having` — conditions on aggregated values
- → `left_joins` + `count("comments.id")` — zero counts kept
- → Aliased columns become readers on the returned objects

THE RAILS WAY

- `Order.all.map(&:total).sum` → `Order.sum(:total)`
- Counting children in a loop → `group(...).count` once
- Frequent counts on big tables → counter caches or a rollup table

CALCULATION METHODS

METHOD	SQL
`count`	`COUNT(*)`
`count(:col)`	`COUNT(col)` (non-null)
`sum(:col)`	`SUM(col)`
`average(:col)`	`AVG(col)`
`minimum / maximum`	`MIN` / `MAX`
`pluck(Arel.sql("..."))`	Any expression

COUNT VS SIZE VS LENGTH

METHOD	BEHAVIOR
`count`	Always runs COUNT query
`size`	Counter cache, or length if loaded, else COUNT
`length`	Loads all records, counts array
`empty? / any?`	Cheap existence check

⚠ COMMON MISTAKES

- Calling `.length` on an unloaded relation, which loads every row just to count
- Grouping by a column but selecting others not in GROUP BY (PostgreSQL rejects it)
- Treating `average` results as Floats in JSON; convert with `.to_f` or format them

✓ TIPS & BEST PRACTICES

- Use `group_by_day` etc. from the `groupdate` gem for time-series charts
- Add an index on grouped and filtered columns used in dashboards
- Cache expensive aggregates with `Rails.cache.fetch` keyed on `maximum(:updated_at)`

QUICK SUMMARY

- `count / sum / average` — aggregates
- `group(...).count` — hash per group
- `having` — filter groups
- `size` — cheapest count

Batches & Bulk Operations

LARGE DATA WITHOUT LARGE MEMORY

Iterating `Model.all.each` loads every row into memory. **Batch** methods walk the table in chunks of 1,000 by primary key. **Bulk** methods write many rows in one SQL statement; they are much faster but skip validations and callbacks, so use them for imports, backfills and maintenance.

NOTES

- Batches order by primary key; custom `order` is ignored (or raises with `error_on_ignored_order`)
- `insert_all` / `upsert_all` take an array of hashes and skip models entirely
- `update_all` / `delete_all` run one statement: no callbacks, no `updated_at`
- `in_batches` yields relations, so you can call `update_all` per batch

BATCHES

```
User.find_each { |u| u.recalculate! }                 # 1,000 at a time
User.where(active: true).find_each(batch_size: 500) { |u| u.sync! }
User.find_each(start: 10_000, finish: 20_000) { |u| u.sync! } # id range
User.find_in_batches { |users| Exporter.write(users) } # arrays
User.in_batches(of: 5_000) { |rel| rel.update_all(flag: true) }
User.in_batches.each_record { |u| u.touch }
```

BULK WRITES

```
Tag.insert_all([{ name: "ruby" }, { name: "rails" }])    # skip dups by unique index
Tag.insert_all!([{ name: "new" }])                       # raise on conflict
Price.upsert_all(rows, unique_by: :sku)                  # insert or update
Post.where(published: false).update_all(status: "draft")
Post.where("created_at < ?", 1.year.ago).delete_all
Post.where(user: user).touch_all                         # updated_at = now
Post.update_counters(post.id, views_count: 1)            # atomic += 1
Post.where(id: ids).update_all("views = views + 1")
```

CODE EXPLAINED

- → `find_each` — yields records one by one, loaded in batches
- → `start:` / `finish:` — split work across processes
- → `insert_all` — `INSERT ... ON CONFLICT DO NOTHING` style
- → `upsert_all` — needs a unique index named by `unique_by:`
- → `update_counters` — race-free increment in SQL

THE RAILS WAY

- `User.all.each` over a big table → `User.find_each`
- Creating 10,000 rows with `create!` in a loop → `insert_all` in chunks
- `post.update(views: post.views + 1)` → `Post.update_counters` (no lost updates)

BULK METHOD BEHAVIOR

METHOD	VALIDATIONS · CALLBACKS · TIMESTAMPS
`insert_all / upsert_all`	no · no · yes (set automatically)
`update_all`	no · no · no
`delete_all`	no · no · —
`destroy_all`	yes · yes · — (loads each row)
`touch_all`	no · no · yes

BATCH OPTIONS

OPTION	MEANING
`batch_size: / of:`	Rows per batch
`start: / finish:`	Primary key bounds
`order: :desc`	Walk backward
`use_ranges: true`	Range queries for in_batches
`load: true`	in_batches with loaded records

⚠ COMMON MISTAKES

- Adding `.order(:name)` before `find_each` and expecting that order
- Using `update_all` and expecting cache keys to change; it doesn't touch `updated_at`
- Running a 2-million-row `update_all` in one statement and locking the table; batch it

✓ TIPS & BEST PRACTICES

- Wrap long backfills in a job using `in_batches` so they can be retried
- Pass `returning: [:id]` to `insert_all` on PostgreSQL to get new ids
- Log progress inside batch loops; long silent scripts are hard to trust

QUICK SUMMARY

- `find_each` — memory-safe iteration
- `insert_all / upsert_all` — bulk insert
- `update_all / delete_all` — one SQL statement
- bulk = no callbacks

Transactions & Locking

ALL OR NOTHING

A **transaction** groups writes so they all succeed or all roll back. Any exception inside the block rolls back and re-raises; `ActiveRecord::Rollback` rolls back silently. **Locking** prevents two requests from changing the same row at once: pessimistic locks block, optimistic locks detect conflicts on save.

NOTES

- `save` and `destroy` already run in their own transaction
- Nested `transaction` blocks join the outer one unless `requires_new: true`
- Jobs, emails and API calls inside a transaction can't be rolled back: use `after_commit`
- Optimistic locking needs an integer `lock_version` column

TRANSACTIONS

```ruby
ActiveRecord::Base.transaction do
  from.update!(balance: from.balance - amount)
  to.update!(balance: to.balance + amount)
  Transfer.create!(from:, to:, amount:)
end                                          # any raise → full rollback

Order.transaction do
  order.update!(status: "cancelled")
  raise ActiveRecord::Rollback if order.refund_failed?   # silent rollback
end

ActiveRecord::Base.transaction do |tx|
  order.save!
  tx.after_commit { OrderMailer.confirm(order).deliver_later }  # 7.2+
end
```

CODE EXPLAINED

- → `update!` (bang) — failures raise, so the transaction rolls back
- → `ActiveRecord::Rollback` — swallowed by `transaction`, not re-raised
- → `tx.after_commit` — run code only if this transaction commits
- → `with_lock` — row lock for the duration of the block
- → `StaleObjectError` — someone else saved first; reload and retry or show a conflict

LOCKING

```ruby
account = Account.lock.find(id)              # SELECT ... FOR UPDATE
account.with_lock do                         # reload + lock + transaction
  account.balance -= 10
  account.save!
end

# optimistic: posts.lock_version integer, default 0, null: false
post = Post.find(1)
post.update(title: "A")                      # lock_version 0 → 1
stale.update(title: "B")                     # raises ActiveRecord::StaleObjectError
```

THE RAILS WAY

- Non-bang `update` inside a transaction → `update!` so failures abort
- Sending email inside the block → `after_commit` or `tx.after_commit`
- Read-modify-write without locking → `with_lock` or `update_counters`

PESSIMISTIC VS OPTIMISTIC

	PESSIMISTIC · OPTIMISTIC
Mechanism	FOR UPDATE row lock · lock_version check
On conflict	Second request waits · StaleObjectError
Best for	Short critical sections, money · Forms edited by people
Cost	Blocking, deadlock risk · Retry / merge UI

TRANSACTION OPTIONS

OPTION	EFFECT
`requires_new: true`	Savepoint for nested block
`isolation: :serializable`	Isolation level (outer only)
`joinable: false`	Inner blocks can't join
`lock("FOR UPDATE NOWAIT")`	Fail fast instead of waiting

⚠ COMMON MISTAKES

- Using `save` (non-bang) inside a transaction and continuing after it returns false
- Doing slow network calls while holding a row lock
- Rescuing exceptions inside the block, which prevents the rollback

✓ TIPS & BEST PRACTICES

- Keep transactions short and focused on the database
- Use `ActiveRecord.after_all_transactions_commit` from service objects (7.2+)
- Add a `lock_version` hidden field in edit forms to detect concurrent edits

QUICK SUMMARY

- `transaction` — all or nothing
- `Rollback` — silent abort
- `with_lock` — pessimistic
- `lock_version` — optimistic

Raw SQL & Multiple Databases

BELOW THE ORM, AND BEYOND ONE DATABASE

When a query is easier in SQL, `find_by_sql` returns models and `select_all` returns rows. Rails also supports **multiple databases**: separate primary databases (Solid Queue, Cache and Cable use this), read **replicas** with automatic switching, and horizontal **shards**.

NOTES

- Rails 8 production config already has `primary`, `cache`, `queue` and `cable` databases
- Models choose a database through an abstract base class with `connects_to`
- Automatic role switching sends GETs to the replica and writes to the primary
- Each database has its own migrations folder (`db/queue_migrate`, etc.)

RAW SQL

```
Post.find_by_sql(["SELECT * FROM posts WHERE views > ? LIMIT 5", 100])
rows = ActiveRecord::Base.connection.select_all(
  "SELECT status, COUNT(*) AS n FROM posts GROUP BY status")
rows.to_a                       # [{"status"=>"live", "n"=>9}, ...]
ActiveRecord::Base.lease_connection.execute("VACUUM")
Post.sanitize_sql_array(["title = ?", t])
```

CODE EXPLAINED

- → `find_by_sql` — returns model instances, with extra selected columns as attributes
- → `select_all` — `ActiveRecord::Result`, plain hashes
- → `replica: true` — read-only, no migrations
- → `connected_to(role: :reading)` — force queries to the replica
- → `database_selector` — reads go to the replica unless you wrote recently

CONFIG/DATABASE.YML (PRODUCTION EXCERPT)

```
production:
  primary:
    <<: *default
    database: storage/production.sqlite3
  primary_replica:
    <<: *default
    database: storage/production_replica.sqlite3
    replica: true
  queue:
    <<: *default
    database: storage/production_queue.sqlite3
    migrations_paths: db/queue_migrate
```

THE RAILS WAY

- String-built SQL with interpolation → `find_by_sql([sql, *binds])`
- Heavy reports on the primary → `connected_to(role: :reading)`
- One giant database for jobs and cache → separate Solid databases (the Rails 8 default)

MODELS & SWITCHING

```
class ApplicationRecord < ActiveRecord::Base
  primary_abstract_class
  connects_to database: { writing: :primary, reading: :primary_replica }
end

ActiveRecord::Base.connected_to(role: :reading) { Report.heavy_query }
# config/environments/production.rb
config.active_record.database_selector = { delay: 2.seconds }
```

DB TASKS WITH MULTIPLE DBS

TASK	EFFECT
`db:migrate`	All databases
`db:migrate:queue`	Only the queue DB
`db:prepare`	Create + load each schema
`g migration X --database=cache`	Migration for one DB

SHARDING

PIECE	EXAMPLE
`connects_to shards:`	`{ default: {writing: :primary}, eu: {...} }`
`connected_to(shard:)`	`connected_to(shard: :eu) { ... }`
`shard_selector`	Middleware picks shard per request

⚠ COMMON MISTAKES

- Reading from a replica right after a write and seeing stale data (replication lag)
- Using `execute` for SELECTs; the result type varies by adapter. Use `select_all`
- Joining tables that live in different databases; SQL can't join across connections

✓ TIPS & BEST PRACTICES

- Keep raw SQL in query objects or scopes with tests, not in controllers
- Use `ActiveRecord::Base.connection.exec_query` when you need column types
- Prefer `lease_connection` / `with_connection` (7.2+) over `connection` in new code

QUICK SUMMARY

- `find_by_sql` — SQL → models
- `select_all` — SQL → hashes
- `connects_to` — pick databases
- `connected_to` — switch role or shard

Active Model

MODEL BEHAVIOR WITHOUT A TABLE

Active Model is the toolkit Active Record is built on. Including its modules gives a plain Ruby class validations, typed attributes, naming and form compatibility. Use it for **form objects**, search filters, API payloads and anything that should behave like a model but isn't a database row.

NOTES

- `ActiveModel::Model` = API + validations + naming + conversion, enough for `form_with`
- `ActiveModel::Attributes` adds typed `attribute :name, :type, default:`
- `has_secure_password` lives in Active Model: it works on any class with a `password_digest`
- Serializers::JSON gives `as_json` / `to_json` from `attributes`

A FORM OBJECT

```ruby
class Signup
  include ActiveModel::Model
  include ActiveModel::Attributes

  attribute :name,  :string
  attribute :email, :string
  attribute :plan,  :string, default: "free"
  attribute :terms, :boolean

  validates :name, :email, presence: true
  validates :terms, acceptance: true

  def save
    return false unless valid?
    ActiveRecord::Base.transaction do
      user = User.create!(name:, email_address: email)
      user.create_subscription!(plan:)
    end
    true
  end
end
```

CODE EXPLAINED

→ `attribute :terms, :boolean` — "1" / "0" cast to true / false

→ `valid?` — same validation API and error messages as AR

→ Shorthand hash `name:` — Ruby 3.1 value omission

→ One transaction keeps both writes consistent

→ `model_name` — gives the `signup[...]` param key automatically

CONTROLLER & VIEW

```ruby
def create
  @signup = Signup.new(params.expect(signup: [:name, :email, :plan, :terms]))
  if @signup.save then redirect_to root_path, notice: "Welcome!"
  else render :new, status: :unprocessable_entity end
end
# view: form_with model: @signup, url: signups_path
```

THE RAILS WAY

- A controller juggling two models and their errors → a form object
- Hashes passed around for search filters → an Active Model class with typed attributes
- Fake ActiveRecord tables for non-persisted data → `ActiveModel::Model`

MODULES

MODULE	ADDS
`ActiveModel::API`	Init from hash, `persisted?`, naming, form support
`ActiveModel::Model`	API (plus extensions over time)
`ActiveModel::Attributes`	Typed attributes, defaults
`ActiveModel::Validations`	`validates`, `errors`
`ActiveModel::Dirty`	`changed?`, `changes`
`ActiveModel::Callbacks`	`define_model_callbacks`
`ActiveModel::Serializers::JSON`	`as_json`, `from_json`
`ActiveModel::SecurePassword`	`has_secure_password`

⚠ COMMON MISTAKES

- Forgetting `persisted?` returns false, so `form_with` posts to the create URL even for "edits"; set `url:` and `method:` explicitly
- Reading attributes as instance variables (`@email`) with `Attributes` ; use the reader `email`
- Putting all business logic in one form object; keep it to coordinating a single form

✓ TIPS & BEST PRACTICES

- Use `ActiveModel::Attributes` types (`:date` , `:integer` , `:decimal`) to parse form strings for free
- Lint any model-like class with `ActiveModel::Lint::Tests` in tests
- Add `validates_with` validators to share rules between a model and its form object

QUICK SUMMARY

- `ActiveModel::Model` — form-ready class
- `Attributes` — typed attributes
- validations — same API as AR
- form object — multi-model forms

Active Support Essentials

RAILS' ADDITIONS TO RUBY

Active Support extends Ruby's core classes and provides utilities used across Rails: blankness, time zones and durations, inflections, hash and array helpers, `delegate`, and caching and instrumentation APIs. Inside a Rails app all of it is loaded. The Ruby set's **In Rails** boxes mark which of these look like core Ruby but aren't.

NOTES

- Outside Rails: `require "active_support/all"` or pick files like `active_support/core_ext/object/blank`
- `Time.current` respects `config.time_zone`; `Time.now` uses the server zone
- Durations (`3.days`) are calendar-aware: months and DST are handled
- Inflections are configurable in `config/initializers/inflections.rb`

EVERYDAY EXTENSIONS

```ruby
nil.blank?; "".blank?; " ".blank?; [].blank?    # all true
params[:q].presence || "all"                    # value or nil when blank
user.try(:name)                                 # nil-safe, ignores missing methods

2.days.ago; 3.hours.from_now; 1.month.since(date)
Time.current.beginning_of_week                  # Monday 00:00 in app zone
Date.current.all_month                          # Range for queries
Time.zone.parse("2026-10-23 09:00")             # TimeWithZone
date.to_fs(:long)                               # "October 23, 2026"

"blog_post".camelize      # "BlogPost"     "BlogPost".underscore  # "blog_post"
"person".pluralize        # "people"       "octopi".singularize   # "octopus"
"My Title!".parameterize  # "my-title"     "  a \n b ".squish     # "a b"
"admin_user".humanize     # "Admin user"   "post".titleize        # "Post"

{ a: 1, b: 2 }.except(:b)                       # {a: 1}
{ a: { b: 1 } }.deep_merge(a: { c: 2 })         # {a: {b: 1, c: 2}}
users.index_by(&:id); posts.group_by(&:user_id)
%w[a b c].to_sentence                           # "a, b, and c"
[1, 2, 3, 4].in_groups_of(2)                    # [[1, 2], [3, 4]]
```

CODE EXPLAINED

- → `presence` — turns blank values into nil for `||` defaults
- → `beginning_of_week` / `all_month` — calendar helpers for ranges
- → `to_fs` — named formats (replaces `to_formatted_s`)
- → `parameterize` — URL-safe slugs
- → `delegate ... prefix: true` — `author_name` without Law of Demeter chains

DELEGATE

```ruby
class Post < ApplicationRecord
  belongs_to :author
  delegate :name, to: :author, prefix: true, allow_nil: true   # post.author_name
end
```

THE RAILS WAY

- `Time.now` / `Date.today` in app code → `Time.current` / `Date.current`
- `if x.nil? || x == ""` → `x.blank?`
- Hand-written getters forwarding to another object → `delegate`

TIME HELPERS

CALL	RESULT
`Time.current`	Now in `config.time_zone`
`1.day.ago / 2.weeks.from_now`	Relative times
`time.in_time_zone("Tokyo")`	Convert zone
`date.next_occurring(:monday)`	Next Monday
`time.end_of_day`	23:59:59.999999
`travel_to (tests)`	Freeze time (sheet 67)

OTHER HANDY PIECES

API	USE
`ActiveSupport::Concern`	Module mixins (sheets 17, 31)
`CurrentAttributes`	Per-request globals
`Rails.cache`	Caching (sheet 63)
`ActiveSupport::Notifications`	Instrumentation (sheet 45)
`ActiveSupport::Deprecation`	Deprecation warnings
`ActiveSupport::NumberHelper`	Numbers outside views

⚠ COMMON MISTAKES

- Using `Time.now` and `Date.today`, which ignore the app time zone and cause off-by-one-day bugs
- Expecting ActiveSupport methods in a plain Ruby script without requiring them
- Overusing `try`, which hides typos in method names; prefer `&.`

✓ TIPS & BEST PRACTICES

- Keep custom acronyms in `inflections.rb` (`inflect.acronym "API"`)
- See the Ruby set's In Rails boxes for core-vs-Rails comparisons
- Use `ActiveSupport::Duration` values in config (`config.x.trial = 14.days`)

QUICK SUMMARY

- `blank? / present? / presence`
- durations + `Time.current` — zone-safe time
- inflections — names and slugs
- `delegate` — forward methods

Sheet 45

Notifications & Structured Events

ActiveSupport::Notifications · instrument · subscribe · Rails.event (8.1) · tagged events · Rails.error.report · Rails.logger

OBSERVING WHAT RAILS DOES

Rails publishes **instrumentation events** for SQL queries, renders, controller actions, jobs and more. You can subscribe to them for metrics, or instrument your own code. Rails 8.1 adds **structured event reporting** (`Rails.event`) for machine-readable business and app events, and `Rails.error` is the unified error reporter.

NOTES

- Event names: `sql.active_record`, `process_action.action_controller`, `perform.active_job` …
- Subscribers run synchronously in the same thread: keep them fast
- `Rails.event.notify` sends structured events to registered subscribers
- `Rails.error.report` / `handle` / `record` feed error trackers (Sentry, Honeybadger, …)

SUBSCRIBE & INSTRUMENT

```
# config/initializers/instrumentation.rb
ActiveSupport::Notifications.subscribe("process_action.action_controller") do |event|
  StatsD.timing("rails.request", event.duration,
                tags: ["action:#{event.payload[:controller]}##{event.payload[:action]}"])
end

ActiveSupport::Notifications.instrument("import.csv", rows: rows.size) do
  CsvImporter.new(file).run
end
```

CODE EXPLAINED

- → `event.duration` — milliseconds for the instrumented block
- → `instrument` — publish your own event around a block
- → `Rails.event.notify` — name + payload hash, plus tags and context
- → `subscribe(obj)` — any object with an `emit(event)` method
- → `handle` vs `record` — swallow vs re-raise after reporting

STRUCTURED EVENTS (8.1)

```
Rails.event.notify("user.signup", user_id: user.id, plan: "pro")
Rails.event.tagged("checkout") do
  Rails.event.notify("order.placed", order_id: order.id, total: order.total)
end
Rails.event.set_context(request_id: request.request_id)   # added to every event

class LogSubscriber
  def emit(event) = Rails.logger.info(event.to_json)       # name, payload, tags, context
end
Rails.event.subscribe(LogSubscriber.new)
```

THE RAILS WAY

- `puts` timing code around slow work → `ActiveSupport::Notifications.instrument`
- Ad-hoc log strings for analytics → `Rails.event.notify` with a structured payload
- `rescue => e; Sentry.capture(e)` → `Rails.error.handle`

ERRORS & LOGS

```
Rails.error.handle(context: { order_id: 1 }) { risky_call }   # report + swallow
Rails.error.record { risky_call }                             # report + re-raise
Rails.error.report(e, handled: true)
Rails.logger.tagged("Billing") { Rails.logger.info "charged" }
```

USEFUL BUILT-IN EVENTS

EVENT	PAYLOAD HIGHLIGHTS
`sql.active_record`	`sql`, `name`, `cached`
`process_action.action_controller`	`controller`, `action`, `status`, `db_runtime`
`render_template.action_view`	`identifier`
`perform.active_job`	`job`, `adapter`
`deliver.action_mailer`	`mailer`, `to`
`cache_read.active_support`	`key`, `hit`

LOGGER SETTINGS

SETTING	EFFECT
`config.log_level`	`:debug` … `:fatal`
`config.log_tags`	`[:request_id]` prefixes
`config.logger`	Custom / STDOUT logger
`ActiveSupport::BroadcastLogger`	Log to several outputs

⚠ COMMON MISTAKES

- Doing slow I/O inside a subscriber, which slows every request that emits the event
- Logging personal data in event payloads without filtering
- Using `Rails.error.handle` around code whose failure the caller must know about

✓ TIPS & BEST PRACTICES

- Use `ActiveSupport::Notifications.monotonic_subscribe` for accurate timings
- Add `request_id` to event context so logs, events and errors line up
- Look at Rails' own `LogSubscriber`s to see payload keys for each event

QUICK SUMMARY

- `instrument` / `subscribe` — custom metrics
- `Rails.event` — structured events (8.1)
- `Rails.error` — unified error reporting
- `tagged` — contextual logs

Authentication Generator

BUILT-IN LOGIN, NO GEM REQUIRED

Rails 8 ships `bin/rails generate authentication`, which writes plain, editable code for email/password login: a `User` with `has_secure_password`, a database-backed `Session` model, an `Authentication` concern, sessions and password-reset controllers, and a mailer. Every controller requires login by default; you opt public actions out.

WHAT YOU GET

- No sign-up form: add a `RegistrationsController` yourself
- Sessions are rows, so users can be logged out per device
- Login is rate limited (10 tries / 3 minutes) out of the box
- Passwords use bcrypt via `has_secure_password` (add the `bcrypt` gem; generator uncomments it)

GENERATE

```
bin/rails generate authentication
bin/rails db:migrate
```

GENERATED MODELS (TRIMMED)

```
class User < ApplicationRecord
  has_secure_password
  has_many :sessions, dependent: :destroy
  normalizes :email_address, with: ->(e) { e.strip.downcase }
end

class Session < ApplicationRecord        # user_id, ip_address, user_agent
  belongs_to :user
end
```

CODE EXPLAINED

- → `has_secure_password` — `password=`, `authenticate`, confirmation validation
- → `allow_unauthenticated_access` — skips `require_authentication`
- → `Current.user` — delegated from `Current.session`
- → `authenticate_by` — avoids user-enumeration timing differences
- → `start_new_session_for` — creates a `Session` row and signed cookie

USING IT IN YOUR CODE

```
class PostsController < ApplicationController
  allow_unauthenticated_access only: %i[index show]
  def create = @post = Current.user.posts.create!(post_params)
end

authenticated?                           # helper in views/controllers
Current.user                             # the signed-in user
start_new_session_for(user)              # log in (e.g. after sign-up)
terminate_session                        # log out
User.authenticate_by(email_address: e, password: p)   # timing-safe lookup
```

THE RAILS WAY

- Reaching for Devise on day one → start with the generator; add pieces as needed
- `User.find_by(email:)&.authenticate(pw)` → `User.authenticate_by(...)`
- `helper_method :current_user` hand-rolled → `Current.user` from the generator

GENERATED FILES

FILE	ROLE
app/controllers/concerns/authentication.rb	Filters and helpers
sessions_controller.rb	Login / logout + rate limit
passwords_controller.rb	Reset via emailed token
app/models/current.rb	`Current.session` / `user`
passwords_mailer.rb	Reset email
db/migrate/*_create_users / sessions	Tables

⚠ COMMON MISTAKES

- Assuming the generator includes sign-up, email confirmation or OAuth; it doesn't
- Forgetting `allow_unauthenticated_access` on public pages, sending visitors to the login form
- Exposing whether an email exists in the reset flow; keep the generic message

✓ TIPS & BEST PRACTICES

- Add sign-up with a `RegistrationsController` that saves a `User`, then calls `start_new_session_for(user)`
- Add `generates_token_for :email_confirmation` for confirmation links (sheet 50)
- Show a "log out other devices" page by listing `Current.user.sessions`

QUICK SUMMARY

- `g authentication` — plain-code login (8.0)
- `Current.user` — signed-in user
- `allow_unauthenticated_access` — public actions
- `Session` rows — revocable logins

Authorization

WHO MAY DO WHAT

Authentication answers "who are you"; **authorization** answers "may you do this". Rails has no built-in authorization framework. The simplest reliable approach is **scoping**: load records through the current user's associations. For role and rule-heavy apps, policy objects (Pundit, Action Policy) keep rules testable.

NOTES

- Scoped lookups (`Current.user.posts.find(id)`) raise 404 for others' records
- Check permissions on the server for every action, not just by hiding buttons
- Return 404 for private records to avoid revealing they exist; 403 when existence is public
- Admin areas: a namespaced base controller with one guard

SCOPING & GUARDS

```
class PostsController < ApplicationController
  before_action :set_post, only: %i[edit update destroy]

  private

  def set_post
    @post = Current.user.posts.find(params.expect(:id))   # 404 if not theirs
  end
end

class Admin::BaseController < ApplicationController
  before_action { head :forbidden unless Current.user&.admin? }
end
```

CODE EXPLAINED

- → Association scoping — authorization by construction
- → Inline `before_action` block — quick guard for a namespace
- → `update?` — one predicate per action
- → `policy_scope` — list only what the user may see
- → `policy(@post).update?` in views — hide links the user can't use

PUNDIT POLICY

```
class PostPolicy < ApplicationPolicy
  def update? = user.admin? || record.user_id == user.id
  def destroy? = update?

  class Scope < ApplicationPolicy::Scope
    def resolve = user.admin? ? scope.all : scope.where(user:)
  end
end

class PostsController < ApplicationController
  include Pundit::Authorization
  def index  = @posts = policy_scope(Post)
  def update
    @post = Post.find(params.expect(:id))
    authorize @post                          # raises NotAuthorizedError
    @post.update!(post_params)
    redirect_to @post, status: :see_other
  end
end
```

THE RAILS WAY

- `Post.find(id)` then `if post.user != current_user` → `Current.user.posts.find(id)`
- Role checks scattered across views and controllers → one policy class per model
- `if user.role == "admin"` strings → `enum :role` with `admin?`

APPROACHES

APPROACH	FITS
Association scoping	Owner-only data
before_action guards	A few role checks
Pundit	Plain Ruby policy objects
Action Policy	Policies + caching + scoping rules
CanCanCan	Central `Ability` file

STATUS CHOICES

CASE	RESPOND WITH
Not logged in	Redirect to login
Logged in, not allowed, record public	403 Forbidden
Record private to someone else	404 Not Found
API client	JSON error + status

⚠ COMMON MISTAKES

- Only hiding the Edit button; a crafted PATCH request still works without a server check
- Loading by id first and authorizing after, then forgetting the check in one action
- Policies that query the database for every list row (N+1 inside `update?`)

✓ TIPS & BEST PRACTICES

- Add `after_action :verify_authorized` (Pundit) to catch actions missing a check
- Test policies as plain Ruby objects: fast and focused
- Keep roles coarse; express fine rules in policy methods

QUICK SUMMARY

- scoping — `Current.user.posts.find`
- guards — `before_action` checks
- policies — rules per model
- 404 / 403 — choose deliberately

Built-in Security

DEFENSES RAILS GIVES YOU

Rails protects against common web attacks by default: **CSRF** tokens on every non-GET form, **auto-escaping** in views, parameterized SQL, signed and encrypted cookies, and filtered logs. You configure a **Content Security Policy**, allowed hosts and SSL in config files. Most vulnerabilities come from opting out of these defaults.

DEFAULTS TO KEEP

- `ActionController::Base` verifies CSRF tokens automatically
- Views escape output unless you use `raw`, `html_safe` or `<%==`
- Production sets `config.force_ssl = true` and secure cookies
- `config.filter_parameters` hides passwords and tokens in logs and errors

CONFIG/INITIALIZERS/CONTENT_SECURITY_POLICY.RB

```ruby
Rails.application.configure do
  config.content_security_policy do |policy|
    policy.default_src :self, :https
    policy.img_src     :self, :https, :data
    policy.script_src  :self, :https
    policy.style_src   :self, :https
    policy.object_src  :none
    policy.connect_src :self, "wss://example.com"
  end
  config.content_security_policy_nonce_generator = ->(request) { request.session.id.to_s }
  config.content_security_policy_nonce_directives = %w[script-src style-src]
  # config.content_security_policy_report_only = true   # trial first
end
```

CODE EXPLAINED

- → CSP — the browser refuses scripts and styles from other sources
- → Nonce — lets your own inline `<script>` tags run under CSP
- → `filter_parameters` — partial matches: `:passw` covers `password_confirmation`
- → `config.hosts` — blocks DNS rebinding / Host header attacks
- → `skip_forgery_protection` — only for APIs not using cookies

OTHER SETTINGS

```ruby
# config/initializers/filter_parameter_logging.rb
Rails.application.config.filter_parameters += [:passw, :email, :secret, :token, :otp, :ssn]

# config/environments/production.rb
config.hosts << "example.com" << /.*\.example\.com/
config.force_ssl = true

# app/controllers/api/base_controller.rb (token-auth APIs only)
skip_forgery_protection
```

THE RAILS WAY

- Turning off CSRF to fix a broken form → include `csrf_meta_tags` and `authenticity_token`
- `.html_safe` on user input → `sanitize` with an allow-list
- Shipping CSP straight to enforcing → run `report_only` first

SAFE OUTPUT

```erb
<%= user.bio %>                                    <%# escaped %>
<%= sanitize user.bio, tags: %w[b i a], attributes: %w[href] %>
<%= javascript_tag nonce: true do %>console.log("ok")<% end %>
```

ATTACK → RAILS DEFENSE

ATTACK	DEFENSE
CSRF	Authenticity tokens, SameSite cookies
XSS	Auto-escaping, `sanitize`, CSP
SQL injection	Hash conditions, placeholders
Mass assignment	Strong parameters
Session hijacking	Encrypted cookies, HTTPS, `reset_session`
Open redirect	`allow_other_host`, `url_from`
Brute force	`rate_limit`

SECURITY TOOLS

TOOL	CHECKS
`bin/brakeman`	Static analysis for Rails code
`bundler-audit`	Gems with known CVEs
`importmap audit`	JS packages with CVEs
`bin/ci (8.1)`	Runs the above locally

⚠ COMMON MISTAKES

- Adding `skip_forgery_protection` to a cookie-authenticated controller
- Interpolating user data into `javascript_tag` or inline event handlers
- Logging full params in custom code, bypassing `filter_parameters`

✓ TIPS & BEST PRACTICES

- Run Brakeman in CI; new apps include it in `bin/ci` and the GitHub workflow
- Add `config.action_dispatch.default_headers` changes rather than removing secure defaults
- Review the Rails Security Guide before launch

QUICK SUMMARY

- CSRF tokens — automatic
- escaping — automatic in ERB
- CSP — configure in initializer
- `filter_parameters` — clean logs

Credentials & Secrets

ENCRYPTED SECRETS IN THE REPO

Rails stores secrets in `config/credentials.yml.enc`, encrypted with `config/master.key`. The encrypted file is committed; the key is not. In production the key comes from the `RAILS_MASTER_KEY` environment variable. Per-environment credential files let production use different secrets than development.

NOTES

- `master.key` is in `.gitignore`; share it through a password manager
- Credentials include `secret_key_base`, which signs cookies and tokens
- Kamal reads `RAILS_MASTER_KEY` from `.kamal/secrets`
- Rails 8.2 (edge) adds `Rails.app.creds`, one API over ENV, `.env` and credentials

EDITING

```
EDITOR="code --wait" bin/rails credentials:edit
bin/rails credentials:edit --environment production   # production.yml.enc + production.key
bin/rails credentials:show
bin/rails credentials:diff --enroll                   # readable git diffs
```

THE DECRYPTED YAML

```
secret_key_base: 3f9a…
aws:
  access_key_id: AKIA…
  secret_access_key: wJal…
stripe:
  secret_key: sk_live_…
```

READING

```
Rails.application.credentials.aws[:access_key_id]
Rails.application.credentials.dig(:stripe, :secret_key)
Rails.application.credentials.stripe!               # raise if missing (bang)
Rails.application.credentials.fetch(:sentry_dsn) { ENV["SENTRY_DSN"] }
ENV.fetch("DATABASE_URL")                           # 12-factor style
```

CODE EXPLAINED

- → `--wait` — editor must block until the file closes
- → `--environment production` — separate file and key
- → `dig` — nested lookup, `nil` if missing
- → Bang reader — fail at boot instead of at the first request
- → `ENV.fetch` — raises `KeyError` when unset, unlike `ENV[]`

THE RAILS WAY

- Secrets in `config/*.yml` or committed `.env` files → credentials or deployment ENV
- `ENV["X"]` with silent nil → `ENV.fetch("X")` or `credentials.x!`
- Same key for all environments → per-environment credentials

FILES

FILE	COMMIT?
`config/credentials.yml.enc`	yes
`config/master.key`	no
`config/credentials/production.yml.enc`	yes
`config/credentials/production.key`	no
`.kamal/secrets`	yes (it references ENV / managers, not values)

CREDENTIALS VS ENV

USE	WHEN
`Credentials`	Secrets that version with code
`ENV`	Values set by the platform (DATABASE_URL)
`Both`	`credentials.x || ENV["X"]` fallback

⚠ COMMON MISTAKES

- Committing `master.key` or pasting it into chat or tickets
- Losing `master.key`: the encrypted file can't be recovered, only re-created
- Editing the `.enc` file directly instead of through `credentials:edit`

✓ TIPS & BEST PRACTICES

- Keep one shared password-manager item per environment key
- Use `credentials:diff --enroll` so `git diff` shows decrypted changes locally
- Rotate a leaked `secret_key_base` with a rotator so existing sessions survive

QUICK SUMMARY

- `credentials:edit` — change secrets
- `master.key` / `RAILS_MASTER_KEY` — never commit
- `credentials.dig` — read nested
- per-env files — separate prod secrets

Sheet 50

Encryption & Signed Tokens

encrypts · deterministic: · generates_token_for (7.1) · signed_id · has_secure_token · MessageVerifier

PROTECTING DATA AND LINKS

Active Record Encryption encrypts columns at the application level, so database dumps don't expose them. Rails can also create **tamper-proof tokens**: `generates_token_for` makes purpose-specific tokens that expire or self-invalidate, `signed_id` signs a record id, and `has_secure_token` stores a random unique token.

NOTES

- Run `bin/rails db:encryption:init` and store the keys in credentials
- Non-deterministic encryption can't be queried; deterministic can (equality only)
- `generates_token_for` tokens embed data from a block: change it and old tokens die
- None of these replace hashing for passwords (`has_secure_password`)

ENCRYPTED ATTRIBUTES

```
class User < ApplicationRecord
  encrypts :ssn                                  # random IV, not queryable
  encrypts :email_address, deterministic: true, downcase: true
end
User.find_by(email_address: "ann@x.com")         # works (deterministic)
user.ciphertext_for(:ssn)                        # stored value
```

PURPOSE TOKENS (7.1+)

```
class User < ApplicationRecord
  generates_token_for :password_reset, expires_in: 15.minutes do
    password_salt&.last(10)              # changes when password changes
  end
  generates_token_for :email_confirmation, expires_in: 24.hours do
    email_address
  end
end

token = user.generate_token_for(:password_reset)
User.find_by_token_for(:password_reset, token)    # nil if expired/used
User.find_by_token_for!(:email_confirmation, t)   # raises if invalid
```

SIGNED IDS, SECURE TOKENS, VERIFIERS

```
id = post.signed_id(expires_in: 1.hour, purpose: :share)
Post.find_signed(id, purpose: :share)

class ApiClient < ApplicationRecord
  has_secure_token :api_key, length: 36          # random, unique column
end
client.regenerate_api_key

v = Rails.application.message_verifier(:unsubscribe)
token = v.generate(user.id, expires_in: 30.days)
v.verified(token)                                # user.id or nil
```

CODE EXPLAINED

- → `deterministic: true` — same input gives same ciphertext, so lookups work
- → `downcase: true` — case-insensitive deterministic matching
- → Token block — any change in its return value invalidates old tokens
- → `signed_id` — shareable links without exposing guessable ids
- → `message_verifier(:name)` — sign any data for links and cookies

THE RAILS WAY

- A `reset_token` column you must clear → `generates_token_for :password_reset`
- Base64-encoding ids in URLs → `signed_id` or `message_verifier`
- Encrypting columns by hand with OpenSSL → `encrypts`

WHICH TOOL?

NEED	USE
Hide data at rest	encrypts
One-time / expiring email links	generates_token_for
Share a record link	signed_id
Long-lived API key	has_secure_token
Sign arbitrary payloads	MessageVerifier
Encrypt arbitrary payloads	MessageEncryptor

⚠ COMMON MISTAKES

- Encrypting a column you need for `LIKE` searches or sorting; encrypted columns only support equality (deterministic)
- Storing encryption keys in the database or committing them in plain text
- Long-lived `generates_token_for` tokens without a block, so they can't be revoked

✓ TIPS & BEST PRACTICES

- Use `support_unencrypted_data` while migrating existing plaintext columns
- Filter encrypted columns from logs automatically: they are added to `filter_parameters`
- Use separate `purpose:` values so a share token can't be reused as an unsubscribe token

QUICK SUMMARY

- `encrypts` — encrypted columns
- `generates_token_for` — expiring purpose tokens
- `signed_id` — signed record ids
- `has_secure_token` — random keys

Propshaft & Asset Delivery

SERVING CSS, IMAGES AND FONTS

Rails 8 uses **Propshaft**, a simple asset pipeline: it doesn't compile or bundle, it just adds a content **digest** to each file name so browsers can cache forever, and rewrites `url()` references in CSS. Modern CSS and import maps cover what Sprockets used to compile.

NOTES

- Load paths: `app/assets/*`, `vendor/assets`, and gem assets
- Digested files: `app-3f2e9a1c.css`; Rails helpers map names to them
- Production: `bin/rails assets:precompile` writes to `public/assets`
- Tailwind / Sass / esbuild output goes into `app/assets/builds` for Propshaft to serve

LAYOUT & VIEWS

```
<%= stylesheet_link_tag :app, "data-turbo-track": "reload" %>   <%# all in app/assets/stylesheets %>
<%= stylesheet_link_tag "admin" %>
<%= image_tag "logo.svg", alt: "Logo" %>
<%= favicon_link_tag "favicon.ico" %>
<link rel="preload" href="<%= asset_path("inter.woff2") %>" as="font" crossorigin>
```

CSS

```
/* app/assets/stylesheets/application.css */
@import url("buttons.css");
.hero { background: url("hero.jpg"); }   /* rewritten to /assets/hero-ab12.jpg */
```

COMMANDS & CONFIG

```
bin/rails assets:precompile     # digest into public/assets
bin/rails assets:clobber        # remove compiled assets
bin/rails assets:reveal         # list logical paths
bin/rails assets:reveal:full    # with full paths
```

CONFIG/INITIALIZERS/ASSETS.RB

```
Rails.application.config.assets.paths << Rails.root.join("node_modules/@fontsource")
Rails.application.config.assets.excluded_paths << Rails.root.join("app/assets/stylesheets/src")
```

CODE EXPLAINED

- → `:app` — every stylesheet in `app/assets/stylesheets`, in order
- → `data-turbo-track: reload` — full reload when assets change after deploy
- → `asset_path` — digested URL for any asset
- → CSS `url()` — Propshaft rewrites to digested paths
- → `excluded_paths` — keep source files out of the load path

THE RAILS WAY

- Sprockets `//= require` directives → native CSS `@import` or a CSS bundler
- Hard-coded `/images/logo.png` → `image_tag "logo.png"`
- Committing `public/assets` → precompile in the Docker build

PROPSHAFT VS SPROCKETS

	PROPSHAFT · SPROCKETS
Default in	Rails 8 · Rails ≤ 7
Compiles Sass/CoffeeScript	no (use bundlers) · yes
Digests + caching	yes · yes
Manifest directives	none · `//= require`
Speed / complexity	fast, tiny · slower, larger

⚠ COMMON MISTAKES

- Referencing assets with relative `../images` paths in ERB; use helpers so digests apply
- Forgetting to precompile in production and getting 404s for `/assets/*`
- Mixing Sprockets and Propshaft gems in one app

✓ TIPS & BEST PRACTICES

- Use `bin/rails assets:reveal` to debug "asset not found" errors
- Thruster adds far-future caching and compression for `/assets` in Rails 8 containers
- For Tailwind, `tailwindcss-rails` builds into `app/assets/builds/tailwind.css`

QUICK SUMMARY

- Propshaft — digest, don't compile
- `stylesheet_link_tag :app` — all styles
- `asset_path` — digested URL
- `assets:precompile` — production build

JavaScript & CSS Options

NO-BUILD BY DEFAULT, BUNDLERS WHEN NEEDED

New Rails apps use **import maps**: the browser loads ES modules directly, mapped from bare names like `"@hotwired/turbo-rails"` to digested files. There's no Node, no build step and no `node_modules`. When you need npm-heavy libraries, JSX or TypeScript, switch to **jsbundling-rails** (esbuild, Bun, Rollup, Webpack). CSS frameworks plug in through **tailwindcss-rails** or **cssbundling-rails**.

NOTES

- Import maps rely on HTTP/2 and modern browsers loading many small modules
- `bin/importmap pin` downloads the package into `vendor/javascript`
- Bundler gems write output to `app/assets/builds`; Propshaft serves it
- `bin/dev` runs the CSS/JS watchers alongside the server

CONFIG/IMPORTMAP.RB

```
pin "application"
pin "@hotwired/turbo-rails", to: "turbo.min.js"
pin "@hotwired/stimulus", to: "stimulus.min.js"
pin "@hotwired/stimulus-loading", to: "stimulus-loading.js"
pin_all_from "app/javascript/controllers", under: "controllers"
pin "chart.js", to: "https://cdn.jsdelivr.net/npm/chart.js@4.4.4/+esm"   # or vendored
```

COMMANDS

```
bin/importmap pin local-time        # vendor a package
bin/importmap unpin local-time
bin/importmap json                  # show the generated map
bin/importmap outdated              # newer versions available
bin/importmap audit                 # known vulnerabilities

./bin/bundle add tailwindcss-rails && bin/rails tailwindcss:install
./bin/bundle add jsbundling-rails  && bin/rails javascript:install:esbuild
```

APP/JAVASCRIPT/APPLICATION.JS

```
import "@hotwired/turbo-rails"
import "controllers"
import { Chart } from "chart.js"
```

CODE EXPLAINED

- → `pin` — map a bare module name to a file
- → `pin_all_from` — every Stimulus controller, by folder
- → `javascript_importmap_tags` in the layout — emits the map and preloads
- → `tailwindcss:install` — config, `bin/dev`, and a build task
- → `javascript:install:esbuild` — `package.json`, build script, `app/assets/builds`

THE RAILS WAY

- `npm install` + Webpacker for a few libraries → `bin/importmap pin`
- Inline `<script>` tags in views → Stimulus controllers (sheet 56)
- Hand-written Tailwind CLI scripts → `tailwindcss-rails` watcher via `bin/dev`

CHOOSING A JS APPROACH

OPTION	BEST WHEN
`importmap-rails`	Hotwire apps, few npm deps, no transpiling
`jsbundling + esbuild`	Many npm packages, JSX, TypeScript
`jsbundling + bun`	Same, with Bun's runtime/tooling
`vite_ruby (gem)`	Vite dev server, HMR, SPA-ish frontends

CSS OPTIONS

GEM	PROVIDES
`tailwindcss-rails`	Standalone Tailwind CLI, no Node
`cssbundling-rails`	Bootstrap, Bulma, PostCSS, Sass via npm
`dartsass-rails`	Sass without Node
`plain CSS`	Nesting, variables, `@layer` in modern browsers

⚠ COMMON MISTAKES

- Pinning a package that ships only CommonJS; import maps need ES modules
- Forgetting `bin/dev` and wondering why Tailwind classes have no styles
- Mixing an import map and a bundler for the same entry point

✓ TIPS & BEST PRACTICES

- Use `bin/importmap outdated` and `audit` in CI
- Vendor pins (download) rather than CDN links so deploys don't depend on a third party
- Start with import maps; switching to esbuild later is a contained change

QUICK SUMMARY

- import maps — no-build default
- `bin/importmap pin` — add packages
- jsbundling — esbuild/Bun when needed
- tailwindcss-rails — Tailwind without Node

Turbo Drive

SPA SPEED WITH SERVER-RENDERED HTML

Turbo Drive intercepts link clicks and form submissions, fetches the next page, and swaps the <body> without a full reload. CSS and JS stay loaded, so navigation feels instant. Turbo 8 adds **prefetch on hover** and **page refreshes with morphing**, which update only what changed and keep scroll position.

NOTES

- Included in new apps via `import "@hotwired/turbo-rails"`
- Forms must answer **422** to re-render with errors, or **303** to redirect
- Links hovered for 100 ms are prefetched (disable with `data-turbo-prefetch="false"`)
- Assets marked `data-turbo-track="reload"` force a full reload when they change

CONTROLLING TURBO IN HTML

```
<%= link_to "Download", report_path, data: { turbo: false } %>   <%# full page load %>
<%= link_to "Log out", session_path, data: { turbo_method: :delete } %>
<%= button_to "Delete", @post, method: :delete,
      form: { data: { turbo_confirm: "Delete this post?" } } %>
<%= link_to "Next", page_path(2), data: { turbo_action: "replace" } %>
<div id="player" data-turbo-permanent>…</div>                    <%# kept across pages %>
```

CODE EXPLAINED

- → `data: { turbo: false }` — opt out for downloads or external pages
- → `turbo_method: :delete` — link that sends DELETE (prefer `button_to`)
- → `turbo_confirm` — native confirm dialog before submit
- → `data-turbo-permanent` — element survives navigation (needs an `id`)
- → `turbo:load` — use instead of `DOMContentLoaded`

MORPHING REFRESHES (LAYOUT HEAD)

```
<%= turbo_refreshes_with method: :morph, scroll: :preserve %>
<%= yield :head %>
```

CONTROLLER RESPONSES TURBO EXPECTS

```
if @post.save
  redirect_to @post, status: :see_other
else
  render :new, status: :unprocessable_entity
end
```

THE RAILS WAY

- jQuery `$(document).ready` → Stimulus `connect()` or `turbo:load`
- `render :new` (200) after failed save → 422 so Turbo shows errors
- Rebuilding the page with JS after an update → morphing refresh

JAVASCRIPT

```
Turbo.visit("/posts", { action: "replace" })
document.addEventListener("turbo:load", () => { /* runs after each visit */ })
```

TURBO EVENTS

EVENT	WHEN
`turbo:click`	Link clicked
`turbo:before-visit`	Before navigation (cancelable)
`turbo:submit-start / end`	Form submission
`turbo:before-render`	Before new body is shown
`turbo:load`	After every page load
`turbo:before-cache`	Clean up before snapshot

DATA- ATTRIBUTES

ATTRIBUTE	EFFECT
`data-turbo="false"`	Disable Turbo
`data-turbo-method`	HTTP verb for links
`data-turbo-confirm`	Confirmation prompt
`data-turbo-action`	`advance` / `replace` history
`data-turbo-frame`	Target a frame (sheet 54)
`data-turbo-preload`	Preload into cache

⚠ COMMON MISTAKES

- Initializing widgets on `DOMContentLoaded`; it fires once, not on Turbo visits
- Returning 200 for invalid forms: the page doesn't change
- Using GET links for logout or delete; prefetching can trigger them

✓ TIPS & BEST PRACTICES

- Use `turbo:before-cache` to close menus and modals before Turbo snapshots the page
- Disable prefetch on links that hit expensive or side-effecting endpoints
- Check the network tab: Turbo requests send `Accept: text/vnd.turbo-stream.html, text/html`

QUICK SUMMARY

- Turbo Drive — fetch + swap body
- 422 / 303 — form contract
- `data-turbo` — opt out
- morphing — smooth refreshes

Turbo Frames

UPDATE PART OF THE PAGE

A **Turbo Frame** is a region of the page that navigates on its own. Links and forms inside a frame load the next page, find the frame with the **same id** in the response, and swap just that frame. Frames can also load lazily from a URL, which is handy for slow sections.

NOTES

- Frame ids must match between the current page and the response
- A response without a matching frame shows "Content missing"
- `target: "_top"` makes links inside the frame navigate the whole page
- Rails sets `request.headers["Turbo-Frame"]`; layouts are skipped for frame requests

INLINE EDITING

```erb
<%# posts/show.html.erb %>
<%= turbo_frame_tag dom_id(@post) do %>
  <h1><%= @post.title %></h1>
  <%= link_to "Edit", edit_post_path(@post) %>              <%# loads in the frame %>
<% end %>

<%# posts/edit.html.erb %>
<%= turbo_frame_tag dom_id(@post) do %>
  <%= render "form", post: @post %>                        <%# submit → show → swap back %>
<% end %>
```

CODE EXPLAINED

- → `dom_id(@post)` — `"post_1"`, a unique, matching frame id
- → Edit link inside the frame — only the frame swaps to the form
- → `loading: :lazy` — fetch when the frame scrolls into view
- → `data-turbo-frame` on a link — drive a different frame by id
- → `turbo_action: "advance"` — update the URL for frame navigation

LAZY LOADING & TARGETS

```erb
<%= turbo_frame_tag "stats", src: stats_path, loading: :lazy do %>
  <p>Loading…</p>
<% end %>

<%= turbo_frame_tag "results", target: "_top" do %> … <% end %>
<%= link_to "Open", post_path(p), data: { turbo_frame: "_top" } %>
<%= link_to "Preview", post_path(p), data: { turbo_frame: "preview" } %>
<%= form_with url: search_path, method: :get,
      data: { turbo_frame: "results", turbo_action: "advance" } do |f| %> … <% end %>
```

THE RAILS WAY

- Custom fetch + innerHTML for inline editing → a Turbo Frame around show/edit
- Slow sidebar delaying the whole page → a lazy `src:` frame
- Modal JS libraries for simple dialogs → a `modal` frame plus a `<dialog>` element

CONTROLLER

```ruby
def stats
  @stats = Stats.compute                  # stats.html.erb wraps output in
end                                       # turbo_frame_tag "stats"
```

FRAME OPTIONS

OPTION	EFFECT
`src:`	Load content from URL
`loading: :lazy`	Load when visible
`target: "_top"`	Links navigate full page
`data-turbo-action`	Push history for frame visits
`refresh: "morph"`	Morph on page refresh
`disabled`	Turn off frame navigation

FRAMES VS STREAMS

	FRAMES · STREAMS
Updates	One region · Many regions
Trigger	Link/form inside frame · Response or broadcast
Server returns	Full page HTML · `<turbo-stream>` actions
Real-time	no · yes (with Action Cable)

⚠ COMMON MISTAKES

- Response template without the matching `turbo_frame_tag` → "Content missing"
- Expecting links inside a frame to change the URL; add `turbo_action: "advance"`
- Wrapping huge sections in frames, making each click download a full page to update a tiny area

✓ TIPS & BEST PRACTICES

- Listen for `turbo:frame-missing` to redirect to the full page instead of showing an error
- Use `turbo_frame_request?` in controllers to skip extra work for frame requests
- Combine lazy frames with HTTP caching for cheap dashboards

QUICK SUMMARY

- `turbo_frame_tag` — independent region
- matching ids — required
- `src: loading: :lazy` — deferred content
- `_top` — break out to full page

Turbo Streams

MANY CHANGES, ONE RESPONSE

A **Turbo Stream** is a small HTML instruction: append, prepend, replace, update or remove the element with a given id. Controllers can return several in one response to a form submit. Models can **broadcast** streams over Action Cable so every open browser updates in real time.

NOTES

- Stream responses are only used for non-GET form submissions (and broadcasts)
- Targets are element ids; use `dom_id` for records
- `broadcasts_refreshes` sends a single "refresh" that morphs the page (Turbo 8)
- Broadcasting uses Action Cable (Solid Cable by default in Rails 8)

CONTROLLER

```
def create
  @comment = @post.comments.create!(comment_params)
  respond_to do |format|
    format.turbo_stream                    # create.turbo_stream.erb
    format.html { redirect_to @post }
  end
end

def destroy
  @comment.destroy
  render turbo_stream: turbo_stream.remove(@comment)
end
```

CODE EXPLAINED

- → `format.turbo_stream` — chosen when Turbo submits a form
- → `turbo_stream.append "comments", @comment` — render partial into `#comments`
- → `update` vs `replace` — inner HTML vs the whole element
- → `broadcasts_to :post` — callbacks broadcast after commit
- → `turbo_stream_from` — signed subscription tag in the view

COMMENTS/CREATE.TURBO_STREAM.ERB

```
<%= turbo_stream.append "comments", @comment %>        <%# renders _comment %>
<%= turbo_stream.update "comment_count", @post.comments.size %>
<%= turbo_stream.replace "new_comment" do %>
  <%= render "form", comment: Comment.new %>
<% end %>
```

REAL-TIME BROADCASTS

```
class Comment < ApplicationRecord
  belongs_to :post
  broadcasts_to :post                      # append/replace/remove on the post's stream
end

class Post < ApplicationRecord
  broadcasts_refreshes                     # page morphs when the post changes
end
# view: <%= turbo_stream_from @post %>  subscribes this page
```

THE RAILS WAY

- Custom JSON + JS DOM updates → Turbo Stream templates
- Polling for new comments → `broadcasts_to` + `turbo_stream_from`
- Many fine-grained broadcasts → `broadcasts_refreshes` and let morphing diff the page

STREAM ACTIONS

ACTION	DOES
`append / prepend`	Add inside target, end / start
`before / after`	Insert as sibling
`replace`	Swap the whole target element
`update`	Swap inner HTML
`remove`	Delete target
`morph`	Morph target to new HTML
`refresh`	Re-fetch and morph page

BROADCAST HELPERS

METHOD	WHEN
`broadcasts_to :post`	Callbacks → stream named by post
`broadcasts`	Stream named after the model
`broadcast_append_later_to`	Manual, via job
`broadcasts_refreshes`	Refresh on change
`broadcast_remove_to`	Manual remove

⚠ COMMON MISTAKES

- Returning Turbo Streams from GET requests; Turbo ignores them for normal visits
- Broadcasting synchronously from heavy callbacks; use `_later` variants (the default for `broadcasts_to`)
- Forgetting `turbo_stream_from` in the view, so broadcasts reach nobody

✓ TIPS & BEST PRACTICES

- Always keep an HTML fallback (`format.html`) for non-Turbo clients
- Scope streams by account or user: `turbo_stream_from Current.user, :notifications`
- Test stream responses with `assert_turbo_stream action: :append, target: "comments"`

QUICK SUMMARY

- `turbo_stream.*` — targeted DOM actions
- `*.turbo_stream.erb` — multi-action responses
- `broadcasts_to` — real-time updates
- `broadcasts_refreshes` — morph refresh

Stimulus

SPRINKLES OF JAVASCRIPT

Stimulus attaches small JavaScript controllers to HTML through `data-` attributes. The HTML stays server-rendered; controllers add behavior such as toggles, clipboard copy, autosave and character counters. Controllers connect and disconnect automatically as Turbo changes the page.

NOTES

- Controller file `hello_controller.js` → identifier `hello`; `admin/menu_controller.js` → `admin--menu`
- Generate with `bin/rails g stimulus dropdown`
- With import maps, controllers are eager-loaded from `app/javascript/controllers`
- State lives in the DOM (values), not in hidden JS variables

APP/JAVASCRIPT/CONTROLLERS/CLIPBOARD_CONTROLLER.JS

```
import { Controller } from "@hotwired/stimulus"

export default class extends Controller {
  static targets = ["source", "button"]
  static values  = { successText: { type: String, default: "Copied!" } }
  static classes = ["success"]

  connect() { this.originalText = this.buttonTarget.textContent }

  copy(event) {
    event.preventDefault()
    navigator.clipboard.writeText(this.sourceTarget.value)
    this.buttonTarget.textContent = this.successTextValue
    this.element.classList.add(this.successClass)
  }

  disconnect() { this.buttonTarget.textContent = this.originalText }
}
```

CODE EXPLAINED

- → `static targets` — `this.sourceTarget`, `this.hasSourceTarget`
- → `static values` — typed, read from `data-*-value`, with `…ValueChanged()` callbacks
- → `data-action="clipboard#copy"` — default event for buttons is click
- → `keydown.esc@window` — key filter + global listener
- → `connect` / `disconnect` — lifecycle as elements enter/leave the DOM

THE HTML

```
<div data-controller="clipboard"
     data-clipboard-success-text-value="Link copied"
     data-clipboard-success-class="copied">
  <input data-clipboard-target="source" value="<%= post_url(@post) %>" readonly>
  <button data-clipboard-target="button" data-action="clipboard#copy">Copy</button>
</div>
<%= tag.div data: { controller: "toggle", action: "keydown.esc@window->toggle#hide" } %>
```

THE RAILS WAY

- `document.querySelector` from global scripts → targets scoped to the controller
- Inline `onclick` handlers → `data-action` (also CSP-friendly)
- One giant app.js → one small controller per behavior

NAMING RULES

HTML	JS
`data-controller="clipboard"`	`clipboard_controller.js`
`data-clipboard-target="source"`	`this.sourceTarget`
`data-clipboard-url-value`	`this.urlValue`
`data-clipboard-active-class`	`this.activeClass`
`data-action="input->search#filter"`	`filter(event)`
`data-search-results-outlet=".list"`	`this.resultsOutlet`

VALUE TYPES

TYPE	EXAMPLE
`String`	`"Copied"`
`Number`	`300`
`Boolean`	`true`
`Array`	`[1,2]` (JSON)
`Object`	`{"a":1}` (JSON)

⚠ COMMON MISTAKES

- Misspelling the identifier (`data-controller="Clipboard"`): identifiers are lowercase with dashes
- Keeping state in JS variables that Turbo snapshots lose; store it in values
- Adding listeners in `connect` without removing them in `disconnect`

✓ TIPS & BEST PRACTICES

- Use `bin/rails g stimulus name` so the file lands where the loader expects it
- Use outlets to let controllers talk without global events
- Check `this.application.debug = true` in the console to log lifecycle events

QUICK SUMMARY

- `data-controller` — attach behavior
- targets — scoped elements
- values — typed DOM state
- `data-action` — event → method

Active Job Basics

BACKGROUND WORK

Active Job is Rails' common interface for background jobs. You write a job class with a `perform` method and enqueue it with `perform_later`; a queue backend runs it in a separate process. Rails 8 uses **Solid Queue** (database-backed) by default, but the same code works with Sidekiq, GoodJob and others.

NOTES

- Jobs live in `app/jobs`; generate with `bin/rails g job SendDigest`
- Model arguments are serialized as **GlobalID** and re-fetched when the job runs
- `enqueue_after_transaction_commit` (7.2+) can defer enqueues until commit; check your version's default
- Development and test can use the `:async` / `:test` adapters

APP/JOBS/SEND_DIGEST_JOB.RB

```ruby
class SendDigestJob < ApplicationJob
  queue_as :mailers
  retry_on Net::OpenTimeout, wait: :polynomially_longer, attempts: 5
  discard_on ActiveJob::DeserializationError        # record was deleted

  def perform(user, since: 1.day.ago)
    posts = user.followed_posts.where(created_at: since..)
    DigestMailer.with(user:, posts:).digest.deliver_now if posts.any?
  end
end
```

CODE EXPLAINED

- → `queue_as` — choose a queue (workers can prioritize queues)
- → `retry_on` — retries with backoff, then re-raises after `attempts`
- → `discard_on` — drop the job silently for that error
- → Passing `user` — stored as `gid://app/User/1`, reloaded in the worker
- → `perform_all_later` — enqueue many in one round trip (7.1+)

ENQUEUING

```ruby
SendDigestJob.perform_later(user)                       # as soon as possible
SendDigestJob.set(wait: 10.minutes).perform_later(user)
SendDigestJob.set(wait_until: Date.tomorrow.noon, queue: :low).perform_later(user)
SendDigestJob.perform_now(user)                         # inline, same process
ActiveJob.perform_all_later(users.map { |u| SendDigestJob.new(u) })   # bulk
```

THE RAILS WAY

- Sending email or calling APIs inside the request → `perform_later`
- Passing serialized hashes of a record → pass the record (GlobalID)
- Rescue-and-sleep loops in jobs → `retry_on` with `wait:`

CONFIG/ENVIRONMENTS/PRODUCTION.RB

```ruby
config.active_job.queue_adapter = :solid_queue
config.solid_queue.connects_to = { database: { writing: :queue } }
```

ARGUMENT TYPES

TYPE	ALLOWED?
`String, Integer, Float, nil, true/false`	yes
`Symbol, Date, Time, BigDecimal, Range`	yes
`Hash / Array of the above`	yes
`Active Record models`	yes (GlobalID)
`Arbitrary objects`	no: add a custom serializer

CALLBACKS & HOOKS

CALLBACK	WHEN
`before_enqueue / after_enqueue`	Around enqueue
`before_perform / after_perform`	Around perform
`around_perform`	Wrap execution
`rescue_from`	Handle errors in-job
`after_discard`	After a job is discarded

⚠ COMMON MISTAKES

- Passing large payloads (whole CSVs) as arguments; store the file and pass an id
- Writing non-idempotent jobs: retries may run them twice
- Using `perform_now` in requests for slow work, defeating the purpose

✓ TIPS & BEST PRACTICES

- Make jobs small and idempotent; split big work into many jobs or use continuations (sheet 58)
- Use `deliver_later` for mail; it's an Active Job under the hood
- Enqueue from `after_commit` callbacks so workers never see uncommitted rows

QUICK SUMMARY

- `perform_later` — enqueue
- `set(wait:)` — schedule
- `retry_on / discard_on` — failure policy
- GlobalID — pass models safely

Solid Queue & Job Continuations

THE RAILS 8 JOB BACKEND

Solid Queue stores jobs in a database table and runs them with workers, dispatchers for scheduled jobs, and a scheduler for recurring tasks. No Redis needed. Rails 8.1 adds **job continuations**: long jobs are split into `steps` with cursors so a deploy or restart resumes where it left off instead of starting over.

NOTES

- Run workers with `bin/jobs`, or inside Puma with `SOLID_QUEUE_IN_PUMA=1` (single-server default)
- Recurring tasks are defined in `config/recurring.yml`
- Concurrency controls limit how many jobs with the same key run at once
- **Mission Control – Jobs** (gem) is a web UI for queues, failures and retries

CONFIG/RECURRING.YML

```yaml
production:
  nightly_digest:
    class: SendAllDigestsJob
    queue: mailers
    schedule: every day at 6am
  clear_finished_jobs:
    command: "SolidQueue::Job.clear_finished_in_batches"
    schedule: every hour at minute 12
```

CONCURRENCY LIMITS

```ruby
class SyncAccountJob < ApplicationJob
  limits_concurrency to: 1, key: ->(account) { account }, duration: 10.minutes
  def perform(account) = account.sync!
end
```

CONTINUATIONS (8.1)

```ruby
class ImportJob < ApplicationJob
  include ActiveJob::Continuable

  def perform(import)
    @import = import
    step :validate do
      @import.validate!
    end
    step :process do |step|
      @import.rows.find_each(start: step.cursor) do |row|
        row.process!
        step.advance! from: row.id          # save progress
      end
    end
    step :finish                            # calls #finish
  end

  private def finish = @import.complete!
end
```

CODE EXPLAINED

- → `schedule:` — natural-language schedules parsed by Fugit
- → `command:` — Ruby string evaluated on schedule, no job class needed
- → `limits_concurrency` — extra jobs wait (blocked) until the slot frees
- → `step.cursor` / `advance!` — checkpoint; resumes after interruption
- → Completed steps are skipped when the job resumes

THE RAILS WAY

- A separate cron box with `rails runner` → `recurring.yml`
- Redis + Sidekiq for a small app → Solid Queue on the existing database
- One giant job restarting from zero after each deploy → continuation steps

CONFIG/QUEUE.YML PIECES

KEY	MEANING
`dispatchers`	Move due scheduled jobs to ready
`workers: queues`	`"*"` or `[critical, default]` order
`threads / processes`	Concurrency per worker

OPERATIONS

TASK	HOW
`Web UI`	`mount MissionControl::Jobs::Engine, at: "/jobs"`
`Retry failed`	UI, or `SolidQueue::FailedExecution#retry`
`Separate DB`	`connects_to` the `queue` database

⚠ COMMON MISTAKES

- Mounting Mission Control without authentication
- Running `SOLID_QUEUE_IN_PUMA` on several web servers and also a job server, doubling workers by accident
- Expecting continuations to save local variables; only the cursor and completed steps persist

✓ TIPS & BEST PRACTICES

- Use separate queues for slow and fast work so one doesn't starve the other
- Keep each continuation step resumable on its own; cursors should be stable ids
- Check failed jobs regularly; Solid Queue keeps them until you retry or discard

QUICK SUMMARY

- Solid Queue — DB-backed jobs
- `recurring.yml` — cron replacement
- `limits_concurrency` — one at a time
- continuations — resumable steps (8.1)

Action Mailer

SENDING EMAIL

Mailers work like controllers: each method sets instance variables and calls `mail`, and a view template renders the body (HTML and/or text). Call `deliver_later` to send through Active Job. **Previews** render emails in the browser while you design them. Inbound email is a separate framework, **Action Mailbox**.

NOTES

- Generate with `bin/rails g mailer User welcome`
- Views: `app/views/user_mailer/welcome.html.erb` and `.text.erb`
- Links need absolute URLs: set `default_url_options[:host]` per environment
- Pass data with `.with(user:)` (parameterized mailers) and read `params[:user]`

APP/MAILERS/USER_MAILER.RB

```ruby
class UserMailer < ApplicationMailer
  default from: email_address_with_name("hello@example.com", "Example App")
  before_action { @user = params[:user] }

  def welcome
    attachments["guide.pdf"] = File.read(Rails.root.join("public/guide.pdf"))
    mail to: email_address_with_name(@user.email_address, @user.name),
         subject: "Welcome, #{@user.name}!"
  end
end
```

CODE EXPLAINED

- → `email_address_with_name` — `"Example App" <hello@example.com>`
- → `before_action` — mailers have callbacks too
- → `attachments[...]` — files; `attachments.inline` for images in HTML
- → `dashboard_url` — `_url` helpers, never `_path`, in emails
- → Previews — no email sent, renders HTML and text parts

SENDING

```ruby
UserMailer.with(user: user).welcome.deliver_later
UserMailer.with(user: user).welcome.deliver_later(wait: 1.hour)
UserMailer.with(user: user).welcome.deliver_now      # scripts, tests
```

APP/VIEWS/USER_MAILER/WELCOME.HTML.ERB

```erb
<h1>Hi <%= @user.name %>,</h1>
<p><%= link_to "Get started", dashboard_url %></p>
```

PREVIEW & CONFIG

```ruby
# test/mailers/previews/user_mailer_preview.rb → /rails/mailers
class UserMailerPreview < ActionMailer::Preview
  def welcome = UserMailer.with(user: User.first).welcome
end

# config/environments/production.rb
config.action_mailer.default_url_options = { host: "example.com" }
config.action_mailer.smtp_settings = { address: "smtp.postmarkapp.com", port: 587,
  user_name: Rails.application.credentials.dig(:smtp, :user),
  password:  Rails.application.credentials.dig(:smtp, :password) }
```

THE RAILS WAY

- `deliver_now` in a controller → `deliver_later`
- Positional mailer arguments → `.with(...)` and `params`
- Sending real mail in development → `letter_opener` or previews

DELIVERY METHODS

METHOD	USE
`:smtp`	Production via provider
`:sendmail`	Local MTA
`:test`	Tests: `ActionMailer::Base.deliveries`
`:letter_opener (gem)`	Open in browser (dev)
`:file`	Write to `tmp/mails`

ACTION MAILBOX (INBOUND)

PIECE	ROLE
`bin/rails action_mailbox:install`	Tables and routes
`ApplicationMailbox routing`	`routing /^reply@/i => :replies`
`RepliesMailbox#process`	Handle `mail` object
`Ingresses`	Postmark, SendGrid, Mailgun, Mandrill, relay

⚠ COMMON MISTAKES

- Forgetting `default_url_options`, so links in emails raise "Missing host to link to!"
- Passing unsaved records to `deliver_later`; GlobalID needs a persisted record
- Only writing an HTML template; some clients and spam filters prefer a text part too

✓ TIPS & BEST PRACTICES

- Inline CSS with `premailer-rails` or keep styles simple; many clients strip `<style>`
- Use `rails_blob_url` for Active Storage images in emails
- Set `config.action_mailer.deliver_later_queue_name = :mailers`

QUICK SUMMARY

- mailer = controller for email
- `deliver_later` — via Active Job
- previews — `/rails/mailers`
- `_url` helpers — absolute links

Active Storage

FILE UPLOADS BUILT IN

Active Storage attaches files to records and stores them on local disk, S3-compatible services or Google Cloud Storage. Metadata lives in two tables (`active_storage_blobs` and `_attachments`), so models need no file columns. Image **variants** are generated on demand with libvips through the `image_processing` gem.

NOTES

- Install tables with `bin/rails active_storage:install` (new apps include it)
- Variants need `image_processing` and libvips (or ImageMagick)
- There are no built-in file validations; check `content_type` and `byte_size` yourself

MODEL

```ruby
class User < ApplicationRecord
  has_one_attached :avatar do |a|
    a.variant :thumb, resize_to_limit: [100, 100], preprocessed: true
  end
  has_many_attached :documents, dependent: :purge_later
  validate { errors.add(:avatar, "too big") if avatar.attached? && avatar.byte_size > 5.megabytes }
end
```

CODE EXPLAINED

- → `variant :thumb` — named variant; `preprocessed: true` creates it after upload
- → `attach(io:, filename:)` — attach from code or scripts
- → `with_attached_avatar` — eager-load attachments and blobs
- → `disposition: "attachment"` — force download
- → `direct_upload: true` — browser uploads to the service, skipping your app server

ATTACHING & DISPLAYING

```ruby
user.avatar.attach(params[:avatar])                    # from a form
user.avatar.attach(io: File.open("me.png"), filename: "me.png")
user.avatar.attached?                                  # true
user.avatar.purge_later                                # delete async
User.with_attached_avatar                              # avoid N+1
```

VIEWS

```erb
<%= image_tag user.avatar.variant(:thumb) if user.avatar.attached? %>
<%= link_to "Download", rails_blob_path(doc, disposition: "attachment") %>
<%= form.file_field :documents, multiple: true, direct_upload: true %>
```

THE RAILS WAY

- A `file_path` string column and manual uploads → `has_one_attached`
- Resizing images on upload by hand → named variants
- Uploading large files through Puma → direct uploads

CONFIG/STORAGE.YML

```yaml
local:
  service: Disk
  root: <%= Rails.root.join("storage") %>
amazon:
  service: S3
  access_key_id: <%= Rails.application.credentials.dig(:aws, :access_key_id) %>
  secret_access_key: <%= Rails.application.credentials.dig(:aws, :secret_access_key) %>
  bucket: my-app-<%= Rails.env %>
  region: us-east-1
```

SERVICES

SERVICE	NOTES
`Disk`	Local; development or single server
`S3`	AWS S3 and compatible (R2, MinIO, Spaces)
`Mirror`	Write to several services during migrations

VARIANT TRANSFORMATIONS

OPTION	EFFECT
`resize_to_limit`	Fit within box, keep ratio
`resize_to_fill`	Crop to exact size
`format: :webp`	Convert format
`saver: { quality: 80 }`	Compression

⚠ COMMON MISTAKES

- Trusting the browser-sent content type; validate and consider `content_type` analysis
- Rendering `image_tag user.avatar` when nothing is attached → error; check `attached?`
- Using the local Disk service on multiple app servers; files exist on only one machine

✓ TIPS & BEST PRACTICES

- Serve files through a CDN with `config.active_storage.resolve_model_to_route = :rails_storage_proxy`
- Use `has_one_attached ... dependent: :purge_later` (the default) so deletes don't block requests
- Add the `active_storage_validations` gem for declarative file validations

QUICK SUMMARY

- `has_one_attached` — single file
- `has_many_attached` — many files
- variants — on-demand images
- direct uploads — straight to storage

Action Text

RICH TEXT CONTENT

Action Text adds rich text fields to any model. The content is stored in a separate `action_text_rich_texts` table, so no column is needed on your model. Images and files dropped into the editor are stored with Active Storage. The default editor is **Trix**; 37signals' newer **Lexxy** editor is available as a gem.

NOTES

- Install: `bin/rails action_text:install` (adds JS, CSS, tables)
- Generator field type: `bin/rails g model Article content:rich_text`
- Content is sanitized on render; embedded attachments render through partials
- Eager-load with `with_rich_text_content` to avoid N+1 queries

MODEL

```
class Article < ApplicationRecord
  has_rich_text :content
  validates :content, presence: true
end
```

FORM & DISPLAY

```
<%= form_with model: @article do |f| %>
  <%= f.label :content %>
  <%= f.rich_textarea :content %>
  <%= f.submit %>
<% end %>

<div class="trix-content"><%= @article.content %></div>
```

WORKING WITH CONTENT

```
article.content.to_plain_text          # "Hello world"
article.content.body.to_html
article.content.embeds                 # attached blobs
Article.with_rich_text_content.limit(10)
article.update!(content: "<p>New <strong>text</strong></p>")
params.expect(article: [:title, :content])   # permit like a normal attribute
```

CODE EXPLAINED

- → `has_rich_text :content` — `article.content` returns an `ActionText::RichText`
- → `f.rich_textarea` — 8.0 name; `rich_text_area` still works
- → `trix-content` class — editor styling for displayed content
- → `to_plain_text` — for search indexes, previews, emails
- → Permit the field like any attribute

THE RAILS WAY

- A `text` column with raw HTML from a JS editor → `has_rich_text`
- Storing uploaded images in a custom table → Action Text embeds via Active Storage
- Sanitizing HTML by hand on display → let Action Text's sanitizer handle it

EDITORS

EDITOR	NOTES
`Trix`	Default, stable, simple toolbar
`Lexxy (gem)`	Lexical-based; tables, code, markdown shortcuts
`Third-party`	Rhino, TipTap wrappers

RENDERING PIECES

FILE	PURPOSE
`layouts/action_text/contents/_content`	Wrapper for content
`active_storage/blobs/_blob`	Embedded file/image
`to_attachable_partial_path`	Custom embeds (mentions)

⚠ COMMON MISTAKES

- Querying `where(content: ...)` on the model; content lives in another table
- Forgetting `with_rich_text_*` when listing many records
- Rendering `content.body.to_s` with `raw`, bypassing the sanitizer

✓ TIPS & BEST PRACTICES

- Make any model embeddable (e.g. @mentions) with `include ActionText::Attachable`
- Index `to_plain_text` for full-text search
- Try Lexxy for new projects that need tables or code blocks

QUICK SUMMARY

- `has_rich_text` — rich field
- `rich_textarea` — editor input
- Active Storage — embeds
- `to_plain_text` — text only

Sheet 62

Action Cable & Solid Cable

channels · stream_from · broadcast · identified_by · cable.yml · Solid Cable · JS consumer

WEBSOCKETS IN RAILS

Action Cable adds WebSockets: a browser subscribes to a **channel** and the server pushes messages to it. Turbo Streams broadcasting uses Action Cable underneath, so many apps never write a channel. Rails 8 uses **Solid Cable**, which relays messages through the database instead of Redis.

NOTES

- `Connection` authenticates the socket once; channels authorize subscriptions
- Use channels directly for chat, presence, or custom client-side handling
- Solid Cable polls a table (default every 0.1 s) and trims old messages

APP/CHANNELS/APPLICATION_CABLE/CONNECTION.RB

```
module ApplicationCable
  class Connection < ActionCable::Connection::Base
    identified_by :current_user

    def connect
      session = Session.find_by(id: cookies.signed[:session_id])
      self.current_user = session&.user || reject_unauthorized_connection
    end
  end
end
```

APP/CHANNELS/ROOM_CHANNEL.RB

```
class RoomChannel < ApplicationCable::Channel
  def subscribed
    @room = current_user.rooms.find(params[:id])
    stream_for @room
  end

  def speak(data)                          # called from JS
    @room.messages.create!(user: current_user, body: data["body"])
  end
end

RoomChannel.broadcast_to(room, { body: "Hi", from: "Ann" })   # from anywhere
```

JAVASCRIPT CONSUMER

```
import consumer from "channels/consumer"
const room = consumer.subscriptions.create({ channel: "RoomChannel", id: 7 }, {
  received(data) { appendMessage(data) },
  speak(body)    { this.perform("speak", { body }) }
})
```

CODE EXPLAINED

- → `cookies.signed[:session_id]` — reuse the auth generator's session cookie
- → `reject_unauthorized_connection` — refuse the socket
- → `stream_for @room` — subscribe to a stream named for the record
- → `perform("speak")` — calls the channel's `speak` method
- → `config/cable.yml`: `adapter: solid_cable` — DB-backed pub/sub, no Redis

THE RAILS WAY

- Custom channels for "update this list" → Turbo Stream broadcasts
- Polling endpoints with `setInterval` → Action Cable or Turbo Streams
- Trusting `params[:id]` in `subscribed` → scope through `current_user`

ADAPTERS

ADAPTER	USE
`solid_cable`	Rails 8 default, database-backed
`redis`	High-volume, existing Redis
`async / test`	Development / tests

CHANNEL METHODS

METHOD	PURPOSE
`stream_from "name"`	Subscribe by string
`stream_for record`	Subscribe by model
`broadcast_to`	Send to a model stream
`unsubscribed`	Cleanup / presence

⚠ COMMON MISTAKES

- Authorizing only in `connect` and letting any user subscribe to any room id
- Broadcasting from inside a transaction; clients may refetch before commit
- Using the `async` adapter in production with multiple processes

✓ TIPS & BEST PRACTICES

- Start with Turbo Streams; write a channel only when the client needs custom JS handling
- Keep payloads small; send ids or rendered HTML, not whole records
- Test channels with `ActionCable::Channel::TestCase` and `assert_broadcast_on`

QUICK SUMMARY

- Connection — authenticate socket
- Channel — subscribe and act
- `broadcast_to` — push messages
- Solid Cable — DB-backed default

Caching & Solid Cache

DO EXPENSIVE WORK ONCE

Rails caches at several levels: **low-level** values with `Rails.cache.fetch`, **fragments** of views with `cache`, and whole responses with HTTP conditional GET. Cache keys include the record's `updated_at` version, so changing a record invalidates its fragments automatically. Rails 8 uses **Solid Cache**, a disk-friendly database-backed store.

NOTES

- Caching is off in development by default; toggle with `bin/rails dev:cache`
- Key-based expiration: no manual deletes when keys include versions
- `touch: true` on `belongs_to` bumps the parent so nested fragments refresh
- Solid Cache can hold far more than RAM-based stores because it uses disk

LOW-LEVEL

```
Rails.cache.fetch(["weather", city], expires_in: 30.minutes) do
  WeatherApi.forecast(city)                    # only on a miss
end
Rails.cache.write("k", 1); Rails.cache.read("k"); Rails.cache.delete("k")
Rails.cache.fetch_multi(*posts) { |post| post.expensive_summary }
Rails.cache.increment("hits")
```

CODE EXPLAINED

- → Array keys — joined; models contribute `cache_key_with_version`
- → `fetch_multi` — one round trip for many keys
- → `cache @post` — fragment invalidates when `@post.updated_at` changes
- → Nested caches reuse inner fragments when only one comment changes
- → `touch: true` — keeps the outer fragment fresh

FRAGMENTS & RUSSIAN DOLLS

```
<% cache @post do %>                              <%# key: views/posts/1-20261023… %>
  <h1><%= @post.title %></h1>
  <% @post.comments.each do |comment| %>
    <% cache comment do %><%= render comment %><% end %>
  <% end %>
<% end %>
<%= render partial: "comments/comment", collection: @post.comments, cached: true %>
```

THE RAILS WAY

- Manual `Rails.cache.delete` in callbacks → version-based keys
- Caching a loop of renders → collection rendering with `cached: true`
- Memcached/Redis just for cache in a small app → Solid Cache on the database

MODEL & CONFIG

```
class Comment < ApplicationRecord
  belongs_to :post, touch: true                  # comment change → post updated_at
end

# config/environments/production.rb
config.cache_store = :solid_cache_store
```

CACHE STORES

STORE	NOTES
`:solid_cache_store`	Rails 8 default, DB-backed, large
`:memory_store`	Per process; dev/test
`:redis_cache_store`	Shared, in-memory
`:mem_cache_store`	Memcached
`:file_store`	Disk, single host
`:null_store`	No caching (tests)

FETCH OPTIONS

OPTION	EFFECT
`expires_in:`	TTL
`race_condition_ttl:`	Avoid dog-piling on expiry
`force: true`	Recompute now
`skip_nil: true`	Don't cache nil
`version:`	Explicit version for recyclable keys

⚠ COMMON MISTAKES

- Caching per-user content under a shared key, leaking data between users
- Forgetting `touch: true`, so edited comments show stale inside a cached post
- Caching ActiveRecord relations (lazy) instead of computed values

✓ TIPS & BEST PRACTICES

- Include `Current.user` or locale in keys for personalized fragments
- Use `bin/rails dev:cache` to test caching locally
- Watch `cache_read.active_support` events to measure hit rates

QUICK SUMMARY

- `Rails.cache.fetch` — compute once
- `cache record` — fragment caching
- `touch: true` — Russian dolls
- Solid Cache — default store

Testing Basics & Fixtures

MINITEST, THE RAILS DEFAULT

Rails uses **Minitest** with Rails-specific test case classes and assertions. Each test runs in a transaction that rolls back, so tests don't affect each other. **Fixtures** are YAML files of sample rows loaded once into the test database and referenced by name.

NOTES

- Folders: `test/models`, `controllers`, `integration`, `system`, `mailers`, `jobs`, `fixtures`
- `test "name" do ... end` defines `test_name` methods
- `parallelize(workers: :number_of_processors)` is on by default
- Fixtures support ERB and reference associations by label

TEST/MODELS/POST_TEST.RB

```ruby
require "test_helper"

class PostTest < ActiveSupport::TestCase
  test "requires a title" do
    post = Post.new(title: "")
    assert_not post.valid?
    assert_includes post.errors[:title], "can't be blank"
  end

  test "publishing sets published_at" do
    post = posts(:draft)
    post.publish!
    assert post.published?
    assert_not_nil post.published_at
  end
end
```

CODE EXPLAINED

- → `posts(:draft)` — load a fixture by label
- → `assert_not` — reads better than `assert !x`
- → `user: ann` — fixture association, no ids needed
- → `:12` — run the test at that line
- → Transactions — each test's writes roll back

TEST/FIXTURES/POSTS.YML

```yaml
hello:
  title: Hello Rails
  user: ann                        # association by fixture label
  published: true
  created_at: <%= 2.days.ago %>
draft:
  title: Work in progress
  user: ann
```

THE RAILS WAY

- Creating every record in `setup` → fixtures for shared baseline data
- `assert post.valid? == false` → `assert_not post.valid?`
- Testing private methods → test public behavior

RUNNING

```
bin/rails test                          # all except system tests
bin/rails test test/models/post_test.rb:12   # one test by line
bin/rails test -n /publish/             # by name pattern
bin/rails test:all                      # including system tests
```

COMMON ASSERTIONS

ASSERTION	CHECKS
`assert / assert_not`	Truthy / falsy
`assert_equal exp, act`	Equality
`assert_includes coll, x`	Membership
`assert_raises(Err) { }`	Raises
`assert_difference / no_difference`	Count changes
`assert_changes`	Value changes `from:` / `to:`

TEST CASE CLASSES

CLASS	FOR
`ActiveSupport::TestCase`	Models, plain Ruby
`ActionDispatch::IntegrationTest`	Requests (sheet 65)
`ApplicationSystemTestCase`	Browser (sheet 66)
`ActionMailer::TestCase`	Mailers
`ActiveJob::TestCase`	Jobs

⚠ COMMON MISTAKES

- Relying on fixture ids; reference by label because ids are hashed from labels
- Tests that depend on run order; parallel and random ordering will expose them
- Fixtures that bypass validations becoming invalid over time; add a test that validates all fixtures

✓ TIPS & BEST PRACTICES

- Use `bin/rails test --fail-fast` (`-f`) while iterating
- Keep fixtures small and realistic; add factories only if fixtures get unwieldy
- Run `bin/rails test` in `bin/ci` (8.1) before pushing

QUICK SUMMARY

- Minitest — Rails default
- fixtures — named YAML records
- `assert_difference` — count changes
- transactions — isolated tests

Model & Integration Tests

TESTING REQUESTS END TO END

Integration tests send real HTTP requests through the full Rails stack (routing, middleware, controllers, views) without a browser. They are fast and catch most bugs. New apps generate controller tests as `ActionDispatch::IntegrationTest`, so "controller test" and "integration test" are the same thing today.

NOTES

- Use `_url` helpers: `get posts_url`
- Params go in `params:`; JSON bodies with `as: :json`
- `assert_select` checks rendered HTML with CSS selectors
- Log in through the real login route, or a helper that does

TEST/TEST_HELPER.RB (SIGN-IN HELPER)

```ruby
class ActionDispatch::IntegrationTest
  def sign_in_as(user, password: "password")
    post session_url, params: { email_address: user.email_address, password: }
  end
end
```

TEST/CONTROLLERS/POSTS_CONTROLLER_TEST.RB

```ruby
require "test_helper"

class PostsControllerTest < ActionDispatch::IntegrationTest
  setup { sign_in_as users(:ann) }

  test "lists posts" do
    get posts_url
    assert_response :success
    assert_select "h2", posts(:hello).title
  end

  test "creates a post" do
    assert_difference("Post.count") do
      post posts_url, params: { post: { title: "New", body: "Text" } }
    end
    assert_redirected_to post_url(Post.last)
    follow_redirect!
    assert_select ".flash", "Post created"
  end
end
```

CODE EXPLAINED

→ `sign_in_as` — posts to the real session route (auth generator)

→ `assert_select "h2", text` — element with that text exists

→ `assert_redirected_to` — checks the `Location` header

→ `follow_redirect!` — perform the next GET

→ `get url, as: :json` + `response.parsed_body` — test JSON endpoints

THE RAILS WAY

- Old `ActionController::TestCase` with `assigns` → integration tests asserting on output
- Stubbing the current user → log in through the real flow
- Testing every branch in system tests → integration tests; save the browser for JS

REQUEST HELPERS

CALL	SENDS
`get url, params:`	GET with query string
`post url, params:`	Form POST
`patch / put / delete`	Other verbs
`as: :json`	JSON body + Accept header
`headers: { ... }`	Custom headers

RESPONSE ASSERTIONS

ASSERTION	CHECKS
`assert_response :success`	2xx
`assert_response :redirect`	3xx
`assert_redirected_to url`	Location
`assert_select "css"`	HTML content
`assert_turbo_stream`	Stream actions

⚠ COMMON MISTAKES

- Using `_path` helpers in integration tests; it works but `_url` matches redirects exactly
- Asserting on instance variables (`assigns`); test what the user sees
- Forgetting fixtures need a known password digest for sign-in: `password_digest: <%= BCrypt::Password.create("password") %>`

✓ TIPS & BEST PRACTICES

- Write one happy-path and one failure-path test per action
- Use `assert_select "form[action=?]", posts_path` to check forms
- Test JSON with `as: :json` and `parsed_body` instead of string matching

QUICK SUMMARY

- `IntegrationTest` — full request stack
- `get / post` — make requests
- `assert_response` / `assert_select` — check output
- `sign_in_as` — real login

System Tests

ApplicationSystemTestCase · driven_by · Capybara visit / click_on / fill_in · assert_selector · headless Chrome · screenshots

TESTING IN A REAL BROWSER

System tests drive a real browser through **Capybara**, so JavaScript, Turbo and Stimulus behavior is exercised. They are the slowest tests, so keep them for critical user journeys. Failures save a screenshot automatically in `tmp/screenshots`.

NOTES

- `bin/rails test` skips system tests; run `bin/rails test:system` or `test:all`
- Default driver: Selenium with headless Chrome
- Capybara waits automatically for elements to appear (up to `default_max_wait_time`)
- Generate with `bin/rails g system_test posts`

TEST/APPLICATION_SYSTEM_TEST_CASE.RB

```ruby
class ApplicationSystemTestCase < ActionDispatch::SystemTestCase
  driven_by :selenium, using: :headless_chrome, screen_size: [1400, 1000]

  def sign_in_as(user)
    visit new_session_url
    fill_in "email_address", with: user.email_address
    fill_in "password", with: "password"
    click_on "Sign in"
  end
end
```

CODE EXPLAINED

- → `driven_by` — browser, headless mode, window size
- → `fill_in "Title"` — finds by label, name or id
- → `fill_in_rich_textarea` — Action Text helper (8.0 name)
- → `accept_confirm { click_on "Delete" }` — handle the Turbo confirm dialog
- → `assert_text` — waits for the text to appear

TEST/SYSTEM/POSTS_TEST.RB

```ruby
require "application_system_test_case"

class PostsTest < ApplicationSystemTestCase
  setup { sign_in_as users(:ann) }

  test "writing a post" do
    visit posts_url
    click_on "New post"
    fill_in "Title", with: "System tests"
    fill_in_rich_textarea "Body", with: "Run in Chrome"
    select "Tutorials", from: "Category"
    check "Featured"
    click_on "Create Post"

    assert_text "Post created"
    assert_selector "h1", text: "System tests"
  end
end
```

THE RAILS WAY

- `sleep 2` waiting for JS → Capybara's waiting matchers (`assert_text`)
- System tests for every validation → integration tests; system tests for flows
- Selecting by CSS classes → by visible labels and text, like a user

CAPYBARA ACTIONS

METHOD	DOES
`visit url`	Navigate
`click_on "text"`	Click link or button
`fill_in "label", with:`	Type into field
`select "opt", from:`	Choose option
`attach_file "label", path`	Upload
`within("#id") { }`	Scope actions

MATCHERS

ASSERTION	WAITS FOR
`assert_text`	Text on page
`assert_selector "css", text:`	Element
`assert_current_path`	URL
`take_screenshot`	Manual screenshot

⚠ COMMON MISTAKES

- Using `find(...).click` then asserting with non-waiting Ruby (`page.html.include?`)
- Sharing state between system tests through the database without fixtures
- Forgetting system tests don't run in plain `bin/rails test`

✓ TIPS & BEST PRACTICES

- Use `driven_by :cuprite` (gem) for faster Chrome without Selenium
- Run with `HEADLESS=0`-style toggles locally to watch the browser
- Keep a small "smoke" suite of system tests for sign-in, checkout and other critical paths

QUICK SUMMARY

- system test — real browser
- Capybara — user-like actions
- waiting matchers — no sleeps
- `test:system` — run them

Testing Jobs, Mailers & Time

SIDE EFFECTS UNDER CONTROL

Rails' test helpers capture enqueued jobs and sent emails instead of running or delivering them, and let you **travel in time**. Assert that something was enqueued, run it inline when you need its effect, and freeze the clock to test expirations and schedules.

NOTES

- The test adapter queues jobs without running them
- `perform_enqueued_jobs { }` runs jobs enqueued inside the block
- Delivered mail collects in `ActionMailer::Base.deliveries`
- Time helpers come from `ActiveSupport::Testing::TimeHelpers` (included by default)

JOBS

```
class SignupTest < ActiveSupport::TestCase
  include ActiveJob::TestHelper

  test "enqueues the welcome email job" do
    assert_enqueued_with(job: SendDigestJob, args: [users(:ann)], queue: "mailers") do
      users(:ann).request_digest!
    end
  end

  test "runs jobs" do
    perform_enqueued_jobs do
      users(:ann).request_digest!
    end
    assert_emails 1                    # delivered by the job
  end
end
```

CODE EXPLAINED

- → `assert_enqueued_with` — job class, args and queue
- → `perform_enqueued_jobs` — run them inline for end-to-end checks
- → `mail.html_part.body` — multipart content
- → `assert_enqueued_email_with` — `deliver_later` was called
- → `travel` / `travel_to` — shift the clock; reset after each test

MAILERS

```
class UserMailerTest < ActionMailer::TestCase
  test "welcome" do
    mail = UserMailer.with(user: users(:ann)).welcome
    assert_equal ["ann@example.com"], mail.to
    assert_equal "Welcome, Ann!", mail.subject
    assert_match "Get started", mail.html_part.body.to_s
  end
end
# anywhere: assert_enqueued_email_with UserMailer, :welcome, params: { user: }
```

THE RAILS WAY

- Stubbing `Time.now` by hand → `travel_to` / `freeze_time`
- Asserting jobs by checking side effects only → `assert_enqueued_with` first, then a job unit test
- Sending real mail in tests → the `:test` delivery method (default)

TIME

```
test "reset token expires after 15 minutes" do
  token = users(:ann).generate_token_for(:password_reset)
  travel 16.minutes
  assert_nil User.find_by_token_for(:password_reset, token)
end

travel_to(Time.zone.local(2026, 12, 25, 9)) { assert Holiday.today? }
```

JOB ASSERTIONS

ASSERTION	CHECKS
`assert_enqueued_jobs n`	Count enqueued
`assert_enqueued_with`	Specific job
`assert_performed_jobs n`	Count performed
`perform_enqueued_jobs(only: X)`	Run selected jobs

MAIL & TIME

HELPER	USE
`assert_emails n { }`	Delivered count
`assert_enqueued_emails n`	`deliver_later` count
`travel_to time`	Set now
`travel_back`	Restore (automatic after test)

⚠ COMMON MISTAKES

- Testing a job only via `perform_enqueued_jobs` in a slow integration test; unit-test `perform_now` too
- Calling `Time.now` in code under test; `travel_to` stubs it, but `Time.current` is the safer habit
- Expecting `assert_emails` to see `deliver_later` mail without running jobs

✓ TIPS & BEST PRACTICES

- Test jobs directly: `SendDigestJob.perform_now(users(:ann))`
- Use mailer previews for visual checks and tests for content
- Freeze time in tests that format dates to avoid midnight flakes

QUICK SUMMARY

- `assert_enqueued_with` — job queued
- `perform_enqueued_jobs` — run inline
- `assert_emails` — mail sent
- `travel_to` — control time

RSpec Rails & FactoryBot

THE POPULAR ALTERNATIVE STACK

Many teams use **RSpec** instead of Minitest, with **FactoryBot** instead of fixtures. RSpec's `describe` / `it` style reads like specification, and factories build records on demand with only the attributes each test cares about. `rspec-rails` adds Rails-aware spec types and generators.

NOTES

- Install: add `rspec-rails` and `factory_bot_rails` to the test group, then `bin/rails g rspec:install`
- Generators create specs instead of tests once rspec-rails is installed
- Run with `bundle exec rspec` or `bin/rspec`

SPEC/FACTORIES/USERS.RB

```
FactoryBot.define do
  factory :user do
    sequence(:email_address) { |n| "user#{n}@example.com" }
    password { "password" }
    name { "Ann" }
    trait(:admin) { role { :admin } }
  end

  factory :post do
    user
    title { "Hello" }
    trait(:published) { published { true } }
  end
end
```

CODE EXPLAINED

- → `sequence` — unique values per record
- → `trait` — named variations: `create(:user, :admin)`
- → `user` inside a factory — builds the association
- → `expect { }.to change(...)` — like `assert_difference`
- → shoulda-matchers — `it { is_expected.to validate_presence_of(:title) }`

SPEC/REQUESTS/POSTS_SPEC.RB

```
require "rails_helper"

RSpec.describe "Posts", type: :request do
  let(:user) { create(:user) }
  before { post session_path, params: { email_address: user.email_address, password: "password" } }

  describe "POST /posts" do
    it "creates a post" do
      expect {
        post posts_path, params: { post: { title: "New" } }
      }.to change(Post, :count).by(1)
      expect(response).to redirect_to(Post.last)
    end
  end
end
```

THE RAILS WAY

- `create` everywhere → `build` / `build_stubbed` when the DB isn't needed
- Huge factories with every attribute → minimal valid defaults + traits
- Controller specs → request specs

FACTORYBOT STRATEGIES

METHOD	SAVES?
`build(:post)`	no, unsaved
`create(:post)`	yes
`build_stubbed(:post)`	no, fake id, fastest
`create_list(:post, 3)`	yes, three

MINITEST ↔ RSPEC

MINITEST	RSPEC
`test "x" do`	`it "x" do`
`assert_equal a, b`	`expect(b).to eq(a)`
`assert_difference`	`change { }.by(n)`
`setup / fixtures`	`before`, `let` / factories

⚠ COMMON MISTAKES

- Factories that create long association chains for every record, making suites slow
- Overusing `let!`, which creates records even in tests that don't need them
- Mixing fixtures and factories for the same model without a clear rule

✓ TIPS & BEST PRACTICES

- Run `FactoryBot.lint` in CI to catch invalid factories
- Use `bin/rspec --only-failures` with `example_status_persistence_file_path`
- Choose one stack per app; both are fully supported

QUICK SUMMARY

- rspec-rails — spec types + generators
- FactoryBot — records on demand
- traits — variations
- `build_stubbed` — fast specs

Local CI & Code Quality

RUN CI ON YOUR MACHINE

Rails 8.1 adds `bin/ci`, which runs the steps in `config/ci.rb`: setup, style, security audits and tests. Modern developer machines are fast, so running CI locally before pushing is quick and cheap. New apps also ship **RuboCop** with the Omakase style, **Brakeman**, **bundler-audit** and a GitHub Actions workflow.

NOTES

- `bin/ci` stops at the first failing step and prints a summary
- The optional `gh signoff` step marks the commit green on GitHub
- `bin/rubocop -a` auto-corrects safe offenses
- Brakeman exits non-zero on new warnings; add an ignore file for accepted ones

CONFIG/CI.RB

```
CI.run do
  step "Setup", "bin/setup --skip-server"
  step "Style: Ruby", "bin/rubocop"
  step "Security: Gem audit", "bin/bundler-audit"
  step "Security: Importmap audit", "bin/importmap audit"
  step "Security: Brakeman", "bin/brakeman --quiet --no-pager --exit-on-warn --exit-on-error"
  step "Tests: Rails", "bin/rails test"
  step "Tests: System", "bin/rails test:system"
  step "Tests: Seeds", "env RAILS_ENV=test bin/rails db:seed:replant"

  if success?
    step "Signoff: Ready to merge", "gh signoff"
  else
    failure "Signoff: CI failed. Do not merge or deploy.", "Fix the issues and try again."
  end
end
```

CODE EXPLAINED

- → `step "name", "command"` — each step is a shell command
- → `success?` — true if every step so far passed
- → `gh signoff` — sets a commit status via the GitHub CLI extension
- → `db:seed:replant` — proves seeds still run on an empty DB
- → `inherit_gem` — start from the Omakase style, override sparingly

RUNNING THE TOOLS

```
bin/ci                          # everything in config/ci.rb
bin/rubocop                     # lint
bin/rubocop -a                  # safe auto-correct
bin/brakeman -I                 # interactive ignore file
bin/bundler-audit check --update
```

THE RAILS WAY

- Waiting on remote CI for lint failures → run `bin/ci` before pushing
- Arguing about style in review → RuboCop with the shared config
- Ignoring Brakeman warnings by turning it off → record accepted ones in `config/brakeman.ignore`

.RUBOCOP.YML

```
inherit_gem:
  rubocop-rails-omakase: rubocop.yml
Layout/LineLength:
  Max: 120
```

QUALITY TOOLS

TOOL	CATCHES
`RuboCop (omakase)`	Style and simple bugs
`Brakeman`	SQL injection, XSS, unsafe redirects, mass assignment
`bundler-audit`	Vulnerable gem versions
`importmap audit`	Vulnerable JS pins
`zeitwerk:check`	Autoload naming errors
`Bullet / strict_loading`	N+1 queries

GENERATED CI FILES

FILE	ROLE
`bin/ci`	Runner (8.1)
`config/ci.rb`	Steps (8.1)
`.github/workflows/ci.yml`	Remote CI
`.github/dependabot.yml`	Gem and Actions updates

⚠ COMMON MISTAKES

- Letting local CI drift from remote CI; keep the steps in `config/ci.rb` the source of truth
- Auto-correcting with `-A` (unsafe) without reviewing the diff
- Disabling cops globally to silence one file; use inline `# rubocop:disable` with a reason

✓ TIPS & BEST PRACTICES

- Add `bin/rails zeitwerk:check` as a step; it catches production-only boot errors
- Run `bin/ci` in a git pre-push hook
- Keep CI under a few minutes so people actually run it

QUICK SUMMARY

- `bin/ci` — local CI runner (8.1)
- `config/ci.rb` — the steps
- RuboCop + Brakeman + audits — built in
- `gh signoff` — mark commit green

API-Only Apps & JSON

RAILS AS A JSON BACKEND

`rails new --api` builds a slimmer app for JSON clients: controllers inherit `ActionController::API`, browser-only middleware (cookies, flash, CSRF) is removed, and generators skip views. You can also add API controllers to a normal app. JSON comes from `render json:` with `as_json`, **Jbuilder** templates, or a serializer gem.

NOTES

- `ActionController::API` excludes views, cookies, sessions and CSRF by default
- Token auth is common: `authenticate_with_http_token` or signed tokens
- Version with URL namespaces: `/api/v1/...`
- Browsers calling from another origin need CORS headers (`rack-cors`)

ROUTES & CONTROLLER

```ruby
namespace :api do
  namespace :v1 do
    resources :posts, only: %i[index show create], defaults: { format: :json }
  end
end

class Api::V1::BaseController < ActionController::API
  include ActionController::HttpAuthentication::Token::ControllerMethods
  before_action :authenticate

  private

  def authenticate
    authenticate_or_request_with_http_token do |token, _opts|
      @client = ApiClient.find_by(api_key: token)
    end
  end
end

class Api::V1::PostsController < Api::V1::BaseController
  def index
    posts = Post.published.order(id: :desc).limit(50)
    render json: posts.as_json(only: %i[id title], include: { user: { only: :name } })
  end

  def create
    post = @client.posts.new(params.expect(post: [:title, :body]))
    if post.save then render json: post, status: :created, location: api_v1_post_url(post)
    else render json: { errors: post.errors }, status: :unprocessable_entity end
  end
end
```

CODE EXPLAINED

→ `defaults: { format: :json }` — no `.json` suffix needed

→ `authenticate_or_request_with_http_token` — parses `Authorization: Token ...` / `Bearer`

→ `as_json(only:, include:)` — shape output without a template

→ `status: :created, location:` — RESTful create response

→ Jbuilder — `show.json.jbuilder` : `json.extract! @post, :id, :title`

THE RAILS WAY

- `to_json` of whole records (leaks columns) → `as_json(only:)` , Jbuilder or a serializer
- Inconsistent error formats → one `rescue_from` + error JSON shape in a base controller
- Breaking changes on the same URL → a new `v2` namespace

JSON OPTIONS

TOOL	NOTES
`render json: obj`	Calls `as_json`
`as_json(only:, except:, methods:)`	Quick shaping
`Jbuilder`	Templates, partials, `json.cache!`
`Alba / Blueprinter`	Fast serializer classes
`ActiveModel::Serializers`	Older, less maintained

CONFIG/INITIALIZERS/CORS.RB

SETTING	EXAMPLE
`origins`	`"app.example.com"`
`resource`	`"/api/*"`
`headers`	`:any`
`methods`	`%i[get post patch delete options]`

⚠ COMMON MISTAKES

- Rendering `@user.to_json` , exposing `password_digest` and other private columns
- Using cookie auth for a cross-site API without CSRF protection
- Setting CORS `origins "*"` together with credentials

✓ TIPS & BEST PRACTICES

- Paginate every list endpoint (`pagy` gem) and return pagination headers
- Return errors as `{ errors: { field: [messages] } }` consistently
- Use request specs or integration tests with `as: :json` for every endpoint

QUICK SUMMARY

- `--api` — lean JSON app
- `ActionController::API` — no browser middleware
- `as_json` / Jbuilder — shape JSON
- `api/v1` — version namespace

Internationalization

TRANSLATING THE APP

Rails' **I18n** API looks up strings by key in YAML files under `config/locales`. Views use `t(".key")` with **lazy lookup** scoped to the template path. Dates, numbers and validation messages are localized through the same files. The locale is set per request, usually from the URL, the user's setting or the `Accept-Language` header.

NOTES

- Keys are nested: `en.posts.index.title`
- Missing translations show "translation missing" spans; raise instead in tests
- Model and attribute names come from `activerecord.models` / `.attributes`
- Add `rails-i18n` (gem) for date, number and error translations in many languages

CONFIG/LOCALES/EN.YML

```yaml
en:
  hello: "Hello, %{name}!"
  posts:
    index:
      title: "All posts"
      count:
        zero: "No posts"
        one: "1 post"
        other: "%{count} posts"
  activerecord:
    models:
      post: "Article"
    attributes:
      post:
        title: "Headline"
```

CODE EXPLAINED

- → `%{name}` — interpolation variable
- → `zero/one/other` — pluralization keys chosen by `count:`
- → `t(".title")` — lazy lookup from `posts/index` template path
- → `Post.model_name.human` / `human_attribute_name` — "Article", "Headline"
- → `with_locale` — set the locale for one request only

USING TRANSLATIONS

```erb
<h1><%= t(".title") %></h1>                          <%# posts.index.title %>
<p><%= t(".count", count: @posts.size) %></p>
<p><%= t("hello", name: Current.user.name) %></p>
<p><%= l(@post.created_at, format: :short) %></p>
<%= link_to t(".new", default: "New post"), new_post_path %>
```

SETTING THE LOCALE

```ruby
# config/application.rb
config.i18n.available_locales = %i[en es]
config.i18n.default_locale = :en
config.i18n.raise_on_missing_translations = true   # in test/development env files

class ApplicationController < ActionController::Base
  around_action :switch_locale
  def switch_locale(&action)
    locale = params[:locale] || Current.user&.locale || I18n.default_locale
    I18n.with_locale(locale, &action)
  end
  def default_url_options = { locale: I18n.locale }
end
```

THE RAILS WAY

- Hard-coded English strings in views → `t(".key")`
- `I18n.locale = x` in a before_action → `around_action` with `I18n.with_locale`
- Custom date formatting → `l(date, format:)` with formats in YAML

I18N HELPERS

HELPER	USE
`t / translate`	Look up a key
`l / localize`	Dates and times
`number_to_currency`	Uses `number.currency.format`
`t(".x_html")`	Keys ending `_html` are HTML-safe

TOOLS

TOOL	DOES
`i18n-tasks (gem)`	Find missing / unused keys
`rails-i18n (gem)`	Built-in translations
`scope "(:locale)"`	Locale in URLs (sheet 09)

⚠ COMMON MISTAKES

- Assigning `I18n.locale` globally in a threaded server, leaking locale between requests
- Concatenating translated fragments; word order differs by language. Interpolate instead
- Forgetting pluralization rules differ by language (some have `few`, `many`)

✓ TIPS & BEST PRACTICES

- Turn on `raise_on_missing_translations` in test so CI catches gaps
- Run `i18n-tasks missing` before releases
- Use `_html` keys only for trusted markup; interpolated values are still escaped

QUICK SUMMARY

- `t(".key")` — lazy lookup
- `l(date)` — localized dates
- `with_locale` — per request
- YAML in `config/locales` — the strings

Rack & Middleware

UNDER THE CONTROLLER

Rails is a **Rack** application. Every request passes through a stack of **middleware** (static files, logging, sessions, cookies, CSRF headers) before reaching the router. Middleware objects wrap the next app and can inspect or change the request and response, which makes them ideal for cross-cutting concerns like throttling, request ids or maintenance mode.

NOTES

- A Rack app responds to `call(env)` and returns `[status, headers, body]`
- `bin/rails middleware` prints the stack in order
- API-only apps start with a smaller stack
- `config.ru` is the Rack entry point that runs `Rails.application`

A CUSTOM MIDDLEWARE

```ruby
# lib/middleware/request_timer.rb
class RequestTimer
  def initialize(app) = @app = app

  def call(env)
    started = Process.clock_gettime(Process::CLOCK_MONOTONIC)
    status, headers, body = @app.call(env)
    ms = ((Process.clock_gettime(Process::CLOCK_MONOTONIC) - started) * 1000).round
    headers["server-timing"] = "app;dur=#{ms}"
    [status, headers, body]
  end
end
```

CODE EXPLAINED

- → `initialize(app)` — receive the next app in the chain
- → `@app.call(env)` — pass the request on, get the response back
- → `server-timing` — shows in browser dev tools
- → `insert_before` — position relative to an existing middleware
- → Rack::Attack — throttling and blocklists before Rails runs

REGISTERING & REORDERING

```ruby
# config/application.rb
require_relative "../lib/middleware/request_timer"
config.middleware.use RequestTimer
config.middleware.insert_before Rack::Runtime, Rack::Attack
config.middleware.insert_after ActionDispatch::Static, Rack::Deflater
config.middleware.delete Rack::Runtime
config.middleware.swap ActionDispatch::ShowExceptions, MyShowExceptions
```

THE RAILS WAY

- A `before_action` that only adds headers to every response → middleware
- Throttling in controllers for everything → `rate_limit` per action, Rack::Attack app-wide
- Monkey-patching Rails internals → insert or swap middleware

RACK::ATTACK THROTTLE

```ruby
# config/initializers/rack_attack.rb
Rack::Attack.throttle("logins/ip", limit: 5, period: 20.seconds) do |req|
  req.ip if req.path == "/session" && req.post?
end
```

NOTABLE DEFAULT MIDDLEWARE

MIDDLEWARE	ROLE
ActionDispatch::HostAuthorization	Allowed hosts
ActionDispatch::Static	Serve `public/`
ActionDispatch::RequestId	`X-Request-Id`
ActionDispatch::ShowExceptions	Error pages
ActionDispatch::Cookies	Cookie jar
ActionDispatch::Session::CookieStore	Session
ActionDispatch::Flash	Flash messages
Rack::ETag / ConditionalGet	HTTP caching

⚠ COMMON MISTAKES

- Returning a body that isn't enumerable or closing it twice; keep `body` from the inner app
- Doing database work in middleware for every request including assets
- Adding middleware that reads the whole body for every request (uploads get slow)

✓ TIPS & BEST PRACTICES

- Keep middleware in `lib/` and `require` it: `app/` isn't autoloadable while config loads
- Use `ActionDispatch::Request.new(env)` for a friendlier request API
- Check order with `bin/rails middleware` after changes

QUICK SUMMARY

- Rack — `call(env)` → response
- middleware — wraps the app
- `config.middleware` — add/reorder
- Rack::Attack — app-wide throttling

Engines & Railties

rails plugin new · --mountable · isolate_namespace · mount Engine · Railtie · engine initializers

MINI-APPS INSIDE YOUR APP

An **engine** is a Rails app packaged as a gem: models, controllers, views, routes and migrations that mount into a host app. Devise, Mission Control and Active Storage are engines. A **Railtie** is the lighter hook gems use to add configuration, initializers or rake tasks without app code.

NOTES

- `--mountable` engines are namespaced and have their own routes
- `isolate_namespace` keeps engine helpers, tables and routes separate
- Engine route helpers are prefixed: `blog.posts_path`; host routes via `main_app.`
- Copy engine migrations into the host with `bin/rails blog:install:migrations`

CREATE & MOUNT

```
rails plugin new blog --mountable --database=sqlite3
# host Gemfile: gem "blog", path: "engines/blog"
bin/rails blog:install:migrations && bin/rails db:migrate
```

ENGINES/BLOG/LIB/BLOG/ENGINE.RB

```
module Blog
  class Engine < ::Rails::Engine
    isolate_namespace Blog                    # Blog::Post, blog_posts table

    initializer "blog.assets" do |app|
      app.config.assets.paths << root.join("app/assets/stylesheets")
    end
    config.generators { |g| g.test_framework :minitest }
  end
end
```

CODE EXPLAINED

- → `--mountable` — namespaced engine with its own router
- → `isolate_namespace` — tables prefixed (`blog_posts`), helpers scoped
- → `initializer` — runs during host boot
- → `main_app.` / `blog.` — cross-router helpers
- → Railtie — config + hooks, no app code

HOST APP

```
# config/routes.rb
mount Blog::Engine, at: "/blog"

# inside engine views
link_to "Home", main_app.root_path
# host views
link_to "Blog", blog.root_path
```

THE RAILS WAY

- A giant app with a self-contained admin or billing area → consider an engine (or just a namespace)
- Copying the same feature between apps → extract a mountable engine gem
- Monkey-patching from a gem's `lib` file → a Railtie initializer

A RAILTIE IN A GEM

```
module MyGem
  class Railtie < ::Rails::Railtie
    config.my_gem = ActiveSupport::OrderedOptions.new
    initializer("my_gem.configure") { |app| MyGem.setup(app.config.my_gem) }
    rake_tasks { load "tasks/my_gem.rake" }
  end
end
```

ENGINE TYPES

TYPE	USE
`--full`	Shares host namespace, no isolation
`--mountable`	Isolated mini-app with routes
`Railtie`	Config / tasks only
`Packwerk packages`	Boundaries without gems

ENGINE COMMANDS

COMMAND	DOES
`bin/rails g model (in engine)`	Namespaced model
`NAME:install:migrations`	Copy migrations to host
`bin/rails test (in engine)`	Run engine tests on `test/dummy`
`Engine.routes.draw`	Engine's own routes

⚠ COMMON MISTAKES

- Forgetting `main_app.` in engine layouts, so host links raise "undefined method"
- Engines depending on host models directly; pass classes in through configuration
- Skipping `install:migrations`, so engine tables never exist in the host

✓ TIPS & BEST PRACTICES

- Use the generated `test/dummy` app to test the engine in isolation
- Expose configuration like `Blog.user_class = "User"` instead of hard-coding
- Reach for a namespace first; extract an engine when a second app needs it

QUICK SUMMARY

- engine — mountable mini-app
- `isolate_namespace` — separation
- `mount` — attach at a path
- Railtie — gem hooks into Rails

Sheet 74

Debugging & Logging

debug gem · binding.break · console --sandbox · web-console · log levels · tagged logging · bullet / rack-mini-profiler

FINDING OUT WHAT HAPPENED

Rails includes the **debug** gem: drop `debugger` (or `binding.break`) in code and the server stops there with an interactive prompt. The **web-console** gem gives a console on error pages. Logs show every request, its params, SQL queries and render times, and query log tags can point each SQL line back to its controller.

NOTES

- The debugger prompt appears in the terminal running `bin/rails server` (or `bin/dev`)
- Development error pages include a live console from web-console
- `config.active_record.verbose_query_logs = true` shows the Ruby line for each query
- `Rails.logger.debug { expensive }` with a block skips work when the level is higher

BREAKPOINTS

```ruby
def update
  @post = Post.find(params.expect(:id))
  debugger                          # or binding.break
  @post.update!(post_params)
end
```

DEBUGGER COMMANDS

```
(rdbg) info               # local and instance variables
(rdbg) p @post.errors.full_messages
(rdbg) n                  # next line          s   # step into
(rdbg) c                  # continue           bt  # backtrace
(rdbg) break Post#publish!   # add a breakpoint
(rdbg) watch @post.title  # stop when it changes
```

CODE EXPLAINED

- → `debugger` — stops execution in the server process
- → `info` / `p` — inspect state at the breakpoint
- → `watch` — break when an expression changes
- → Block form of `debug` — only builds the string when logged
- → Query log tags — SQL comments name the controller/action/job

LOGGING

```ruby
Rails.logger.info "Imported #{rows.size} rows"
Rails.logger.debug { "payload: #{payload.inspect}" }
Rails.logger.tagged("Import", import.id) { Rails.logger.warn "skipped row" }

# config/environments/development.rb
config.log_level = :debug
config.active_record.verbose_query_logs = true
config.active_record.query_log_tags_enabled = true   # /*controller:posts,action:show*/
```

THE RAILS WAY

- `puts` debugging and restarting → `debugger` and inspect live
- `pp` in production code → tagged, leveled `Rails.logger` calls
- Guessing which line ran a query → `verbose_query_logs`

CONSOLE TRICKS

```ruby
Post.where(published: true).to_sql
app.get "/posts"; app.response.status      # simulate a request
ActiveRecord::Base.logger = Logger.new($stdout)   # SQL in a script
```

LOG LEVELS

LEVEL	USE
`:debug`	Development details, SQL
`:info`	Requests, key events (production default)
`:warn`	Something odd but handled
`:error`	Failures
`:fatal`	App can't continue

HELPFUL GEMS

GEM	SHOWS
`rack-mini-profiler`	Timing badge on every page
`bullet`	N+1 and unused eager loads
`web-console`	Console on error pages
`letter_opener`	Emails in the browser
`memory_profiler / stackprof`	Memory and CPU hot spots

⚠ COMMON MISTAKES

- Leaving `debugger` calls in committed code; CI or production processes hang waiting for input
- Logging secrets or full params in custom log lines
- Debugging with `bin/dev` and missing the prompt among watcher output; run the server alone

✓ TIPS & BEST PRACTICES

- Add a RuboCop rule or grep in `bin/ci` for leftover `debugger` calls
- Use `RUBY_DEBUG_OPEN=true` for remote debugging from VS Code
- Read logs by request id when concurrent requests interleave

QUICK SUMMARY

- `debugger` — breakpoint
- web-console — console on error pages
- `Rails.logger.tagged` — contextual logs
- query log tags — SQL → code

Production Configuration

READY FOR REAL TRAFFIC

`config/environments/production.rb` turns on eager loading, caching, SSL and STDOUT logging. Puma serves requests with a few threads per process. In Rails 8, **SQLite in production** is a supported default for single-server apps, with separate databases for the Solid Queue, Cache and Cable data.

NOTES

- `RAILS_ENV=production` and `RAILS_MASTER_KEY` must be set
- `/up` returns 200 when the app boots: point load balancers and Kamal at it
- Puma: `RAILS_MAX_THREADS` (default 3) and `WEB_CONCURRENCY` processes
- Run `bin/rails db:prepare` on deploy (the Docker entrypoint does this)

CONFIG/ENVIRONMENTS/PRODUCTION.RB (KEY LINES)

```
Rails.application.configure do
  config.enable_reloading = false
  config.eager_load = true
  config.consider_all_requests_local = false
  config.action_controller.perform_caching = true
  config.public_file_server.headers = { "cache-control" => "public, max-age=#{1.year.to_i}" }
  config.active_storage.service = :local
  config.assume_ssl = true                    # behind a TLS-terminating proxy
  config.force_ssl  = true                    # redirect + HSTS + secure cookies
  config.log_tags = [:request_id]
  config.logger   = ActiveSupport::TaggedLogging.logger(STDOUT)
  config.log_level = ENV.fetch("RAILS_LOG_LEVEL", "info")
  config.silence_healthcheck_path = "/up"
  config.cache_store = :solid_cache_store
  config.active_job.queue_adapter = :solid_queue
  config.action_mailer.default_url_options = { host: "example.com" }
  config.hosts = ["example.com", /.*\.example\.com/]
end
```

CODE EXPLAINED

- → `eager_load` — load all code at boot: faster requests, catches autoload errors
- → `assume_ssl` — trust that the proxy terminated HTTPS
- → STDOUT logging — containers and Kamal collect it
- → `silence_healthcheck_path` — keep `/up` out of the logs
- → `plugin :solid_queue` — one-box deploys run jobs in the web process

CONFIG/PUMA.RB

```
threads_count = ENV.fetch("RAILS_MAX_THREADS", 3)
threads threads_count, threads_count
port ENV.fetch("PORT", 3000)
workers ENV.fetch("WEB_CONCURRENCY", 1)
plugin :tmp_restart
plugin :solid_queue if ENV["SOLID_QUEUE_IN_PUMA"]   # jobs inside Puma
```

THE RAILS WAY

- Log files inside containers → STDOUT with tags
- Postgres + Redis for a small single-server app → SQLite + Solid trio (Rails 8 default)
- Custom health endpoints → the built-in `/up`

PRODUCTION CHECKLIST

ITEM	CHECK
Secrets	RAILS_MASTER_KEY set, credentials per env
Database	`db:prepare` runs, backups configured
Assets	Precompiled in the image
Storage	Persistent volume or S3 for `storage/`
Errors	Error reporter subscribed to `Rails.error`
Email	SMTP and `default_url_options` host
Security	`force_ssl`, `hosts`, Brakeman clean

SIZING PUMA

SETTING	GUIDE
RAILS_MAX_THREADS	3 is the Rails default
WEB_CONCURRENCY	≈ CPU cores
DB pool	≥ threads per process
YJIT	On by default (Ruby 3.3+)

⚠ COMMON MISTAKES

- Storing SQLite files or uploads inside the container filesystem without a volume; they vanish on deploy
- Setting the DB pool below Puma's thread count → connection timeouts
- Leaving `config.hosts` empty behind a proxy that forwards arbitrary Host headers

✓ TIPS & BEST PRACTICES

- Use `bin/rails runner -e production 'p Rails.configuration.eager_load'` to check config in the image
- Run Litestream or volume snapshots to back up SQLite databases
- Report errors with an error tracker subscribed via `Rails.error.subscribe`

QUICK SUMMARY

- `production.rb` — prod settings
- `force_ssl` / `assume_ssl` — HTTPS
- `/up` — health check
- Puma threads × workers — capacity

Sheet 76

Dockerfile, Puma & Thruster

generated Dockerfile · multi-stage build · Thruster · Puma workers / threads · bin/docker-entrypoint · db:prepare on boot

THE GENERATED CONTAINER

New Rails apps include a production **Dockerfile**. It builds in stages (gems and assets in a build stage, a slim runtime stage), runs as a non-root user, and starts the app through **Thruster**, a small proxy in front of Puma that adds HTTP/2, X-Sendfile, asset caching and gzip, plus automatic TLS when you don't use another proxy.

NOTES

- The Dockerfile targets production; use `.devcontainer` for development
- `bin/docker-entrypoint` runs `db:prepare` before the server starts
- Thruster listens on port 80 and forwards to Puma on 3000
- jemalloc is enabled to reduce memory fragmentation

DOCKERFILE (ABRIDGED)

```
ARG RUBY_VERSION=3.4.7
FROM docker.io/library/ruby:$RUBY_VERSION-slim AS base
WORKDIR /rails
ENV RAILS_ENV="production" BUNDLE_DEPLOYMENT="1" \
    BUNDLE_PATH="/usr/local/bundle" BUNDLE_WITHOUT="development"
RUN apt-get update -qq && apt-get install --no-install-recommends -y \
    curl libjemalloc2 libvips sqlite3 && rm -rf /var/lib/apt/lists/*

FROM base AS build
RUN apt-get update -qq && apt-get install --no-install-recommends -y build-essential git
COPY Gemfile Gemfile.lock ./
RUN bundle install && bundle exec bootsnap precompile --gemfile
COPY . .
RUN SECRET_KEY_BASE_DUMMY=1 ./bin/rails assets:precompile

FROM base
COPY --from=build "${BUNDLE_PATH}" "${BUNDLE_PATH}"
COPY --from=build /rails /rails
RUN groupadd --system --gid 1000 rails && \
    useradd rails --uid 1000 --gid 1000 --create-home --shell /bin/bash && \
    chown -R rails:rails db log storage tmp
USER 1000:1000
ENTRYPOINT ["/rails/bin/docker-entrypoint"]
EXPOSE 80
CMD ["./bin/thrust", "./bin/rails", "server"]
```

CODE EXPLAINED

- → `AS build` — compilers and git stay out of the final image
- → `SECRET_KEY_BASE_DUMMY=1` — precompile assets without real secrets
- → `USER 1000` — don't run as root
- → `bin/thrust` — Thruster wraps the Rails server
- → `-v blog_storage:` — persist SQLite and uploads

BUILD & RUN LOCALLY

```
docker build -t blog .
docker run -d -p 80:80 -e RAILS_MASTER_KEY=$(cat config/master.key) \
  -v blog_storage:/rails/storage --name blog blog
```

THE RAILS WAY

- Hand-written Dockerfiles from blog posts → start from the generated one
- Nginx just for gzip and asset caching → Thruster
- Running migrations by hand after deploy → the entrypoint's `db:prepare`

THRUSTER FEATURES

FEATURE	DETAIL
HTTP/2	Multiplexed asset loading
Asset caching	Long cache headers, in-memory cache
Compression	gzip responses
X-Sendfile	Efficient file downloads
TLS	Let's Encrypt when TLS_DOMAIN is set

ENVIRONMENT VARIABLES

VARIABLE	PURPOSE
RAILS_MASTER_KEY	Decrypt credentials
RAILS_MAX_THREADS	Puma threads
WEB_CONCURRENCY	Puma workers
SOLID_QUEUE_IN_PUMA	Run jobs in web container
TLS_DOMAIN	Thruster TLS

⚠ COMMON MISTAKES

- Baking `master.key` into the image; pass it at runtime
- Forgetting the storage volume, so the SQLite database resets on each container replace
- Building on Apple Silicon for amd64 servers without `--platform linux/amd64`

✓ TIPS & BEST PRACTICES

- Keep `.dockerignore` excluding `log/`, `tmp/`, `storage/` and `.env*`
- Bump `RUBY_VERSION` in the Dockerfile when you change `.ruby-version`
- Use Kamal (sheet 77) instead of raw `docker run` for real deploys

QUICK SUMMARY

- multi-stage Dockerfile — slim image
- Thruster — HTTP/2, caching, TLS
- entrypoint — `db:prepare`
- volume — persist `storage/`

Deploying with Kamal

ZERO-DOWNTIME DEPLOYS TO YOUR OWN SERVERS

Kamal (included in new apps) deploys the Docker image to any Linux server over SSH. **kamal-proxy** switches traffic to the new container only after its health check passes, giving zero-downtime deploys with automatic Let's Encrypt TLS. Accessories run supporting services such as a database. Rails 8.1's generated config can deploy without a remote image registry.

NOTES

- Servers need only SSH access and Docker (Kamal installs Docker if missing)
- Secrets are read from `.kamal/secrets`, which pulls from ENV or password managers
- Each deploy builds, pushes, pulls, boots, health-checks `/up`, then switches traffic
- Roles let one image run web and job processes on different hosts

CONFIG/DEPLOY.YML

```yaml
service: blog
image: bob/blog

servers:
  web:
    - 203.0.113.10
  job:
    hosts:
      - 203.0.113.10
    cmd: bin/jobs

proxy:
  ssl: true
  host: blog.example.com

registry:
  username: bob
  password:
    - KAMAL_REGISTRY_PASSWORD

env:
  secret:
    - RAILS_MASTER_KEY        # .kamal/secrets: RAILS_MASTER_KEY=$(cat config/master.key)

volumes:
  - "blog_storage:/rails/storage"
asset_path: /rails/public/assets
builder:
  arch: amd64
```

CODE EXPLAINED

- → `servers` roles — `web` behind the proxy, `job` runs `bin/jobs`
- → `proxy.ssl` + `host` — automatic Let's Encrypt certificate
- → `env.secret` — names only; values come from `.kamal/secrets`
- → `asset_path` — bridges old and new assets during deploys
- → `builder.arch` — build for the server's CPU

THE RAILS WAY

- Heroku-style PaaS for a single app → Kamal on a VPS you control
- Copying files with scp and restarting → `bin/kamal deploy`
- Secrets in `deploy.yml` → `.kamal/secrets` referencing ENV or a password manager

COMMANDS

```
bin/kamal setup              # first time: install Docker, proxy, deploy
bin/kamal deploy             # build + zero-downtime switch
bin/kamal app logs -f        # tail logs
bin/kamal accessory boot db  # start an accessory
```

DEPLOY.YML KEYS

KEY	PURPOSE
`servers`	Hosts per role (web, job)
`proxy`	kamal-proxy host, SSL, health check
`accessories`	Extra services, e.g. `postgres:17`
`aliases`	`console`, `shell`, `logs` shortcuts

⚠ COMMON MISTAKES

- Pointing DNS at the server after `kamal setup`; Let's Encrypt needs DNS first
- Committing real secret values in `.kamal/secrets` instead of references
- Forgetting volumes, so SQLite data lives only inside a replaced container

✓ TIPS & BEST PRACTICES

- Run `bin/kamal config` to see the merged configuration before deploying
- Use `bin/kamal lock` features to avoid two people deploying at once
- Add `bin/kamal deploy` to a CI job on main after `bin/ci` passes

QUICK SUMMARY

- `deploy.yml` — servers, proxy, env
- `kamal setup` / `deploy` — ship it
- kamal-proxy — zero downtime + TLS
- `.kamal/secrets` — secret references

Upgrading Rails

STAYING CURRENT

Upgrade one minor version at a time (7.2 → 8.0 → 8.1). Fix deprecation warnings first, bump the gem, run `bin/rails app:update` to merge config changes, then flip new defaults one by one using the generated `new_framework_defaults` file. Each minor release gets bug fixes for about a year and security fixes for about two.

NOTES

- Read the version's "Upgrading Ruby on Rails" guide section and release notes
- Deprecations in version N become errors in N+1
- `app:update` asks before overwriting each file; review the diff
- Keep `load_defaults` at the old version until every new default is enabled

THE STEPS

```
# 1. green test suite and zero deprecation warnings on the current version
bin/rails test 2>&1 | grep -i deprecat

# 2. bump Rails
bundle update rails --conservative       # Gemfile: gem "rails", "~> 8.1.0"

# 3. update config files and binstubs
bin/rails app:update                     # interactive: diff / overwrite / skip

# 4. run tests, fix failures, deploy
bin/ci
```

CODE EXPLAINED

- → `--conservative` — update Rails without unrelated gems
- → `app:update` — new config, binstubs, initializers
- → Framework defaults file — opt in to behavior changes gradually
- → `load_defaults 8.1` — final switch once each default is on
- → `deprecation = :raise` — make warnings fail tests

CONFIG/INITIALIZERS/NEW_FRAMEWORK_DEFAULTS_8_1.RB

```
# Uncomment each line, run tests, deploy, repeat.
# Rails.configuration.action_controller.escape_json_responses = false
# Rails.configuration.active_support.escape_js_separators_in_json = false
# Rails.configuration.active_record.raise_on_missing_required_finder_order_columns = true
# Rails.configuration.action_controller.action_on_path_relative_redirect = :raise
# Rails.configuration.action_view.render_tracker = :ruby
```

THEN FINISH

```
# config/application.rb
config.load_defaults 8.1          # after all new defaults are enabled
# delete new_framework_defaults_8_1.rb
```

THE RAILS WAY

- Jumping from 6.1 straight to 8.1 → one minor at a time, deploying between
- Bumping `load_defaults` with the gem → flip defaults individually first
- Ignoring deprecation output → treat warnings as the next upgrade's to-do list

SURFACING DEPRECATIONS

```
# config/environments/test.rb
config.active_support.deprecation = :raise
```

UPGRADE TOOLS

TOOL	USE
`next_rails (gem)`	Dual boot: run tests on next Rails
`bundle outdated`	Find gems blocking the upgrade
`railsdiff.org`	Compare generated apps across versions
`rails-new output`	Diff a fresh app for config ideas
`app:update`	Official config merge

MAINTENANCE POLICY

SUPPORT	DURATION PER MINOR
`New features`	Only in the next minor
`Bug fixes`	About 1 year
`Security fixes`	About 2 years
`Ruby version`	Check the minimum per release

⚠ COMMON MISTAKES

- Overwriting customized config files in `app:update` without reading the diff
- Upgrading Rails and Ruby in the same deploy; change one variable at a time
- Leaving the framework defaults file around for years, half applied

✓ TIPS & BEST PRACTICES

- Upgrade Ruby first (separately), then Rails
- Use railsdiff.org to see exactly what a fresh app changed between versions
- Schedule upgrades every minor release; small jumps are cheap

QUICK SUMMARY

- one minor at a time
- `app:update` — merge config
- framework defaults file — opt in gradually
- `load_defaults` — finish the upgrade